



Progressive Education Society's
Modern College of Engineering, Pune - 411005
Department of Electronics & Computer

AY-2024-25 Semester - I

Microcontroller and Applications

Course In-charge
Mrs. R. S. Chaudhari

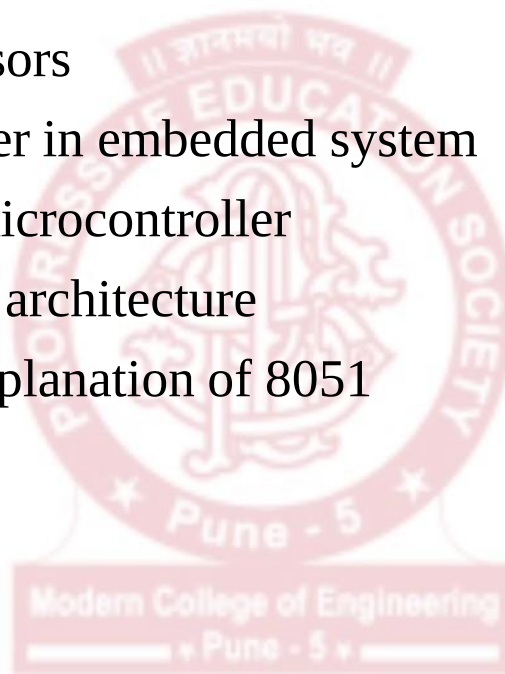
Unit I

Introduction to Microcontroller Architecture



Syllabus

- Microprocessor and microcontroller comparison
- Harvard & Von Neumann architecture
- RISC & CISC processors
- Role of microcontroller in embedded system
- Selection criteria of microcontroller
- Overview of MCS-51 architecture
- Block diagram and explanation of 8051
- Port structure
- Memory organization
- Interrupt structure
- Timers and its modes
- Serial communication modes
- Overview of Instruction set
- Programming of Timer 0&1



Introduction to Microcontroller Architecture

It is a Multifunctional, Reprogrammable logic device

Without memory and I/O---- Its Microprocessor

Single chip Microcomputer ---- **Microcontroller**

- **Hardware :Interface to the real world**
- **Software: order how to deal with inputs**

Data Handling Capacity- Data Bus

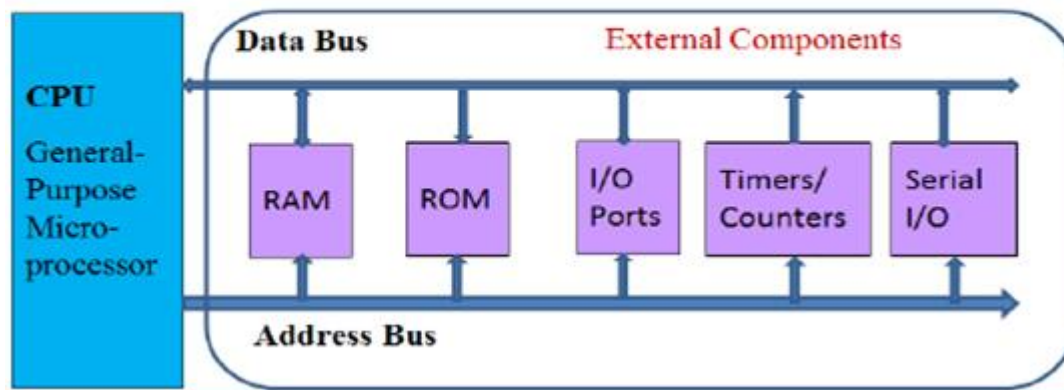
Speed Of Operation- Clock

Storage Capacity- Memory

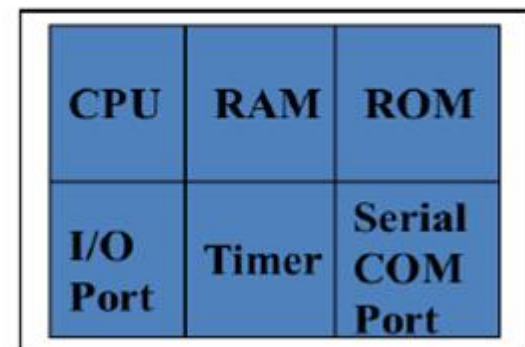
Interfacing- I/O Ports

1,0 = Bits
4 bits = Nibble
8 bits = Byte
16 bits = Word
32 bits = Double

Microprocessor Features- Supporting SFRs



Single Chip-MC



Ref: T1: Page No 2

Microprocessor & Microcontroller Comparison

S. N	Parameter	Microprocessor	Microcontroller
1	Data bus width	4,8,16,32,64,and 128 bits	8,16 and 32 bits
2	Memory addressing capacity	Up to Tera Byte (TB) and can use virtual memory also	Up to 4 MB and each has fixed RAM and ROM
3	Multifunction Pins	Number of Multifunction Pins less	More number of Multifunction pins
4	Interfacing	Need external chip for interfacing I/O devices	Has in-built ports for interfacing of I/O devices.
5	Architecture	Normally CISC (single data and address bus for data and code memory)	May be CISC and RISC (separate data and address bus for data and code memory)
6	Peripherals	CPU is stand-alone, (RAM, ROM, I/O, timer are separate and connected externally)	CPU, RAM, ROM, I/O and timer are integral part of single chip
7	Instruction set	Focus on processing-intensive operations (instructions operate on nibbles, bytes, words, or even double words)	Instruction sets focus on control and bit-level operations (instructions to set and clear individual bits and input/output.)
8	Access time	High	Low

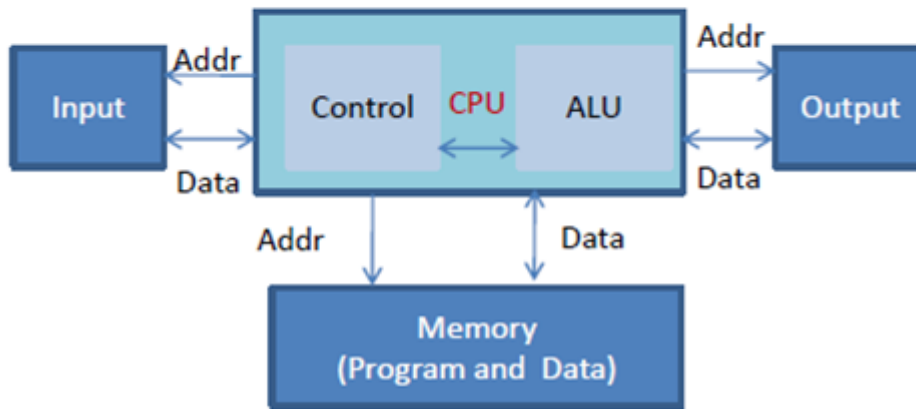
Microprocessor & Microcontroller Comparison

9	Cost	High - Multifunctional	Low-single-purpose (Control-oriented)
10	Software development	Has Memory Management Unit (MMU) and Virtual Memory (VM)	Does not have Memory Management Unit (MMU) and Virtual Memory (VM)
11	Examples	4004,8085,8086- Pentium	INTEL 8051, 80960, PIC16F877 etc.
12	Applications	Data base management, Control applications, Personal Computers (PC), Instrumentation	In hand held devices such as digital camera, mobile phones, washing machine etc.
13	Power Consumption	Due to external components, the entire power consumption is high. Hence it is not suitable to used with devices running on stored power like batteries	Since external components are low, total power consumption is less and can be used with devices running on stored power like batteries.

Harvard & Von Neumann Architecture

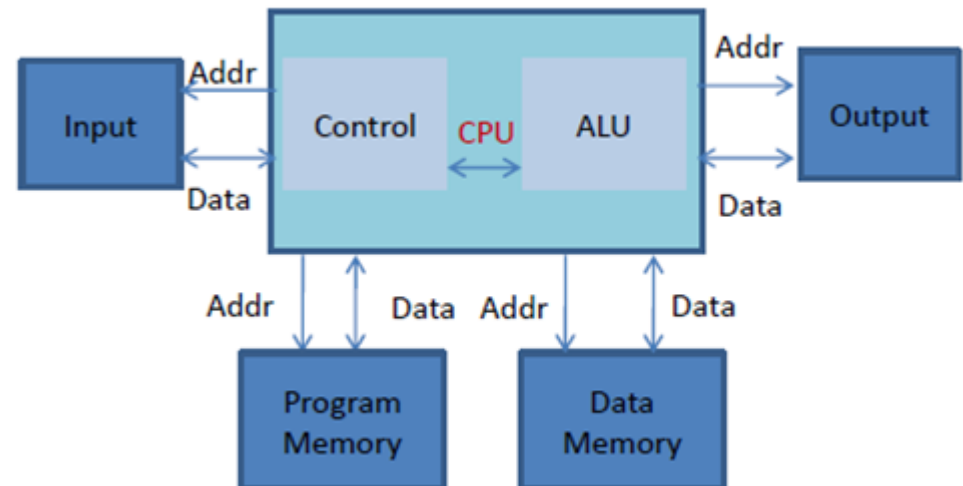
Von Neumann Architecture--- CISC

- Uses one memory spaces for program instructions and data
- Limits operating bandwidth
- Execution of data and code occurs in sequential fashion
- Reduced Speed of Operation



Harvard Architecture --- CISC

- Uses two separate memory spaces for program instructions and data
- Improved operating bandwidth
- Allows for different bus widths
- Improved Speed of Operation



Ref: T1: Page No 57-58

RISC & CISC processors

RISC processors	CISC processors
Reduced Instruction Set Computing Processing	Complex Instruction Set Computing Processing
RISC chips have fewer components and a smaller instruction set, allowing faster accessing of “common” instructions	CISC chips have an increasing number of components and an ever increasing instruction set and so are always slower and less powerful at executing “common” instructions
RISC chips execute an instruction in one machine cycle	CISC chips execute an instruction in two to ten machine cycles
RISC chips distribute some of their processing to other chips	CISC chips do all of the processing themselves
RISC chips are finding their way into components that need faster processing of a limited number of instructions, such as print	CISC chips are more common in computers that have a wider range of instructions to execute

System and Embedded system

What is a system?

A system is an arrangement of all its units assembled to work together according to a set of rules

Now what is an embedded system??

An embedded system is a microprocessor- or microcontroller-based system of hardware and software designed to perform dedicated tasks.

This embedded system can be an independent system like; keyboard, mouse etc. or a part of a large system, working independently.

Ex: A mobile phone

- The camera module with its lens, sensor and preprocessor constitute an Embedded System.
- The fingerprint sensor system
- The battery and its power management IC is an Embedded System
- WiFi module in phone.

Role of microcontroller in Embedded Systems

- An embedded system must have a microcontroller to process data, it must have a memory (ROM) to store program and it must have connectors to connect peripherals. Eg - A printer is an embedded system since the processor inside it performs only one task ; namely, getting the data and printing it.
- Pentium-based PC (or any x86 IBM-compatible PC) can be used for any number of applications such as word processor, print server, bank teller terminal, video game player, network server, or internet terminal.
- Software for a variety of applications can be loaded and run. Of course the reason a PC can perform myriad tasks is that it has RAM memory and an operating system that loads the application software into RAM and lets the CPU run it.
- In an embedded system, there is only one application software that is typically burned into ROM.
- An x86 PC contains or is connected to various embedded products such as the keyboard, printer, modem, disk controller, sound card, CD-ROM driver, mouse, and so on.
- Each one of these peripherals has a microcontroller inside it that performs only one task.
- For example, inside every mouse there is a microcontroller that performs the task of finding the mouse position and sending it to the PC.

Selection Criteria of Microcontroller

A. Meeting Computing Needs of Task at Hand efficiently and Cost Effectively

- 1.Data Handling Capacity- Bits, Nibble Bytes, Words, Double words, Quad Words etc
- 2.Speed --- Depends on Clock
- 3.Amount of RAM/ ROM/ EPROM/ Flash/ Static
- 4.Number of I/O pins, Timers – All SFRS
- 5.Power consumption – Based on the modes
- 6.Packaging – 40 PIN DIP,/ QFP/ other– Important – Space, assembly, Prototyping the end Product
- 7.Added features like ADC/ DAC/ CCP, Bus support like CAN, SPI, I2C, USB.
- 8.Watchdog timer, Timer modes, Data EEPROM etc.
- 9.Easy to upgrade --Higher Performances or low power operations

B. Availability of Software and Hardware Development Tools

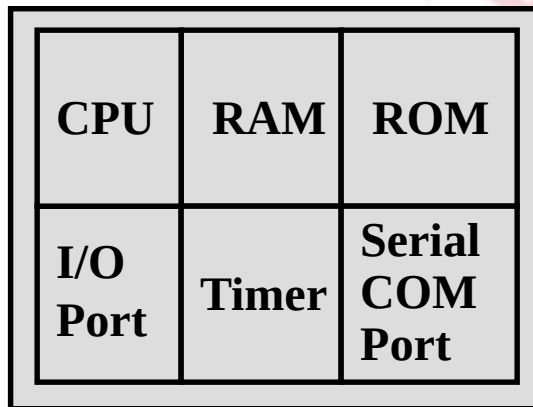
- 1.Assembler
- 2.Compilers – Code efficient C language
- 3.Debuggers
- 4.Emulators
- 5.Technical Support

C. Wide Availability and Reliable Sources of Microcontrollers

- 1.More important than previous two – Since most diversified suppliers are available to cell same product
- 2.Intel, Atmel, Infineton, Philips,Dallas and Microchip Technology has massive dedicated resources for supply
- 3.All the products are Stable, Matured and single sourced.

8051 Basic Components

- 4K bytes internal **ROM**
- 128 bytes internal **RAM**
- Four 8-bit **I/O ports** (P0 - P3).
- Two 16-bit **timers**/counters
- One **serial** interface



← A single chip
Microcontroller

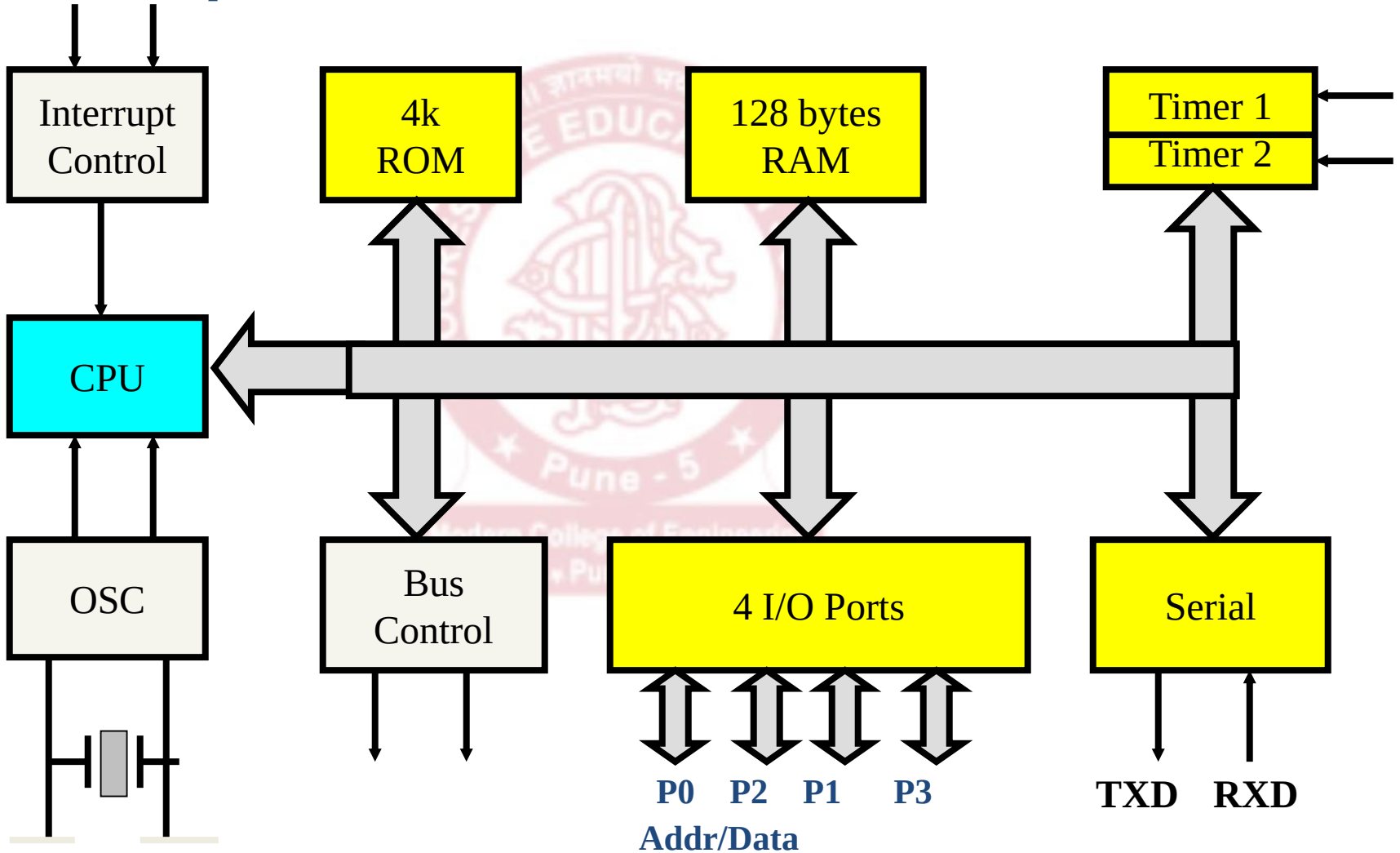


8051 features

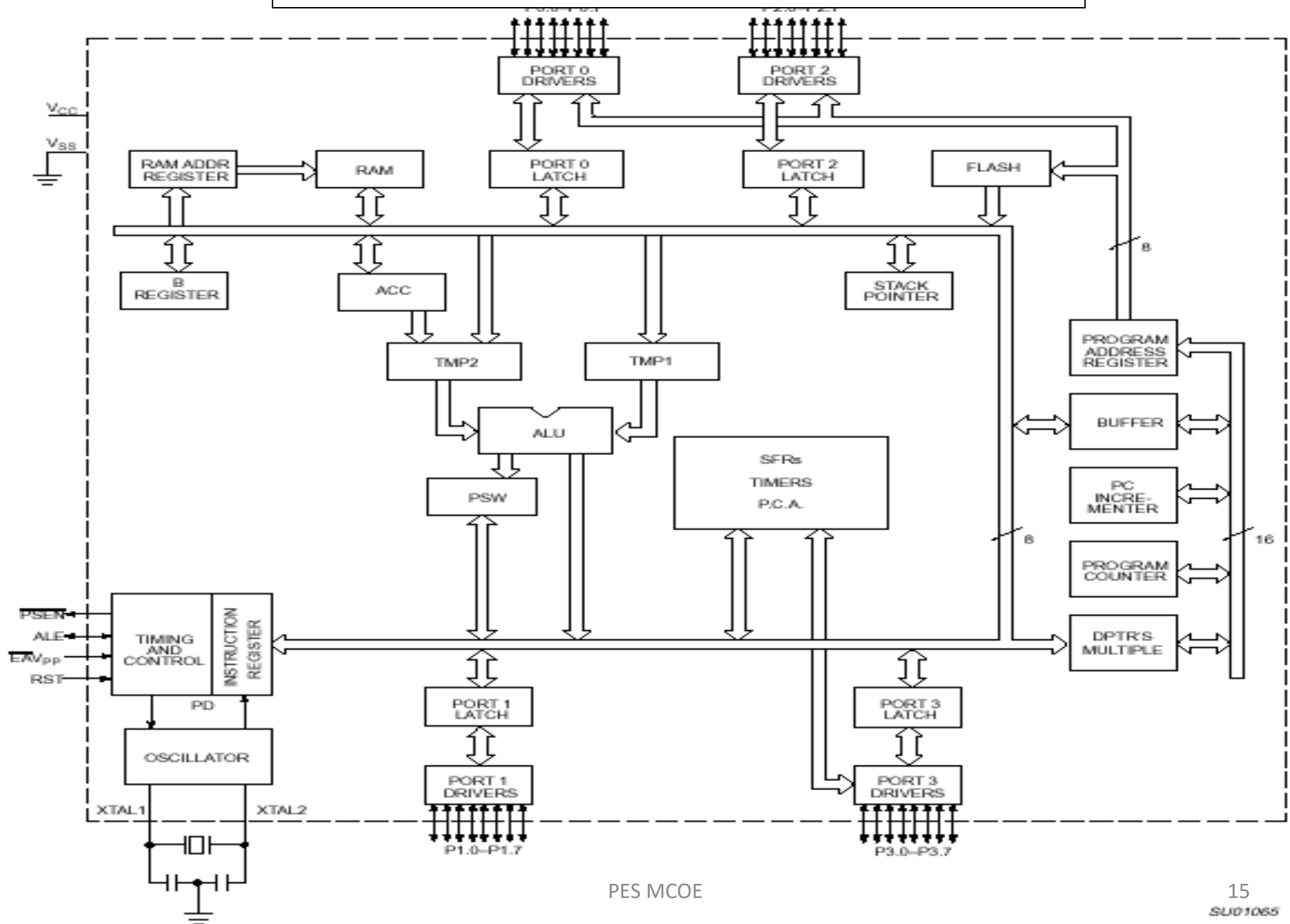
- 1 On-chip oscillator.
- External crystal oscillator connected to drive the on-chip oscillator.
- 6 interrupt sources (2 external , 3 internal, Reset)
- 64K external code (program) memory(only read) by PSEN
- 64K external data memory(can be read and write) by RD,WR
- Code memory is selectable by EA (internal or external)
- We may have External memory as data and code

Block Diagram of 8051

External Interrupts

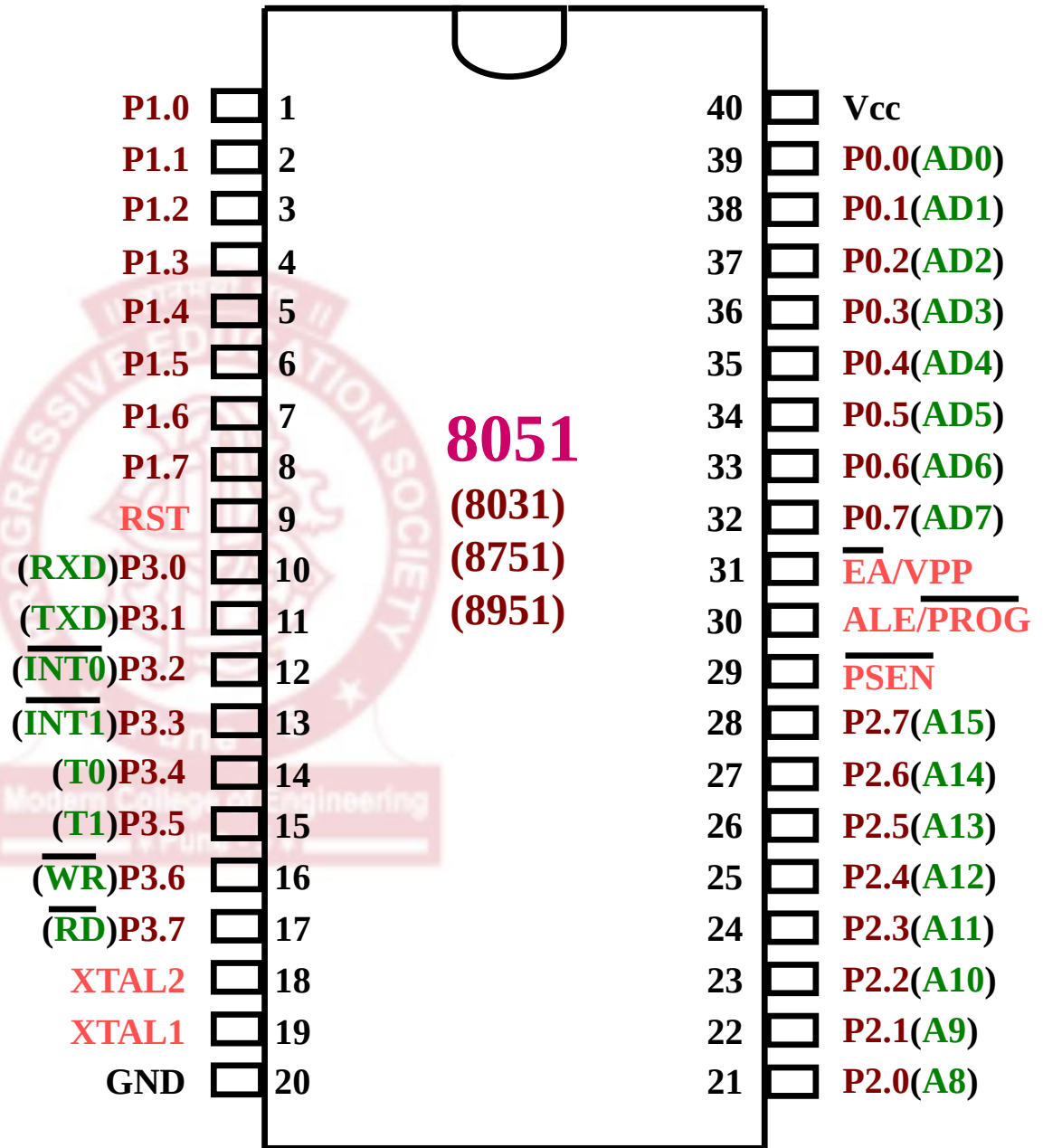


8051 Internal Block Diagram

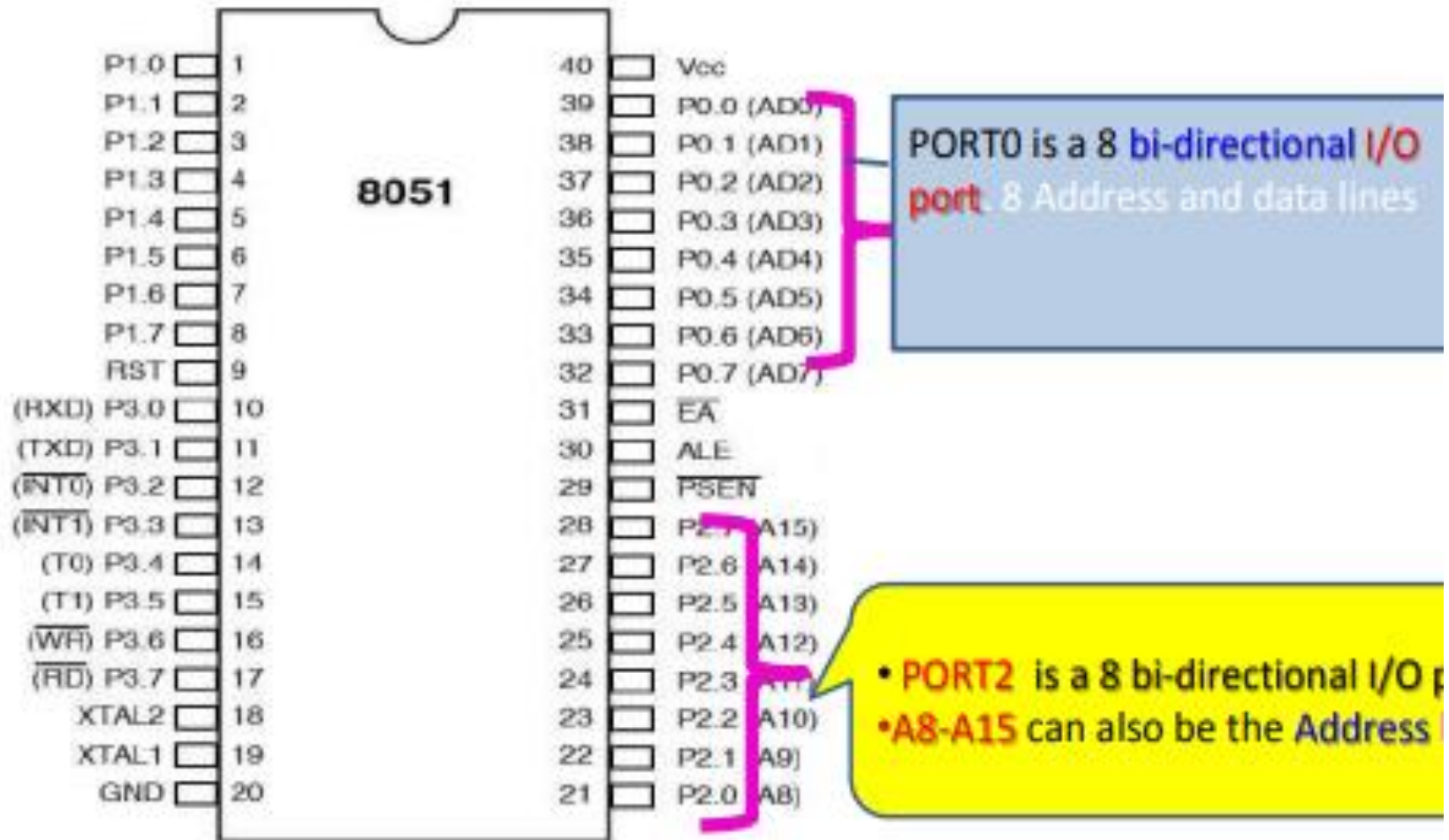


8051

Pin out



8051 Ports

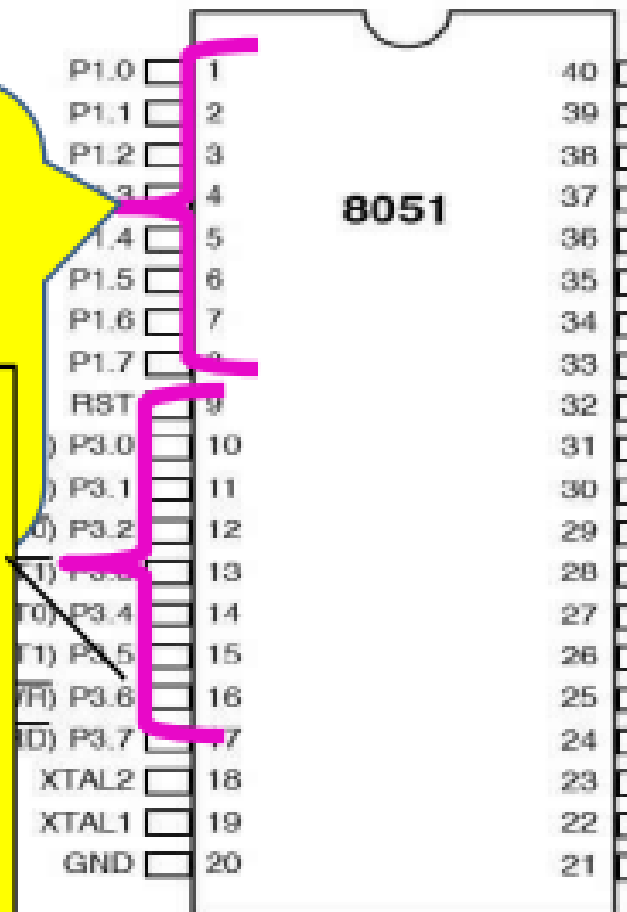


8051 Ports

- **PORT 1** is a bi-directional I/O port.

- **P1.0**
- **P1.1**
- **P1.2**
- **P1.3**

P3.0 / RXD can also be the Receive data
P3.1/TXD can also be used Transmit
P3.2 / INT0 can also be used for external INT0.
P3.3 / INT1 can also be used for external INT0.
P3.4 / T0 can be used as Inputs to timer 0
P3.5 / T1 can be used as Inputs to timer 1
P3.6 / WR can be used as write data to ext memory
P3.7/ RD can be used as reading data from ext memory



Important Pins (IO Ports)

- **One of the most useful features of the 8051 is that it contains four I/O ports (P0 - P3)**
- Port 0 (pins 32-39) : P0 (P0.0~P0.7)
 - 8-bit R/W - General Purpose I/O
 - Or acts as a multiplexed low byte address and data bus for external memory design
- Port 1 (pins 1-8) : P1 (P1.0~P1.7)
 - Only 8-bit R/W - General Purpose I/O
- Port 2 (pins 21-28) : P2 (P2.0~P2.7)
 - 8-bit R/W - General Purpose I/O
 - Or high byte of the address bus for external memory design
- Port 3 (pins 10-17) : P3 (P3.0~P3.7)
 - General Purpose I/O
 - if not using any of the internal peripherals (timers) or external interrupts.
- **Each port can be used as input or output (bi-direction)**

Important Pins

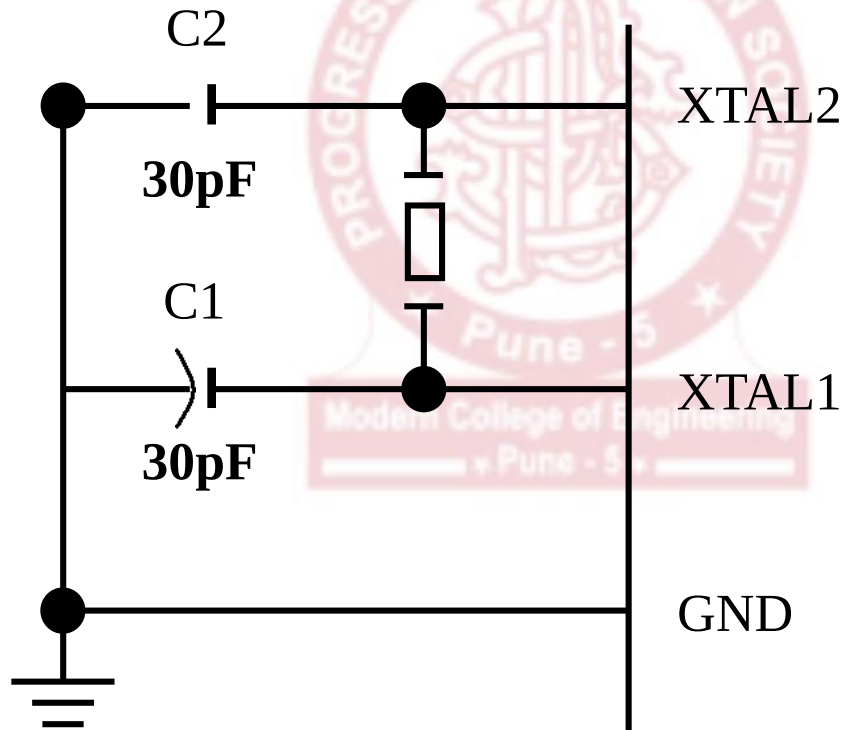
- **PSEN** (out): **P**rogram **S**tore **E**nable, the read signal for external program memory (active low).
- **ALE** (out): **A**ddress **L**atch **E**nable, to latch address outputs at Port0 and Port2
- **EA** (in): **E**xternal **A**ccess Enable, active low to access external program memory locations 0 to 4K
- **RXD,TXD**: UART pins for serial I/O on Port 3
- **XTAL1 & XTAL2**: Crystal inputs for internal oscillator

Pins of 8051

- Vcc (pin 40) :
 - Vcc provides supply voltage to the chip.
 - The voltage source is +5V.
- GND (pin 20) : ground
- XTAL1 and XTAL2 (pins 19,18) :
 - These 2 pins provide external clock.
 - using a quartz crystal oscillator
 - Example 4-1 shows the relationship between XTAL and the machine cycle.

XTAL Connection to 8051

- Using a quartz crystal oscillator
- We can observe the frequency on the XTAL2 pin.



Reset in 8051

- RST (pin 9) : Reset
 - input pin and active high (normally low) .
 - The high pulse must be high at least 2 machine cycles.
 - power-on reset.
 - Upon applying a high pulse to RST, the microcontroller will reset and all values in registers will be lost.
 - Reset values of some 8051 registers
 - power-on reset circuit

Register	Reset Value
PC	0000
ACC	00
B	00
PSW	00
SP	07
DPTR	0000
RAM are all zero.	

31

1F

30 pF

Modern College of Engineering

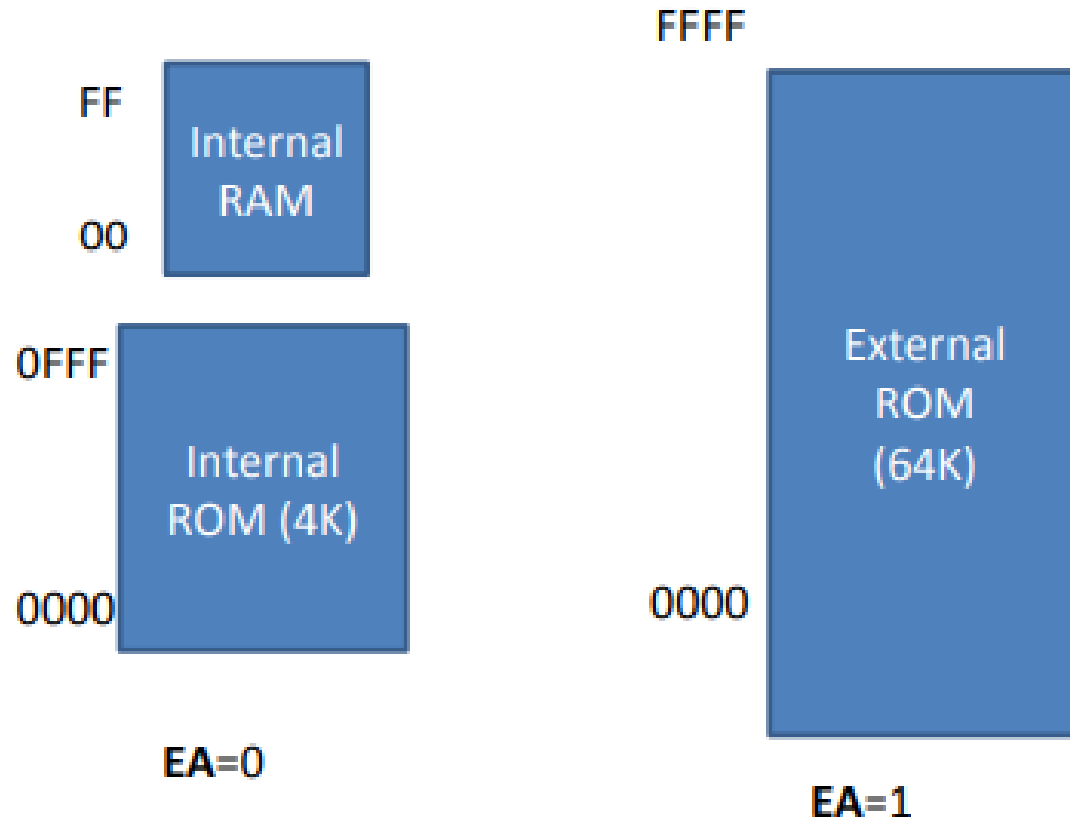
9

+ Pune - 5 +

External Access and PSEN

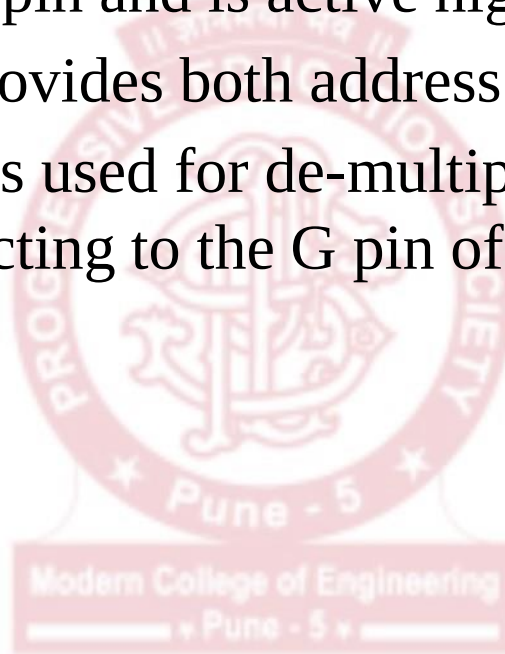
- /EA (pin 31) : external access
 - There is no on-chip ROM in 8031 and 8032.
 - The /EA pin is connected to GND to indicate the code is stored externally.
 - /PSEN & ALE are used for external ROM.
 - For 8051, /EA pin is connected to Vcc.
 - “/” means active low.
- /PSEN (pin 29) : program store enable
 - This is an output pin and is connected to the OE pin of the ROM.
 - See Chapter 14 of Mazidi

External Access - External ROM selection of 8051



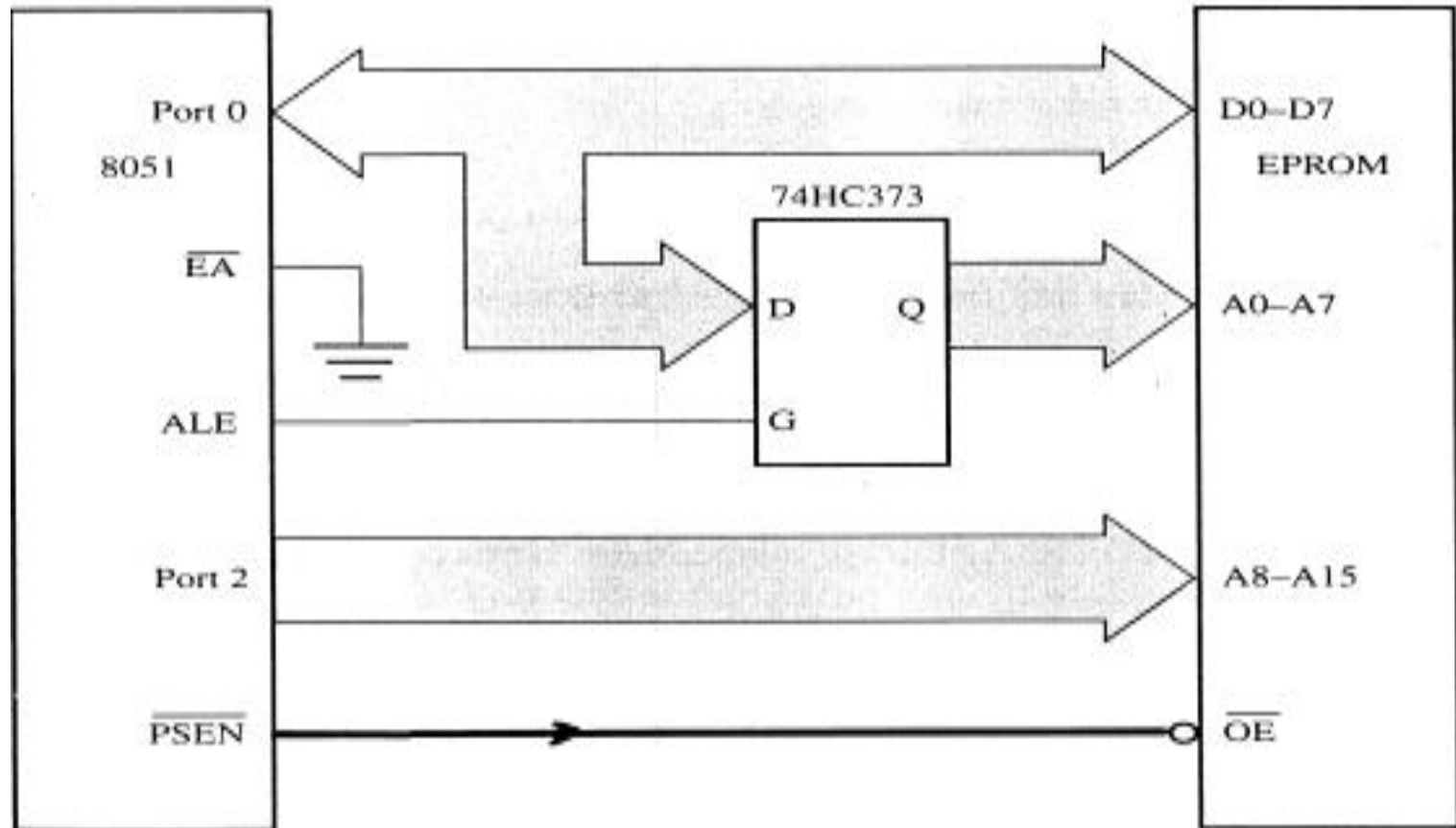
Pins of 8051

- ALE (pin 30) : address latch enable
 - It is an output pin and is active high.
 - 8051 port 0 provides both address and data.
 - The ALE pin is used for de-multiplexing the address and data by connecting to the G pin of the 74LS373 latch.

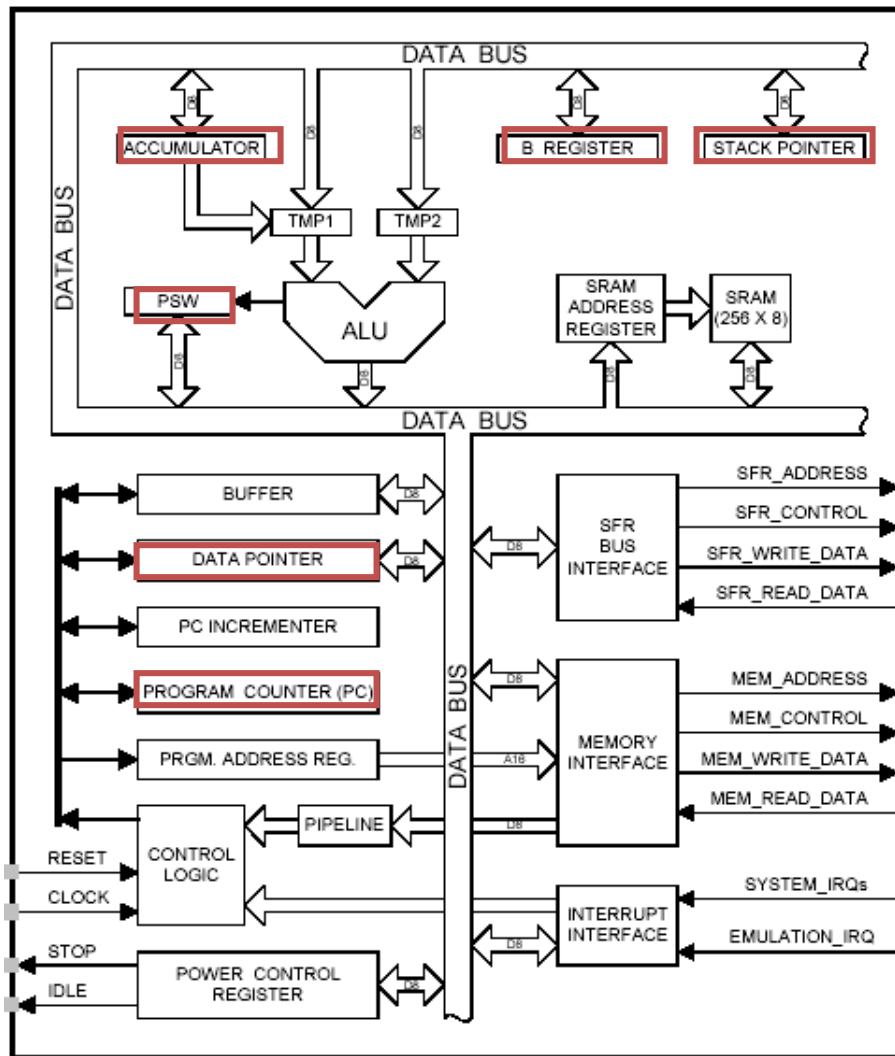


Address Multiplexing for External Memory

Figure
Accessing
external
code
memory



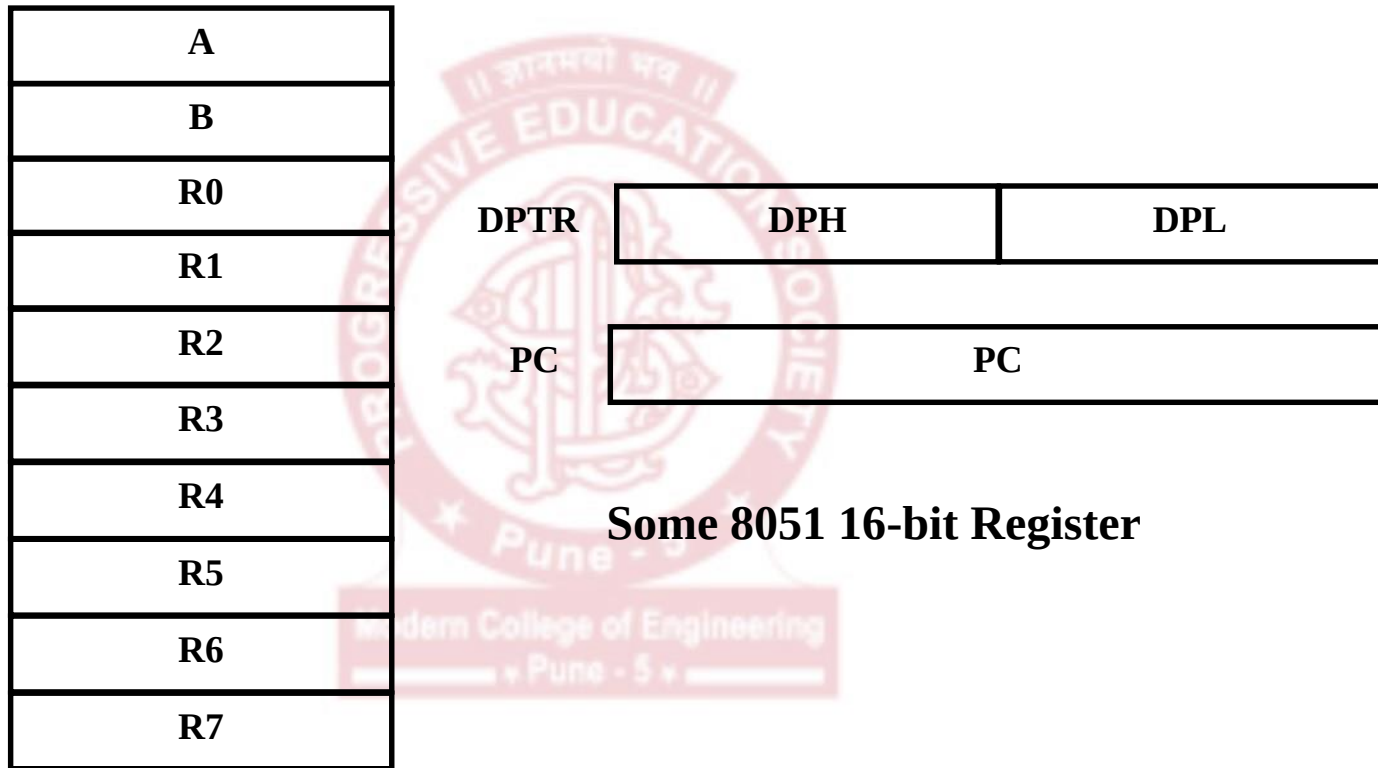
8051 CPU Registers



- A (Accumulator)
- B
- PSW (Program Status Word)
- SP (Stack Pointer)
- PC (Program Counter)
- DPTR (Data Pointer)

Used in assembler instructions

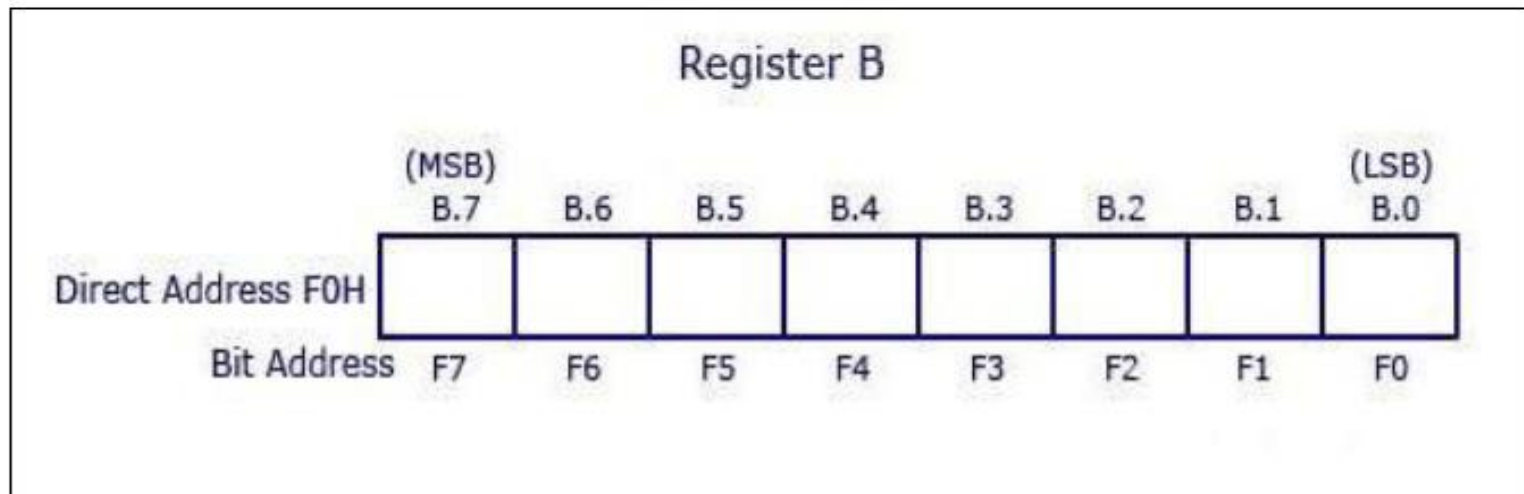
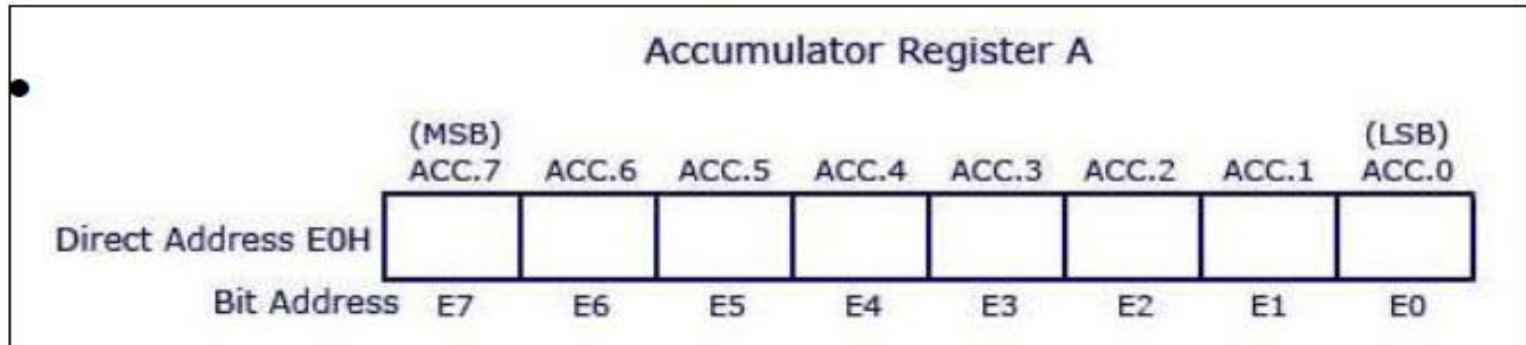
Registers



Some 8051 16-bit Register

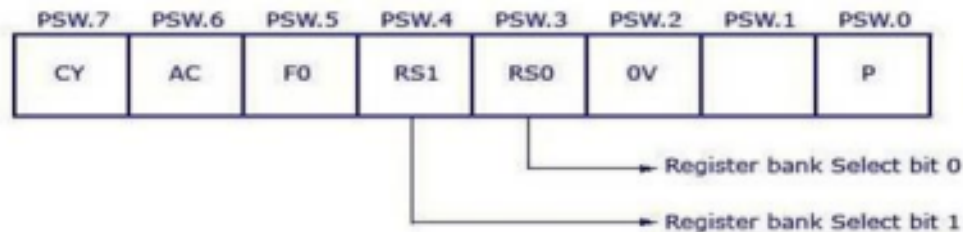
**Some 8-bit Registers of
the 8051**

A and B Registers



Program Status Register (PSW)

It is an 8-bit register referred as flag registers and all are conditional



RS1	RS0	Register Bank
0	0	0
0	1	1
1	0	2
1	1	3

- Parity Flag

Set P=1 if ACC numbers are odd after operation otherwise P=0 for even

- OV (Overflow Flag)

- Set when result of an operation of signed numbers goes beyond 7-bits- used to represent the errors { carry from B6 to B7 but no carry out of B7[C=0];Carry out from from B7[C=1]}

- AC (Auxiliary Carry Flag)

- Set when carry generated from B3 to B4 in an BCD arithmetic operation (Addition or Subtraction)

- CY(Carry Flag)

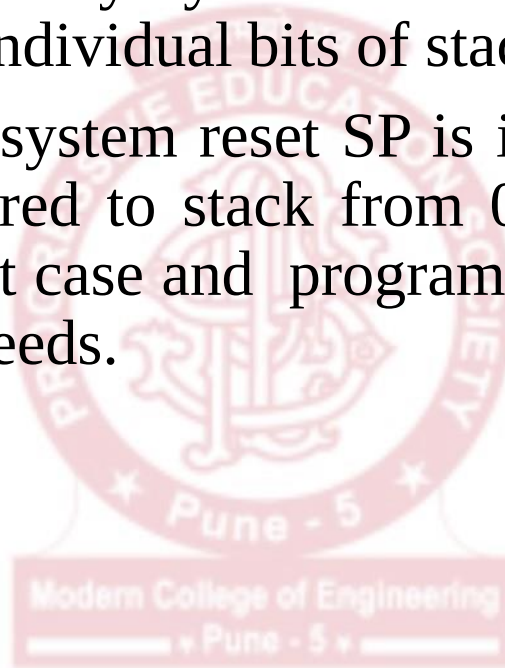
- Set when an addition or subtraction generates a carry out of D7 Bits (unsigned Nos.)

- F0(Flag 0)

- User defined flag

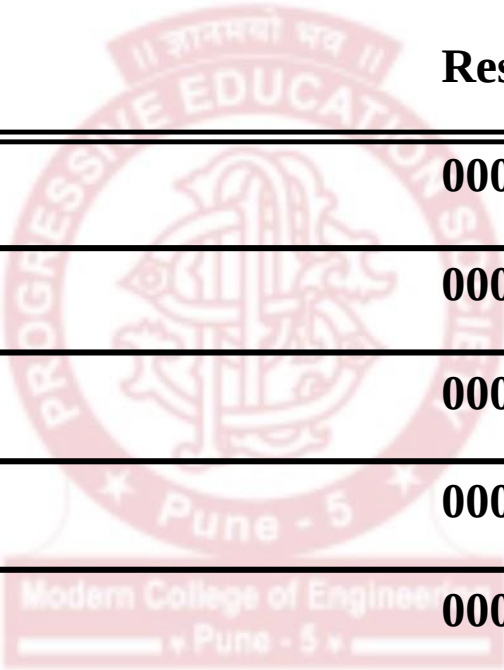
Stack Pointer (SP)

- Stack pointer is an 8 bit register, the direct address of SP is 81H and it is only byte addressable, which means you cannot access individual bits of stack pointer.
- Usually after a system reset SP is initialized as 07H and data can be stored to stack from 08H onwards. This is usually a default case and programmer can alter values of SP to suit his needs.



Status of 8051 Registers after RESET

After reset the contents in the registers is as below



Register	Reset Value
PC	0000
ACC	0000
B	0000
PSW	0000
SP	0007
DPTR	0000

RAM are **all zero**

RAM - Data Memory in 8051

Data RAM is 256 bytes – 128 bytes + 128 bytes (SFR)

RAM

Byte address	Bit address							
27	3F	3E	3D	3C	3B	3A	39	38
26	37	36	35	34	33	32	31	30
25	2F	2E	2D	2C	2B	2A	29	28
24	27	26	25	24	23	22	21	20
23	1F	1E	1D	1C	1B	1A	19	18
22	17	16	15	14	13	12	11	10
21	0F	0E	0D	0C	0B	0A	09	08
20	07	06	05	04	03	02	01	00
1F	Bank 3							
18								
17	Bank 2							
10								
0F	Bank 1							
08								
07	Default register bank for R0–R7							
00								

Bit-addressable locations

Byte address	Bit address							
7F	General purpose RAM							
30								
2F								
2E								
2D								
2C								
2B								
2A								
29								
28								
	7F	7E	7D	7C	7B	7A	79	78
	77	76	75	74	73	72	71	70
	6F	6E	6D	6C	6B	6A	69	68
	67	66	65	64	63	62	61	60
	5F	5E	5D	5C	5B	5A	59	58
	57	56	55	54	53	52	51	50
	4F	4E	4D	4C	4B	4A	49	48
	47	46	45	44	43	42	41	40

Bit-addressable locations

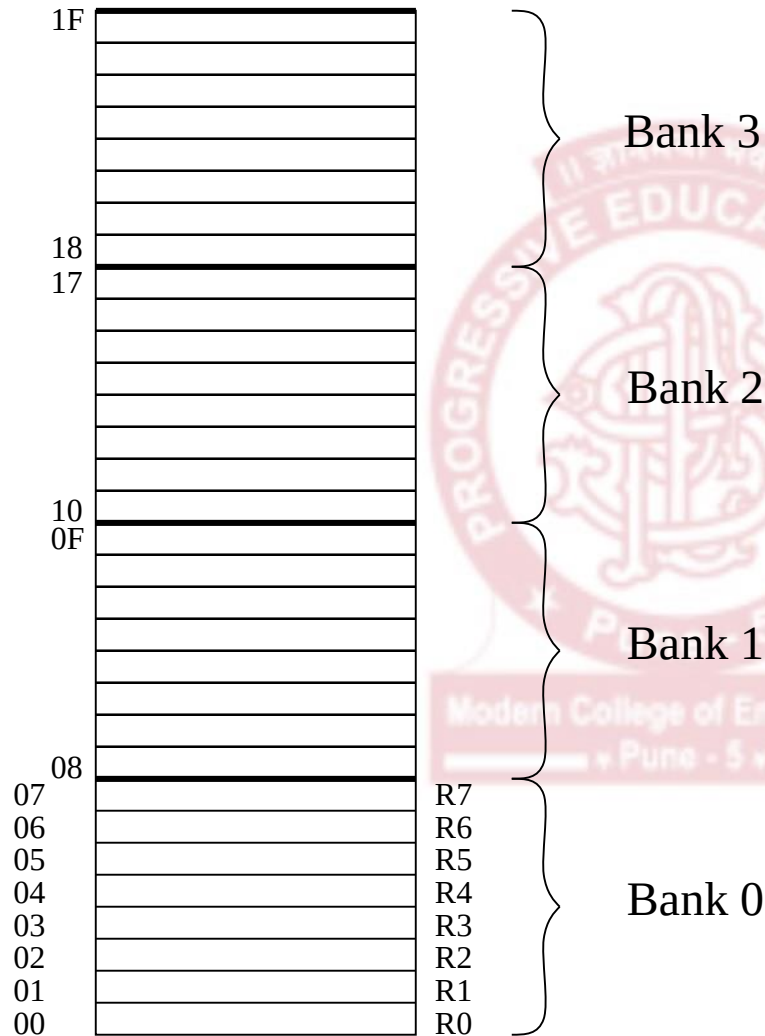
Bank 3

Bank 2

Bank 1

Modern College of Engineering
Pune - 5

0xFF	Upper 128 RAM (Indirect Addressing Only)	Special Function Register's (Direct Addressing Only)
0x80 0x7F		
	(Direct and Indirect Addressing)	Lower 128 RAM (Direct and Indirect Addressing)
0x30 0x2F	Bit Addressable	
0x20 0x1F	General Purpose Registers	
0x00		



RAM – Register Banks

Register bank 0		Register bank 1		Register bank 2		Register bank 3	
00	R0	08	R0	10	R0	18	R0
01	R1	09	R1	11	R1	19	R1
02	R2	0A	R2	12	R2	1A	R2
03	R3	0B	R3	13	R3	1B	R3
04	R4	0C	R4	14	R4	1C	R4
05	R5	0D	R5	15	R5	1D	R5
06	R6	0E	R6	16	R6	1E	R6
07	R7	0F	R7	17	R7	1F	R7

RAM - Bit Addressable Memory

2F	7F							78
2E								
2D								
2C								
2B								
2A								
29								
28								
27								
26								
25								
24								
23							1A	
22								10
21	0F							08
20	07	06	05	04	03	02	01	00

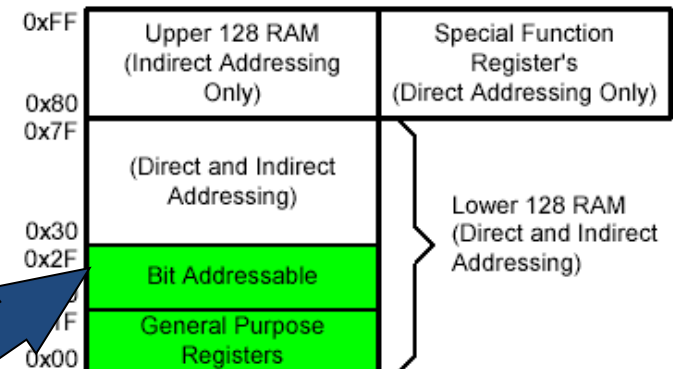
20h – 2Fh (16 locations X 8-bits
= 128 bits)

Bit addressing:

mov C, 1Ah

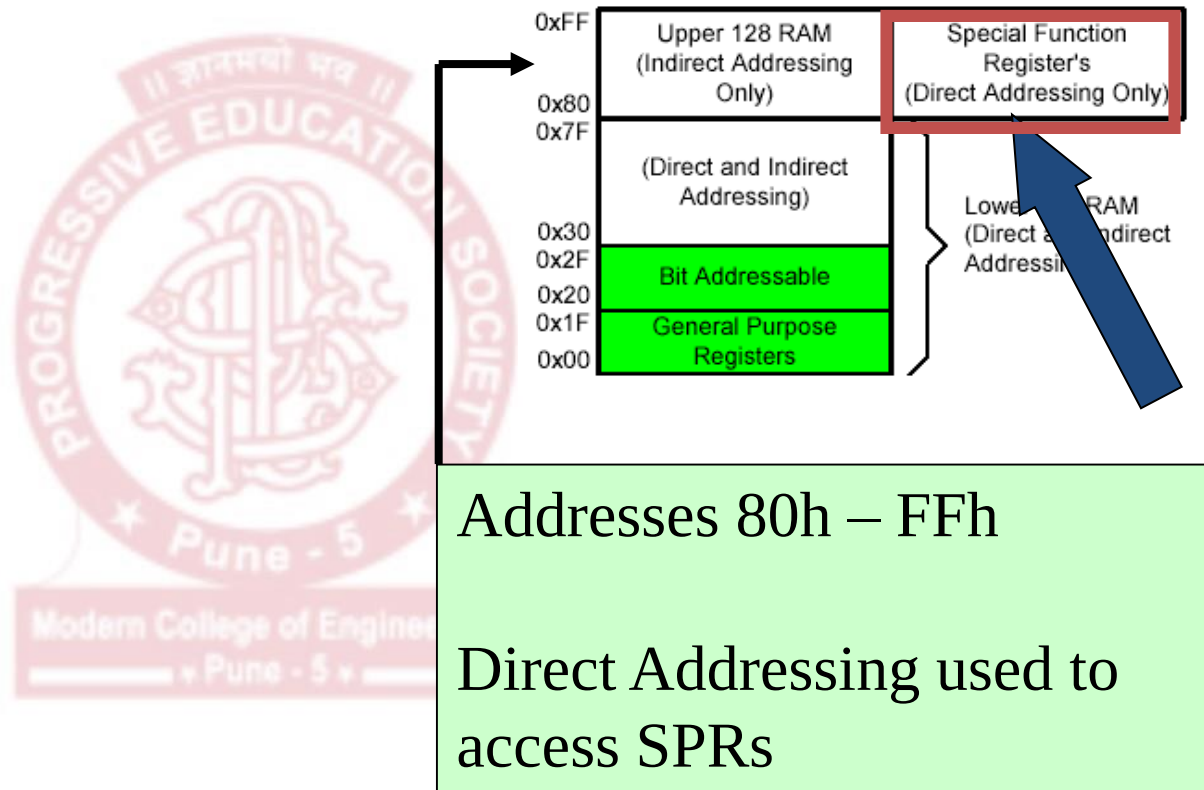
or

mov C, 23h.2



RAM - Special Function Registers (SFR)

- ❑ DATA registers
- ❑ CONTROL registers
 - ❖ Timers
 - ❖ Serial ports
 - ❖ Interrupt system



SFR MEMORY MAP

8 Bytes

F8								FF
F0	B							F7
E8								EF
E0	ACC							E7
D8								DF
D0	PSW							D7
C8	T2CON		RCAP2L	RCAP2H	TL2	TH2		CF
C0								C7
B8	IP							BF
B0	P3							B7
A8	IE							AF
A0	P2							A7
98	SCON	SBUF						9F
90	P1							97
88	TCON	TMOD	TL0	TL1	TH0	TH1		8F
80	P0	SP	DPL	DPH			PCON	87

↑
Bit
Addressable

Figure 5

Bit Addressable RAM - SFR

Figure
Summary
of the
8051 on-
chip data
memory
(Special
Function
Registers)

Byte address	Bit address									Byte address	Bit address								
98	9F	9E	9D	9C	9B	9A	99	98	SCON	FF									
										F0	F7	F6	F5	F4	F3	F2	F1	F0	B
90	97	96	95	94	93	92	91	90	P1										
										E0	E7	E6	E5	E4	E3	E2	E1	E0	ACC
8D	not bit addressable								TH1										
8C	not bit addressable								TH0	D0	D7	D6	D5	D4	D3	D2	-	D0	PSW
8B	not bit addressable								TL1										
8A	not bit addressable								TL0	B8	-	-	-	BC	BB	BA	B9	B8	IP
89	not bit addressable								TMOD										
88	8F	8E	8D	8C	8B	8A	89	88	TCON	B0	B7	B6	B5	B4	B3	B2	B1	B0	P3
87	not bit addressable								PCON										
										A8	AF	-	-	AC	AB	AA	A9	A8	IE
83	not bit addressable								DPH										
82	not bit addressable								DPL	A0	A7	A6	A5	A4	A3	A2	A1	A0	P2
81	not bit addressable								SP										
80	87	86	85	84	83	82	81	80	P0	99	not bit addressable								SBUF

Memory organization

1. Memory chips are organized into number of location within the IC.
2. Each location can hold 1 bit, 4bit,8 bits or 16 bits data depending upon how they are designed internally.
3. The number of bits each location in memory can hold is always equal to the data pins (8 pins) on the chip.

How many locations exists inside a memory chip??

1. This depends on the address pins (16 pins).
2. The number of locations within a memory IC always equals 2 to the power of number of address pins.
3. Things to remember:
4. A memory chip contains 2^x locations where x is the number of address pins.
5. Each location can hold y bits where y is the number of data pins.
6. The entire chip will contain $2^x * y$ bits.

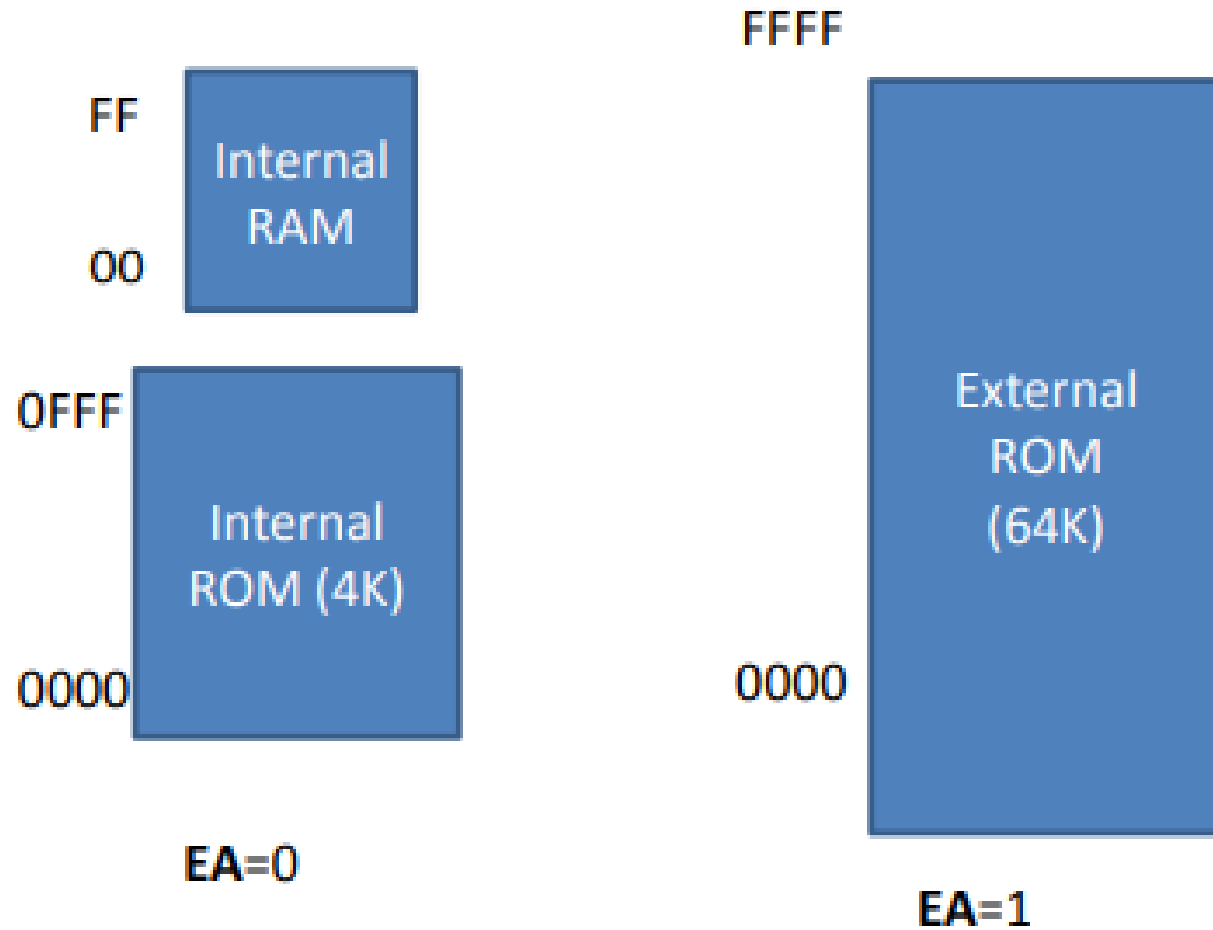
Ex: $2K * 8$

Which means 2K locations and each location can store 8 bits.

But 2K means how many locations right???

x	2^x (memory in bytes)	Memory in Kbytes
10	1024	1K
11	2048	2K
12	4096	4K
13	8192	8K
14	16384	16K
15	32768	32K
16	65536	64K
17		128K
18		256K
19		512K
20		1M
21		2M
22		4M
23		8M
24		16M
25		32M
26		64M
27		128M

ROM - Program or Code memory



Sample code stored in ROM memory

```
1 0000          ORG 0H          ;start at location 0
2 0000 7D25     MOV R5,#25H     ;load 25H into R5
3 0002 7F34     MOV R7,#34H     ;load 34H into R7
4 0004 7400     MOV A,#0        ;load 0 into A
5 0006 2D       ADD A,R5        ;add contents of R5 to A
                                ;now A = A + R5
6 0007 2F       ADD A,R7        ;add contents of R7 to A
                                ;now A = A + R7
7 0008 2412     ADD A,#12H      ;add to A value 12H
                                ;now A = A + 12H
8 000A 80FE HERE: SJMP HERE     ;stay in this loop
9 000C          END            ;end of asm source file
```

Working of PC and contents in ROM

	Machine Language	Assembly Language
0000	7D25	MOV R5,#25H
0002	7F34	MOV R7,#34H
0004	7400	MOV A,#0
0006	2D	ADD A,R5
0007	2F	ADD A,R7
0008	2412	ADD A,#12H
000A	80FE	HERE: SJMP HERE

Address	Code
0000	7D
0001	25
0002	7F
0003	34
0004	74
0005	00
0006	2D
0007	2F
0008	24
0009	12
000A	80
000B	FE

Starting address in PC is 0000

PC 0000
 0002
 0004
 0006
 0007
 0008
 000A

Program Counter (PC) and ROM space

- ❖ An important register in the 8051 is the PC (program counter).
- ❖ The program counter points to the address of the next instruction to be executed. As the CPU fetches the opcode from the program ROM, the program counter is incremented to point to the next instruction.
- ❖ The program counter in the 8051 is 16 bits wide. This means that the 8051 can access program addresses 0000 to FFFFH, i.e a total of 64K bytes of code.
- ❖ But in case of 8051 the on chip ROM is 4K bytes thus, PC can access program address from 0000 to 0FFF

The question arises is

Where does the 8051 wake up when it is powered or at what address does the CPU wake up?

- ❖ The microcontroller wakes up at memory address 0000 when it is powered up
- ❖ When the 8051 is powered up the PC (program counter) has the value of 0000 in it.
- ❖ This means that it expects the first opcode to be stored at ROM address 0000H.
- ❖ For this reason in the 8051 system, the first opcode must be burned into memory location 0000H of program ROM since this is where it looks for the first instruction when it is booted.
- ❖ We achieve this by the ORG statement in the source program. ORG 0000H



8051

Assembly Language and Instruction set

Structure of Assembly Language

[*label:*] *mnemonic* [*operands*] [; *comment*]

mnemonic **destination**, **source**

Example:

MOV R1, #25H * ; load data 25H into R1



Instruction Groups

- The 8051 has 256 instructions
- The instructions are grouped into 5 groups
 - Data Transfer
 - Arithmetic
 - Logic
 - Boolean
 - Branching



Addressing Modes

Addressing Modes – The term addressing modes refers to the way in which the operand of an instruction is specified.

Different addressing modes are:

1. Immediate Addressing mode
2. Register Addressing mode
3. Direct Addressing mode
4. Register indirect/Indirect Addressing mode
5. Register Indexed addressing mode

Immediate Addressing Modes

Specify data by its **value and #**, source is always a **DATA**

MOV A, #25H (This instruction will move the data 25H to accumulator.)

MOV A, #0 ;put 0 in the accumulator, A = 00000000

MOV R4, #11h ;put 11hex in the R4 register, R4 = 00010001

MOV B, #11 ;put 11 decimal in b register, B = 00001011

MOV DPTR, #7521h ;put 7521 hex in DPTR
;DPTR = 0111010100100001

Immediate Addressing Modes

MOV DPTR,#7521h

MOV DPL,#21H

MOV DPH, #75

COUNT EQU 30 ; count=30

MOV R4, #COUNT ;move in R4 30

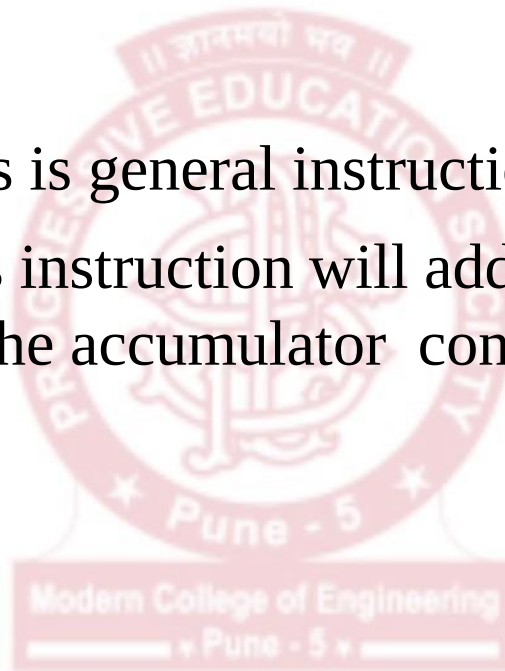
MOV DPTR,#MYDATA

ORG 200H

MYDATA:DB “IRAN”

Register Addressing Modes

- In register addressing mode; the source and/or destination is a register
- For example;
- MOV A, Rn (This is general instruction).
- ADD A, R5 (This instruction will add the contents of register R5 with the accumulator contents).



Register Addressing Modes

Either source or destination is one of **CPU register**

MOV R0,A

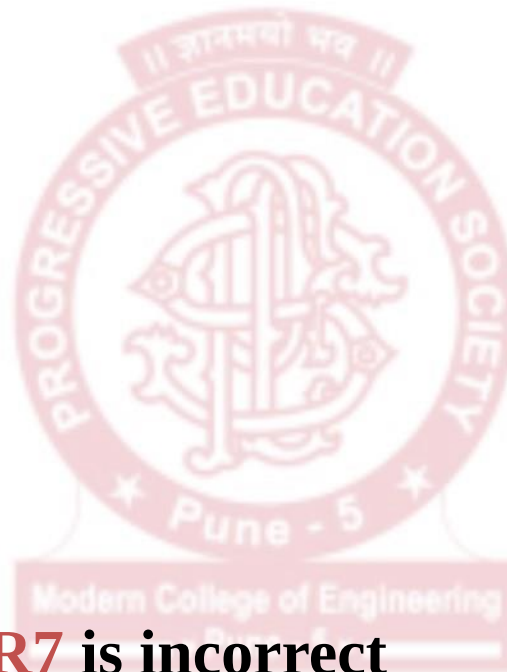
MOV A,R7

ADD A,R4

ADD A,R7

MOV R5,DPL

MOV R,DPH

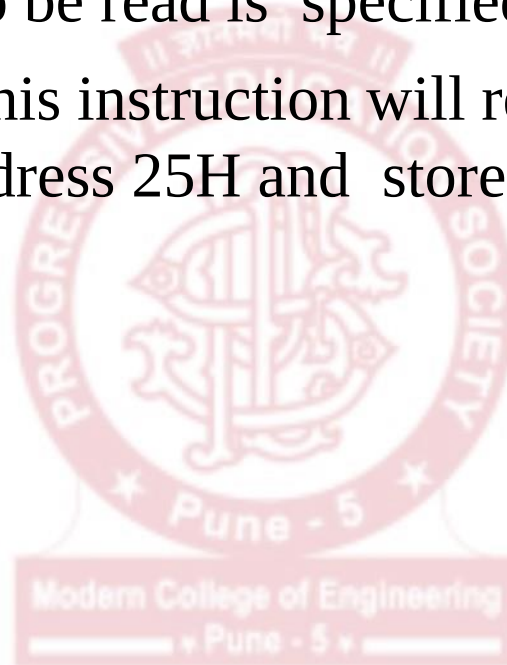


Note that **MOV R4,R7** is incorrect

Rn to Rn is not allowed

Direct Addressing Modes

- In direct addressing mode; the address of memory location containing data to be read is specified in instruction
- **MOV A, 25H** (This instruction will read/move the data from internal RAM address 25H and store it in the accumulator.)



Direct Addressing Modes

Specify data by its 8-bit address, usually for 30h-7Fh of RAM

MOV A,70h ;copy contents of RAM at 70h to a

MOV R0,40h ;copy contents of RAM at 40h to R0

MOV 56h,A ;put contents of a to 56h of RAM

MOV 0D0h,A ;put contents of a into PSW

DATA MEMORY (RAM)		
INTERNAL DATA ADDRESS SPACE		
0xFF	Upper 128 RAM (Indirect Addressing Only)	Special Function Register's (Direct Addressing Only)
0x80 0x7F	(Direct and Indirect Addressing)	
0x30 0x2F	Bit Addressable	Lower 128 RAM (Direct and Indirect Addressing)
0x20 0x1F		
0x00		
	General Purpose Registers	

Direct Addressing Modes

Use R0-R7 by direct address

MOV A,4 $\equiv$ MOV A,R4

MOV A,7 $\equiv$ MOV A,R7

MOV 7,2 $\equiv$ ~~MOV R7,R6~~

MOV R2,#5 ;Put 5 in R2

MOV R2,5 ;Put content of RAM at 5 in R2

Register Indirect Addressing Modes

The address of the source or destination is specified in registers.

MOV A,@R0 This instruction moves the data from the register whose address is in the R0 register into the accumulator

Uses registers R0 or R1 for 8-bit address:

MOV R0, #0x3C

MOV @R0, #3 ; memory at 3C gets #3
; M[3C] = 3

Uses DPTR register for 16-bit addresses:

MOV DPTR, #0x9000 ; DPTR = 9000h

MOVX A, @DPTR ; A = M[9000]

Note that 9000 is an address in external memory

Register Indexed Addressing Modes

MOVC A, @ A + DPTR (This instruction moves the data from the memory to accumulator; whose address is computed by adding the contents of accumulator and DPTR)

MOVX A, @ A + DPTR (External memory)

Source or destination address is the sum of the base address and the accumulator(Index)

@A + DPTR OR @A + PC

- Base address can be DPTR or PC

MOV DPTR, #4000h

MOV A, #5

MOVC A,@A + DPTR ;A = M[4005]

Register Indexed Addressing Modes

- Base address can be DPTR or PC

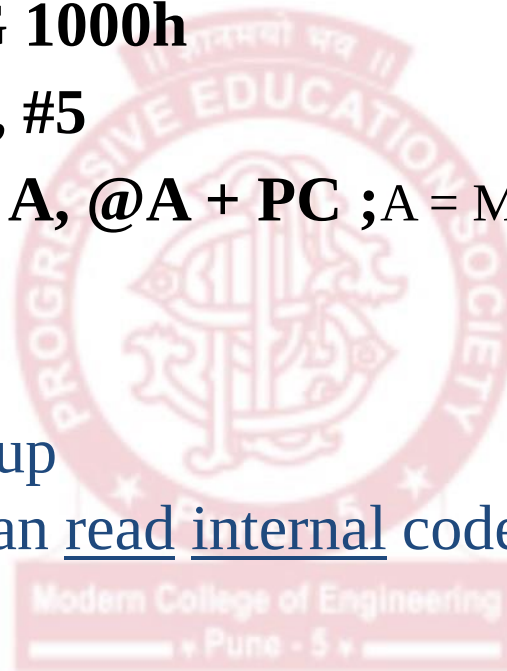
ORG 1000h

1000 MOV A, #5

1002 MOVC A, @A + PC ; A = M[1008] = 5+1003=@1008

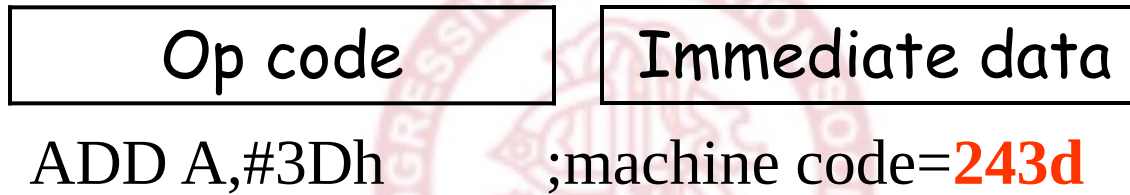
1003 NOP

- **PC** → Table Lookup
- **MOVC** **only** can read internal code memory

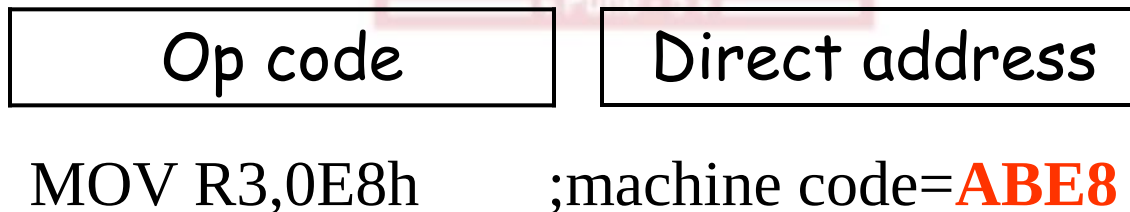


8051 Instruction Format

- Immediate addressing



- Direct addressing



Data Transfer Instructions

- MOV dest, source
- Stack instructions

PUSH BYTE ;increment stack pointer,
;move byte on stack

POP BYTE ;move from stack to byte,
;decrement stack pointer

- Exchange instructions

XCH A, BYTE ;exchange accumulator and byte

XCHD A, BYTE ;exchange low nibbles of accumulator and byte

Data Transfer Instructions

- MOV
 - 8-bit data transfer for internal RAM and the SFR.
MOV A, Rn
MOV A, @Ri
MOV Rn, A
MOV Rn, #data
MOV direct, Rn
MOV direct, @Ri
MOV @Ri, A
MOV @Ri, #data
 - MOV A, direct
MOV A, #data
MOV Rn, direct
MOV direct, A
MOV direct, direct
MOV direct, #data
MOV @Ri, direct

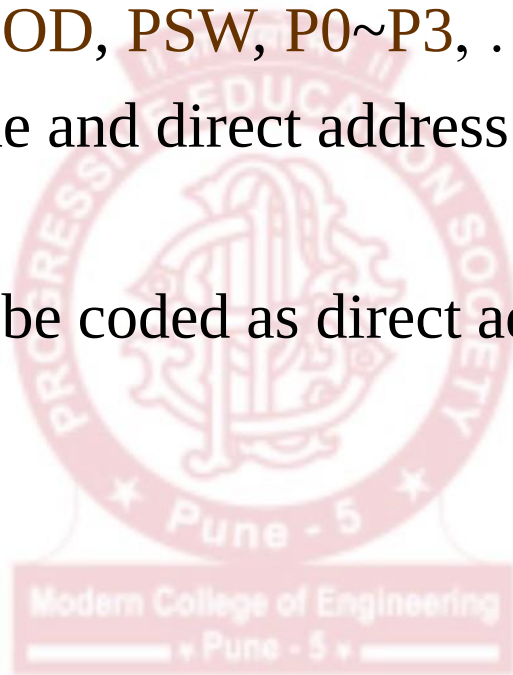
SFRs Address

All SFRs such as

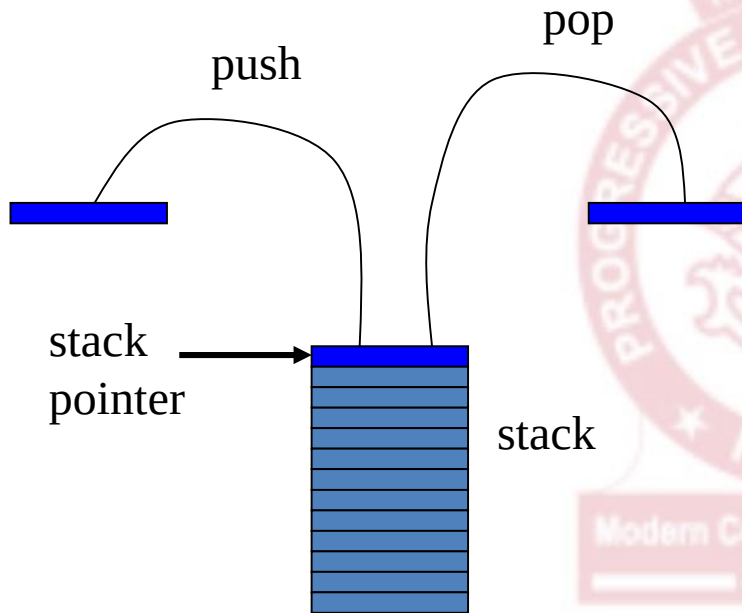
(ACC, B, PCON, TMOD, PSW, P0~P3, ...)

are accessible by name and direct address

But all of them must be coded as direct address



Stacks



RAM

Byte address	Bit address							
27	3F	3E	3D	3C	3B	3A	39	38
26	37	36	35	34	33	32	31	30
25	2F	2E	2D	2C	2B	2A	29	28
24	27	26	25	24	23	22	21	20
23	1F	1E	1D	1C	1B	1A	19	18
22	17	16	15	14	13	12	11	10
21	0F	0E	0D	0C	0B	0A	09	08
20	07	06	05	04	03	02	01	00
1F	Bank 3							
18								
17	Bank 2							
10								
0F	Bank 1							
08								
07	Default register bank for R0–R7							
00								

Bit-addressable locations

Bit-addressable locations

Byte address	Bit address															
7F	General purpose RAM															
30																
2F	7F	7E	7D	7C	7B	7A	79	78								
2E	77	76	75	74	73	72	71	70								
2D	6F	6E	6D	6C	6B	6A	69	68								
2C	67	66	65	64	63	62	61	60								
2B	5F	5E	5D	5C	5B	5A	59	58								
2A	57	56	55	54	53	52	51	50								
29	4F	4E	4D	4C	4B	4A	49	48								
28	47	46	45	44	43	42	41	40								

Stack

- Stack-oriented data transfer
 - Only one operand (**direct** addressing)
 - SP is other operand – register indirect - implied
- Direct addressing mode must be used in Push and Pop

MOV SP, #0x40 ; Initialize SP

**PUSH 0x55 ; SP = SP+1, M[SP] = M[55]
; M[41] = M[55] = part of RAM memory**

POP B ; B = M[55]

Note: One can only specify RAM or SFRs (direct mode) to push or pop. Therefore, to push/pop the accumulator, must use **ACC, not **A****

Stack (push, pop)

- Therefore

PUSH A ;is invalid

PUSH R0 ;is invalid

PUSH R1 ;is invalid

PUSH ACC ;is correct

PUSH PSW ;is correct

PUSH B ;is correct

PUSH 13h

PUSH 0

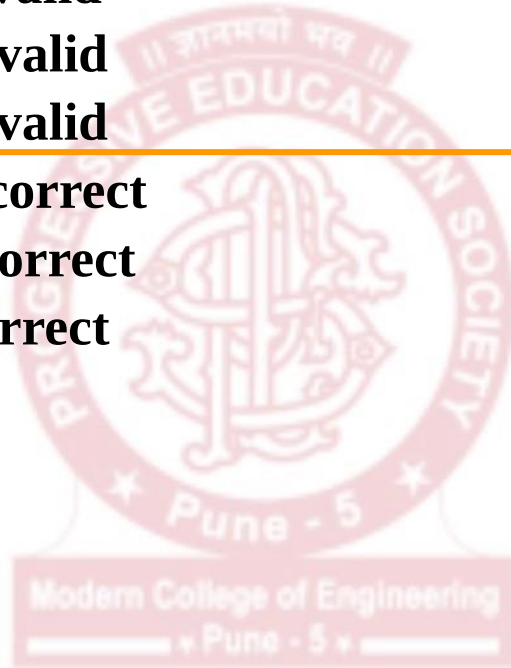
PUSH 1

POP 7

POP 8

PUSH 0E0h ;ACC

POP 0F0h ;B



Exchange Instructions

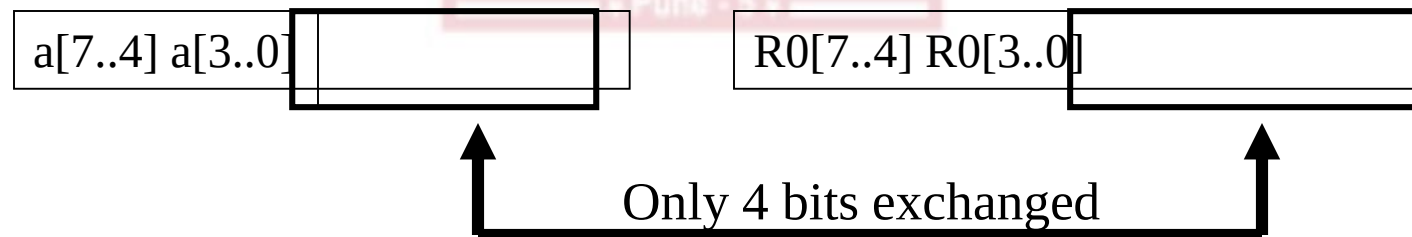
Two way data transfer

XCH A, 30h ; A == M[30]

XCH A, R0 ; A == R0

XCH A, @R0 ; A == M[R0]

XCHD A, R0 ; exchange “digit”



Bit-Oriented Data Transfer

PSW: PROGRAM STATUS WORD. BIT ADDRESSABLE.

CY	AC	F0	RS1	RS0	OV	—	P
----	----	----	-----	-----	----	---	---

- Transfers between individual bits.
- Carry flag (C) (bit 7 in the PSW) is used as a single-bit accumulator
- RAM bits in addresses 20-2F are bit addressable

MOV C, P0.0

MOV C, 67h

MOV C, 2Ch.7

RAM

Byte address	Bit address							
27	3F	3E	3D	3C	3B	3A	39	38
26	37	36	35	34	33	32	31	30
25	2F	2E	2D	2C	2B	2A	29	28
24	27	26	25	24	23	22	21	20
23	1F	1E	1D	1C	1B	1A	19	18
22	17	16	15	14	13	12	11	10
21	0F	0E	0D	0C	0B	0A	09	08
20	07	06	05	04	03	02	01	00
1F	Bank 3							
18								
17	Bank 2							
10								
0F	Bank 1							
08								
07	Default register bank for R0–R7							
00								

Bit-addressable locations

Byte address	Bit address							
7F	General purpose RAM							
30								
2F								
2E								
2D								
2C								
2B								
2A								
29								
28								

Bit-addressable locations

SFRs that are Bit Addressable

SFRs with addresses ending in 0 or 8 are bit-addressable.

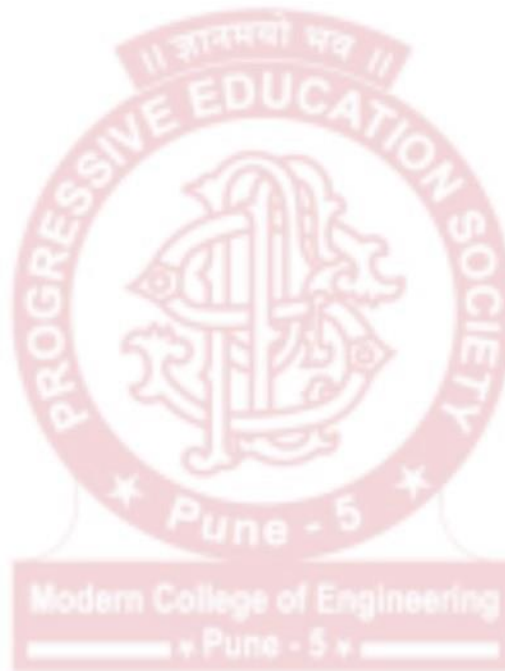
(80, 88, 90, 98, etc)

Notice that all 4 parallel I/O ports are bit addressable.

Byte address	Bit address		Byte address	Bit address	
98	9F 9E 9D 9C 9B 9A 99 98	SCON	FF		
			F0	F7 F6 F5 F4 F3 F2 F1 F0	B
90	97 96 95 94 93 92 91 90	P1			
			E0	E7 E6 E5 E4 E3 E2 E1 E0	ACC
8D	not bit addressable	TH1			
8C	not bit addressable	TH0	D0	D7 D6 D5 D4 D3 D2 - D0	PSW
8B	not bit addressable	TL1			
8A	not bit addressable	TL0	B8	- - - BC BB BA B9 B8	IP
89	not bit addressable	TMOD			
88	8F 8E 8D 8C 8B 8A 89 88	TCON	B0	B7 B6 B5 B4 B3 B2 B1 B0	P3
87	not bit addressable	PCON			
			A8	AF - - AC AB AA A9 A8	IE
83	not bit addressable	DPH			
82	not bit addressable	DPL	A0	A7 A6 A5 A4 A3 A2 A1 A0	P2
81	not bit addressable	SP			
80	87 86 85 84 83 82 81 80	P0	99	not bit addressable	SBUF

Arithmetic Instructions

- Add
- Subtract
- Increment
- Decrement
- Multiply
- Divide
- Decimal adjust



Arithmetic Instructions

Mnemonic	Description
ADD A, byte	add A to byte, put result in A
ADDC A, byte	add with carry
SUBB A, byte	subtract with borrow
INC A	increment A
INC byte	increment byte in memory
INC DPTR	increment data pointer
DEC A	decrement accumulator
DEC byte	decrement byte
MUL AB	multiply accumulator by b register
DIV AB	divide accumulator by b register
DA A	decimal adjust the accumulator

Add Instructions

ADD A, BYTE ; a = a + byte

ADDC A, BYTE ; a = a + byte + C

These instructions affect 3 bits in PSW:

C = 1 if result of add is greater than FF

AC = 1 if there is a carry out of bit 3

OV = 1 if there is a carry out of bit 7, but not from bit 6, or
visa versa.

Program Status Word (PSW)

Bit	7	6	5	4	3	2	1	0
Flag	CY	AC	F0	RS1	RS0	OV	F1	P
Name	Carry Flag	Auxiliary Carry Flag	User Flag 0	Register Bank Select 1	Register Bank Select 0	Overflo w flag	User Flag 1	Parity Bit

Add Examples

MOV A, #3Fh
ADD A, #D3h

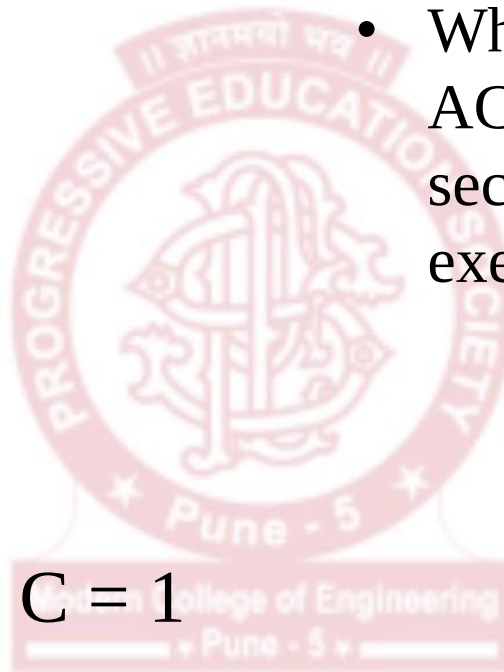
0011 1111
1101 0011
0001 0010

- What is the value of the C, AC, OV flags after the second instruction is executed?

C = 1

AC = 1

OV = 0



Addition Example 8 bit

```
; Computes Z = X + Y
; Adds values at locations 78h and 79h and puts them in 7Ah
;-----
X      EQU      78h      ; X = 78h
Y      EQU      79h      ; Y = 79h
Z      EQU      7Ah
;-----
          ORG 00h
          LJMP MAIN
;-----
          ORG 100h
MAIN:
          MOV A, X
          ADD A, Y
          MOV Z, A
          END
```

Try: Addition of two 16 bit numbers

Subtract

SUBB A, BYTE	subtract with borrow
---------------------	----------------------

Example:

SUBB A, #0x4F

Notice that

There is **no** subtraction WITHOUT borrow.

Therefore, if a subtraction without borrow is desired,
it is necessary to **clear the C** flag.

Example:

CLR C

SUBB A, #0x4F

Increment and Decrement

INC A	increment A
INC BYTE	increment byte in memory
INC DPTR	increment data pointer
DEC A	decrement accumulator
DEC BYTE	decrement byte

- The increment and decrement instructions do **NOT** affect the C flag.
- Notice we can **only** INCREMENT the data pointer, not decrement.

Example: Increment 16-bit Word

- Assume 16-bit word in R3:R2

mov a, r2

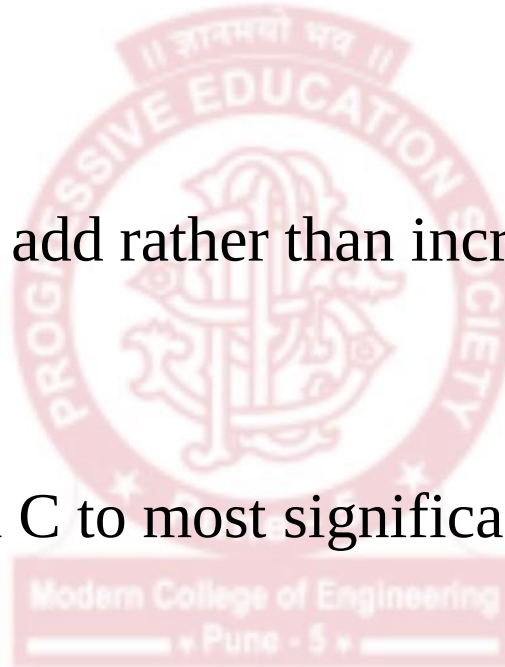
add a, #1 ; use add rather than increment to affect C

MOV r2, a

MOV a, r3

addc a, #0 ; add C to most significant byte

MOV r3, a



Multiply

When multiplying two 8-bit numbers, the size of the maximum product is 16-bits

$$\text{FF} \times \text{FF} = \text{FE01}$$
$$(255 \times 255 = 65025)$$

MUL AB

Note : **B** gets the **High** byte
A gets the **Low** byte

Division

- Integer Division

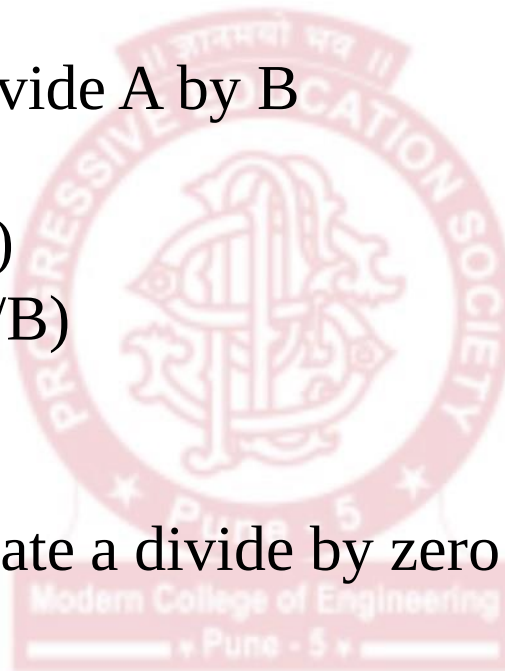
DIV AB ; divide A by B

A - Quotient(A/B)

B - Remainder(A/B)

OV - used to indicate a divide by zero condition.

C – set to zero A



Decimal Adjust

DAA ; decimal adjust a

Used to facilitate BCD addition.

Adds “6” to either high or low nibble after an addition to create a valid BCD number.

Example:

MOV A, #23h

MOV B, #29h

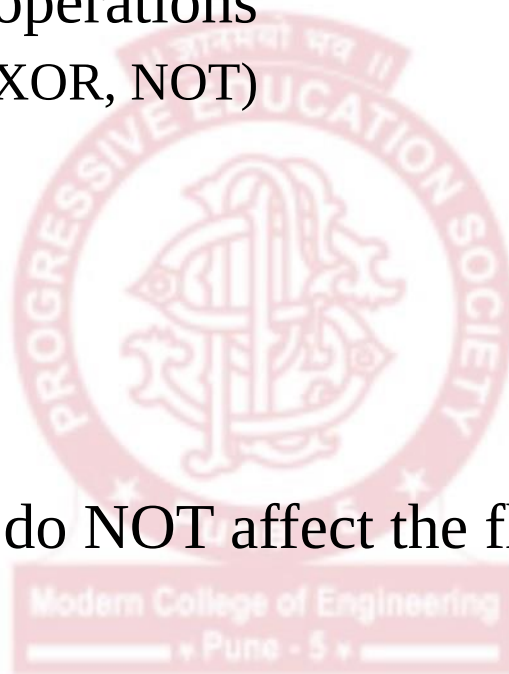
ADD A, B ; $A = 23h + 29h = 4Ch$ (wanted 52)

DAA ; $A = A + 6 = 52$

Logic Instructions

- ❑ Bitwise logic operations
 - ❖ (AND, OR, XOR, NOT)
- ❑ Clear
- ❑ Rotate
- ❑ Swap

Logic instructions do NOT affect the flags in PSW



Bitwise Logic

ANL ↔ AND

ORL ↔ OR

XRL ↔ XOR

CPL ↔ Complement

Examples:

ANL 00001111
 10101100
 00001100

ORL 00001111
 10101100
 10101111

XRL 00001111
 10101100
 10100011

CPL 10101100
 01010011



Uses of Logic Instructions

- Force individual bits low, without affecting other bits.

ANL PSW, #0xE7

;PSW AND 11100111

- Force individual bits high.

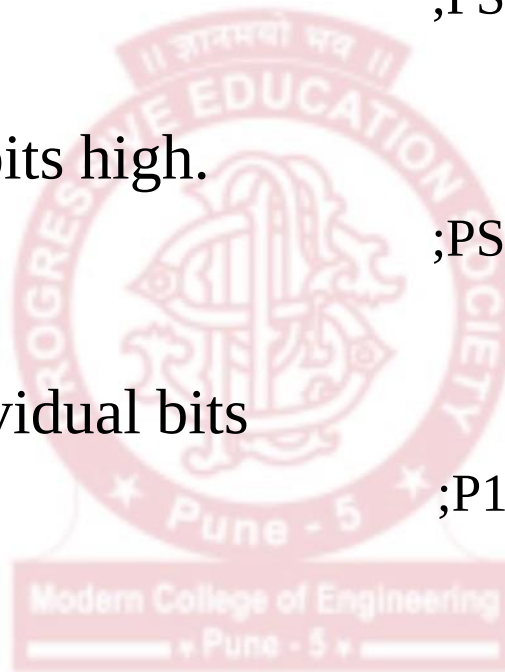
ORL PSW, #0x18

;PSW OR 00011000

- Complement individual bits

XRL P1, #0x40

;P1 XRL 01000000



Other Logic Instructions

CLR - clear

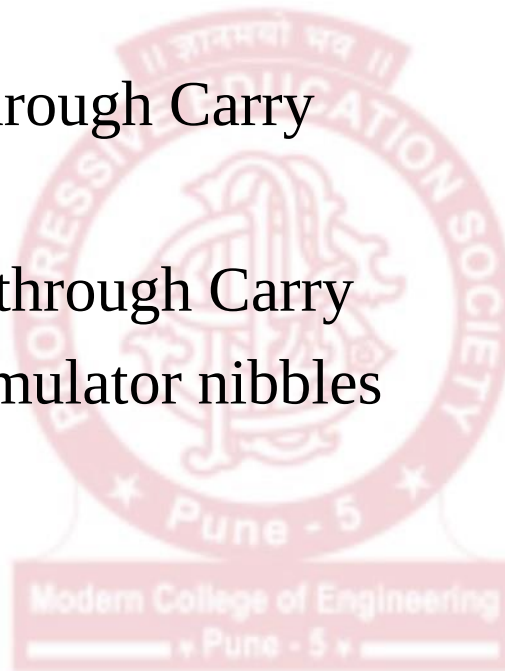
RL – rotate left

RLC – rotate left through Carry

RR – rotate right

RRC – rotate right through Carry

SWAP – swap accumulator nibbles



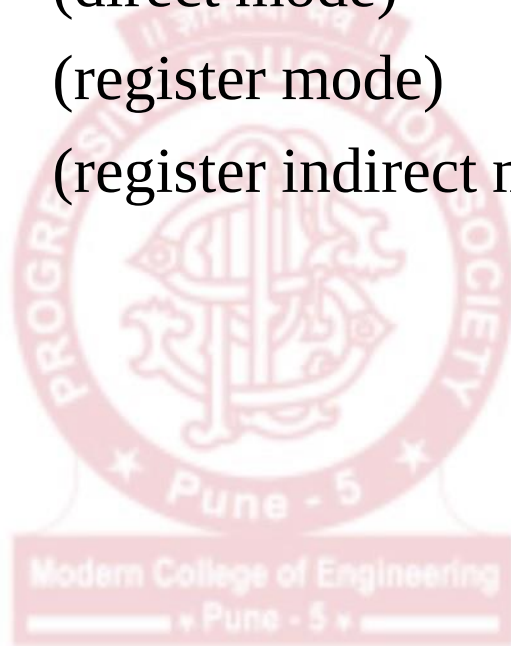
Clr (Set all bits to 0)

CLR A

CLR byte (direct mode)

CLR Ri (register mode)

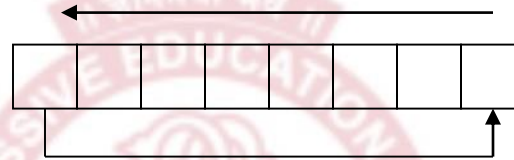
CLR @Rn (register indirect mode)



Rotate

- Rotate instructions operate **only** on A

RL A



MOV A,#0xF0

; a = 11110000

RR A

; a = 11100001

RR A



MOV A, #0xF0

; a = 11110000

RR A

; a = 01111000

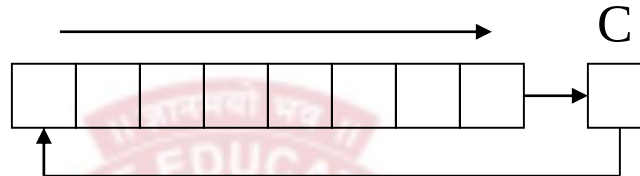
Rotate through Carry

RRC A

MOV A, #0A9h

ADD A, #14h

RRC A



; A = A9

; A = BD (10111101), C = 0

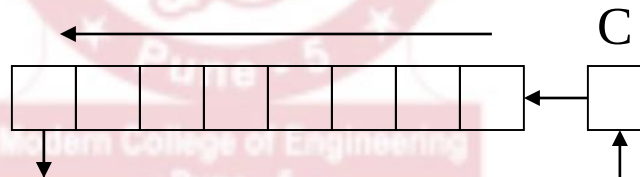
; A = 01011110, C = 1

RLC A

MOV A, #3Ch

SETB C

RLC A



; A = 3Ch(00111100)

; C = 1

; A = 01111001, C = 1

Swap

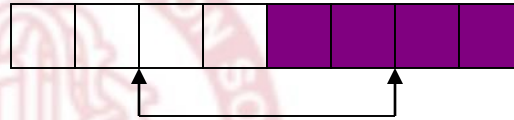
SWAP A

MOV A, #72h

SWAP A

; A = 72h

; A = 27h



Bit Logic Operations

- Some logic operations can be used with single bit operands

ANL C, bit

ORL C, bit

CLR C

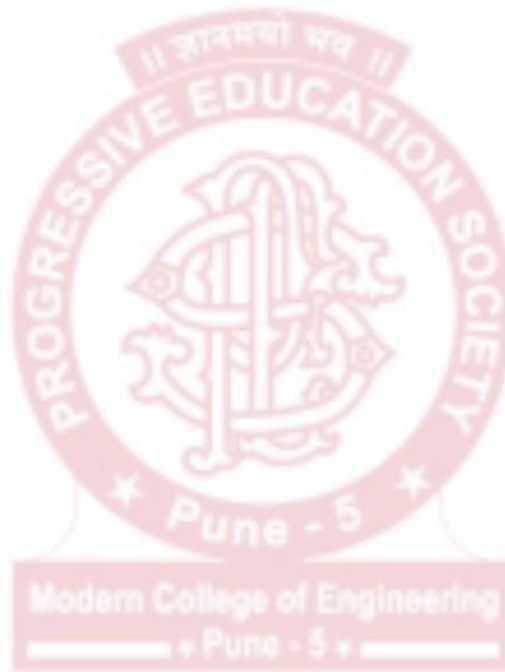
CLR bit

CPL C

CPL bit

SETB C

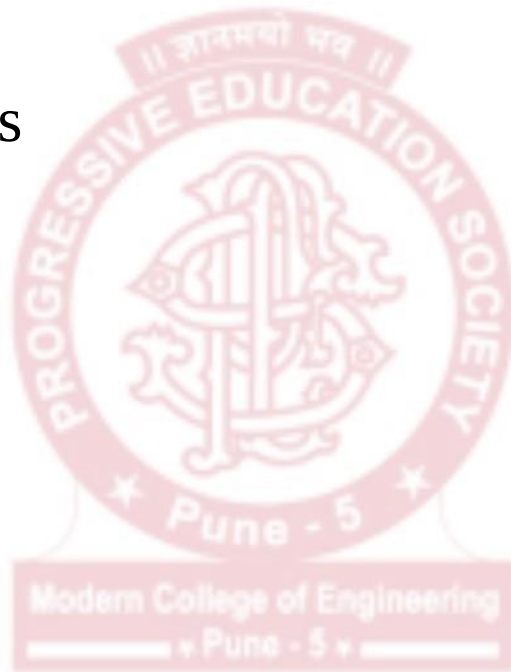
SETB bit



- “bit” can be any of the bit-addressable RAM locations or SFRs.

Program Flow Control

- Unconditional jumps (“go to”)
- Conditional jumps
- Call and return

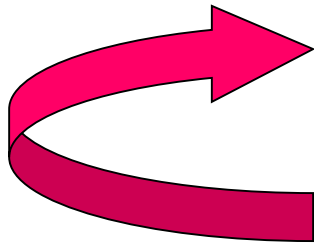


Unconditional Jumps

- **SJMP <rel addr>** ; Short jump, relative address is 8-bit 2's complement number, so jump can be up to 127 locations forward, or 128 locations back.
- **LJMP <address 16>** ; Long jump
- **AJMP <address 11>** ; Absolute jump to anywhere within 2K block of program memory
- **JMP @A + DPTR** ; Long indexed jump

Infinite Loops

Transfer data from P3.7 to P1.6 actually use MOV p1.6, p3.7, but not possible hence use C; Here is a Label



```
HERE: MOV C, p3.7  
      MOV P1.6, C  
      SJMP HERE
```

Microcontroller application programs are almost always infinite loops!

Conditional Jump

- These instructions cause a jump to occur **only** if a condition is **true**. Otherwise, program execution continues with the next instruction.

LOOP: MOV A, P1

JZ LOOP ; if a=0, goto loop, else goto next instruction

MOV B, A

- There is **no** zero flag (Z)
- Content of A checked for zero **on time**

Conditional jumps

Mnemonic	Description
JZ <rel addr>	Jump if a = 0
JNZ <rel addr>	Jump if a != 0
JC <rel addr>	Jump if C = 1
JNC <rel addr>	Jump if C != 1
JB <bit>, <rel addr>	Jump if bit = 1
JNB <bit>,<rel addr>	Jump if bit != 1
JBC <bir>, <rel addr>	Jump if bit =1, &clear bit
CJNE A, direct, <rel addr>	Compare A and memory, jump if not equal

Example: Conditional Jumps

if (A = 0) is true
 send A 0 to LED
else
 send A 1 to LED

If A = 0 then P1.6 to be cleared else setb P1.6

JZ LED_OFF
SETB P1.6
SJMP **SKIPOVER**
LED_OFF: CLR P1.6
MOV A, P0
SKIPOVER: END

More Conditional Jumps

Mnemonic	Description
CJNE A, #data <rel addr>	Compare A and data, jump if not equal
CJNE Rn, #data <rel addr>	Compare Rn and data, jump if not equal
CJNE @Rn, #data <rel addr>	Compare Rn and memory, jump if not equal
DJNZ Rn, <rel addr>	Decrement Rn and then jump if not zero
DJNZ direct, <rel addr>	Decrement memory and then jump if not zero

Iterative Loops

Using CJNE

For A = 0 to 4 do
 {...}

CLR A

LOOP: ...

INC A

CJNE A, #4, LOOP

Using DJNZ

For A = 4 to 0 do
 {...}

MOV A, #04

MOV R0, #4

LOOP: ...

DEC A

DJNZ R0, LOOP

Iterative Loops(examples)

Program to move data 25h to memory locations from 10 h to 15h without loop

```
MOV 10h,#25h
MOV 11h,#25h
MOV 12h,#25h
MOV 13h,#25h
MOV 14h,#25h
MOV 15h,#25h
END
```

Program to move data 25h to memory locations from 10 h to 15h with loop

```
MOV A,#25h
MOV R0,#10h
MOV R2,#6
AGAIN: MOV @R0,A
INC R0
DJNZ R2,AGAIN
END
```

Program to understand loop in a loop –
Complement A for 500 times = 50*10

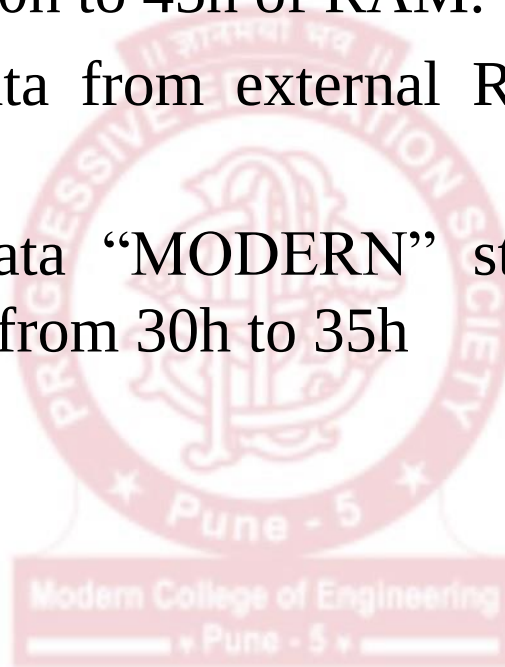
```
MOV A,#0AAh ; A = AA=10101010
MOV B,#10
BACK1:MOV R6,#50
BACK2:CPL A ;A = 01010101
DJNZ R6,BACK2
DJNZ B,BACK1
END
```

Add data 5 12 times (repetitive addition)
5+5+5+5....5

```
MOV A,#0h
MOV R4,#12h
BACK: ADD A,#05
DJNZ R4,BACK
MOV R5,A
END
```

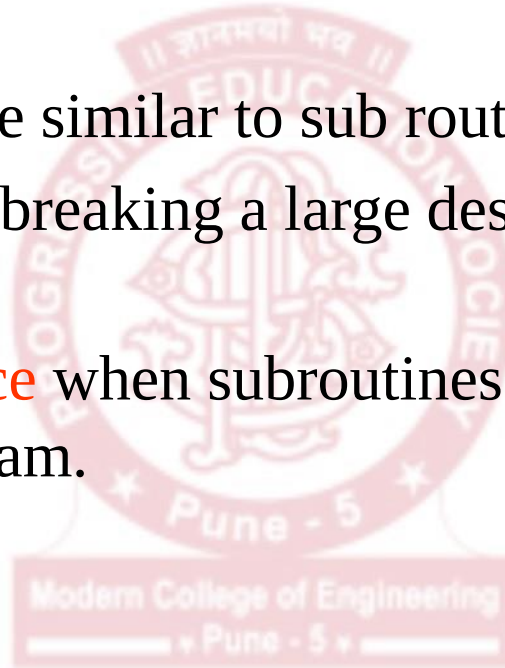
Loops

- WAP to move data from memory locations 30h to 35h of internal RAM to 40h to 45h of RAM.
- WAP to move data from external RAM addresses 0225h to 0229h
- WAP to move data “MODERN” stored in ROM to RAM addresses starting from 30h to 35h



Why Subroutines?

- Subroutines allow us to have "structured" assembly language programs.
- In C, functions are similar to sub routines.
- This is useful for breaking a large design into manageable parts.
- It **saves code space** when subroutines can be called many times in the same program.



Call and Return

- Call is similar to a jump, but
 - Call **pushes PC** on stack before branching

ACALL <address 11> ; stack = PC
; PC = address 11 bit

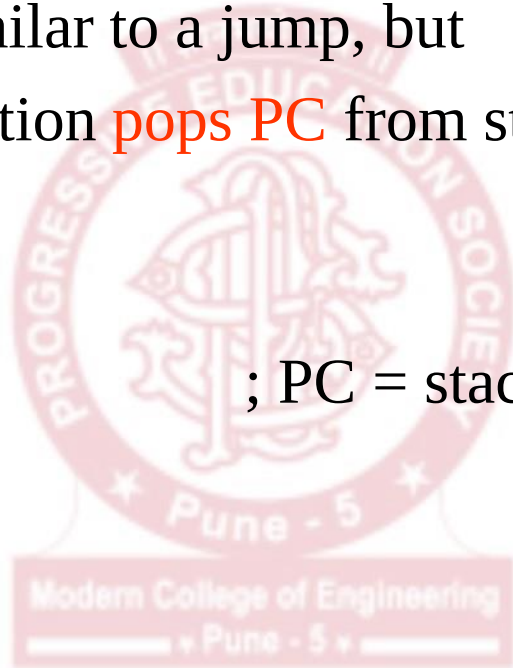
LCALL <address 16> ; stack = PC
; PC = address 16 bit

Return

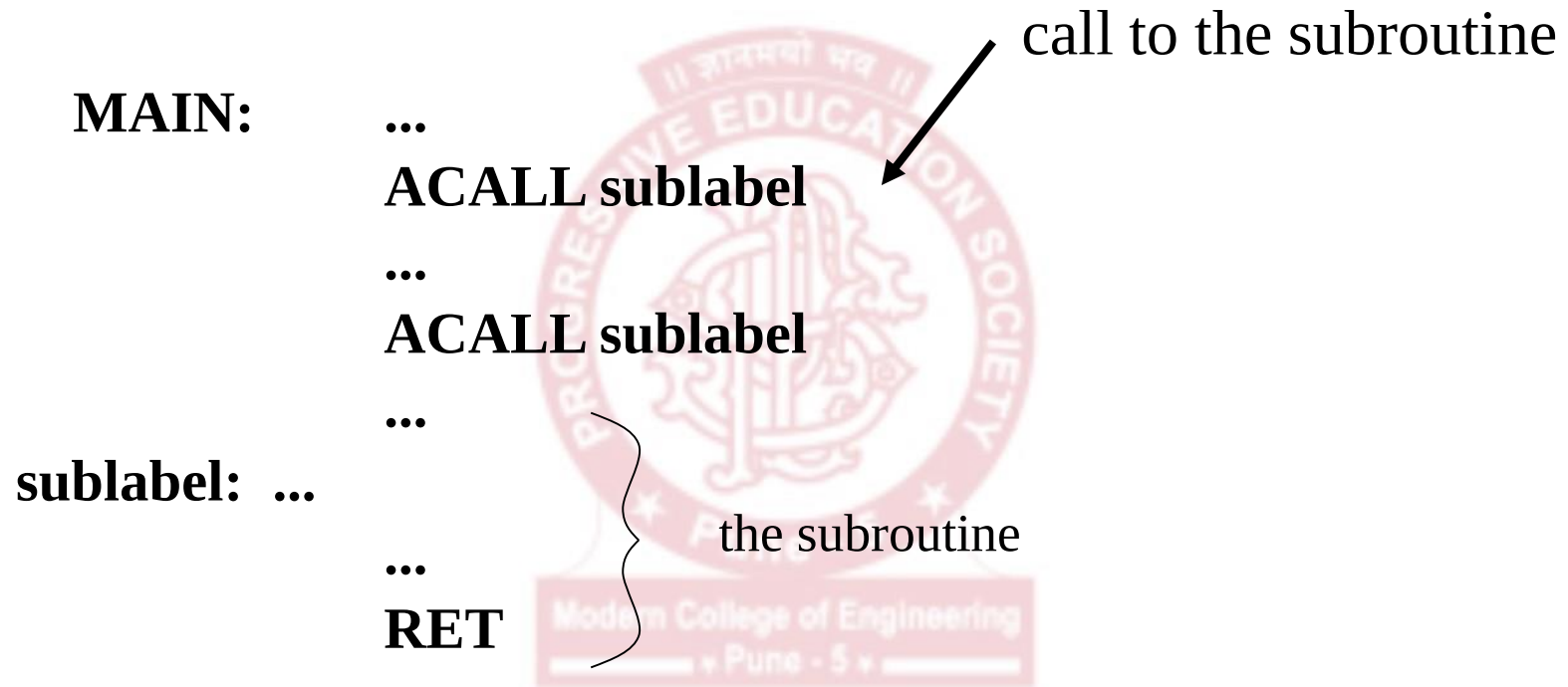
- Return is also similar to a jump, but
 - Return instruction **pops PC** from stack to get address to jump to

RET

; PC = stack



Subroutines



Let us consider an example of blinking of LED's at Port 0 continuously

PC = 0000 fetch MOV P0
 PC = 0003 fetch ACALL
 SP = 07 on reset

Upon execution of ACALL
 instruction address 0005 is
 saved on to stack

0008 = 00

0009 = 05

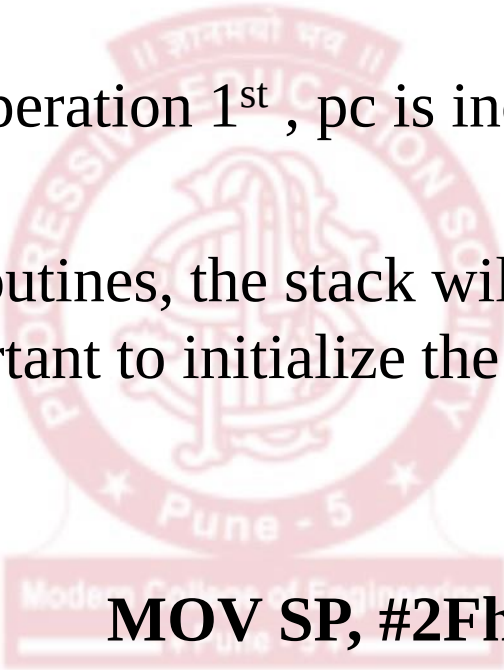
Upon execution of RET
 instruction

Contents from stack are
 popped and SP is initialised to
 07h

0000	75	UP: MOV P0,#00h
0001		
0002	00h	
0003	11	ACALL DELAY
0004		
0005	75	MOV P0,#0FFh
0006		
0007	ffh	
0008	11	ACALL DELAY
0009		
000A	80	SJMP UP
000B		
000E	00	DELAY: NOP
000F	00	NOP
0010	00	NOP
0011	22	RET

Initializing Stack Pointer

- SP is initialized to 07 after reset.(Same address as R7)
- With each push operation 1st, pc is increased
- When using subroutines, the stack will be used to store the PC, so it is very important to initialize the stack pointer. Location **2Fh** is often used.

The logo of PES MCOE is a circular emblem with a red border. Inside the circle, the text "PES MCOE" is written in a stylized font. Below the circle, the text "Pune - 5" is visible. The logo is partially obscured by the text "MOV SP, #2Fh".

MOV SP, #2Fh

Machine cycle and Delay calculation

Find the machine cycle for

(a) XTAL = 11.0592 MHz

(b) XTAL = 16 MHz.

© XTAL = 22 MHz

Solution:

(a) $11.0592 \text{ MHz} / 12 = 921.6 \text{ kHz}$;

machine cycle = $1 / 921.6 \text{ kHz} = 1.085 \mu\text{s}$

(b) $16 \text{ MHz} / 12 = 1.333 \text{ MHz}$;

machine cycle = $1 / 1.333 \text{ MHz} = 0.75 \mu\text{s}$

© $22 \text{ MHz} / 12 = 1.833 \text{ MHz}$,

machine cycle = $1 / 1.833 \text{ MHz} = 0.5454 \mu\text{s}$

Example of delay calculation

```
MOV A,#0AAh
BACK1: MOV P0,A
      LCALL DELAY1
      CPL A
      SJMP BACK1
```

```
DELAY1: MOV R0,#0FFh
      ;1cycle
      HERE: DJNZ R0,HERE ;2cycle
      RET ;2cycle
      END
```

Delay = $1 + 255 * 2 + 2 = 513$ cycles

$513 * 1.085\text{usec}$

```
DELAY2: MOV R6,#0FFh
```

```
BACK1: MOV R7,#0FFh ;1cycle
      HERE: DJNZ R7,HERE ;2cycle
      DJNZ R6,BACK1 ;2cycle
```

```
RET ;2cycle
END
```

Delay = $\{[(255 * 2) + 1 + 2] * 255\} + 1 + 2$
= 130818 machine cycle

$130818 * 1.085\text{usec}$

Example

; Move string from code memory to RAM

```
ORG 0
MOV DPTR,#STRING
MOV R0,#10h
LOOP1: CLR A
        MOVC A,@A+DPTR
        JZ STOP
        MOV @R0,A
        INC DPTR
        INC R0
        SJMP LOOP1
STOP:   SJMP STOP
```

; on-chip code memory used for string

```
ORG 18h
STRING: DB 'this is a string',0
END
```

Example

BACK: CPL P1.2
ACALL DELAY
SJMP BACK

BACK: SETB P1.2
ACALL DELAY
CLR P1.2
ACALL DELAY
SJMP BACK

Example

BACK:

SETB P1.2

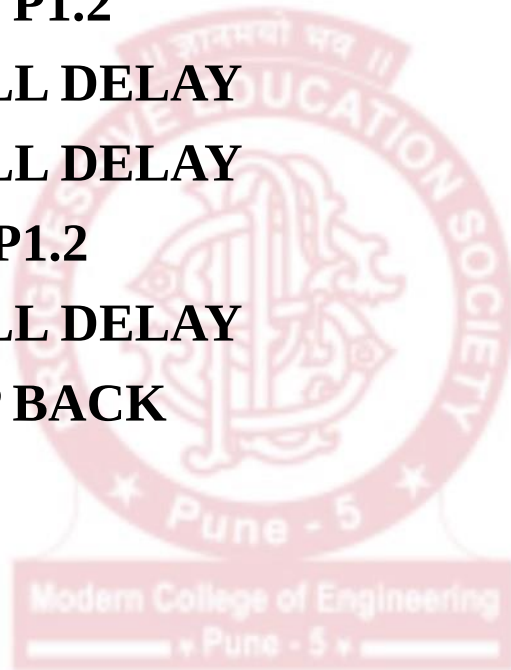
ACALL DELAY

ACALL DELAY

CLR P1.2

ACALL DELAY

SJMP BACK



Programs to practice

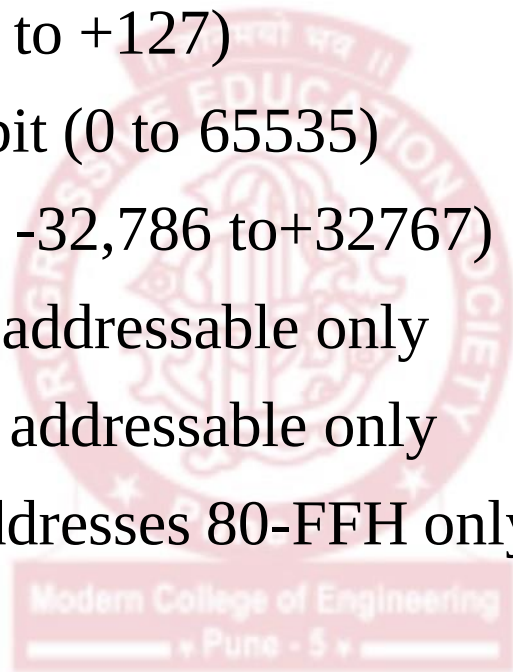
- Write an ALP to display numbers 0 to 9 on seven segment display connected to port 1. (0-3F,1-06,2-5B,3-4F,4-66,5-6D,6-7D,8-7F,9-6F)
- Write an ALP to transfer value 41h serially one bit at a time to pin P2.1. Send LSB first.
- Write an ALP to generate a rotational pattern if the LED's are connected in the following way.



- Write an ALP to Convert a decimal number to ASCII.
- Write a program to add the following numbers and save the result in R2 and R3 of bank 2. The data is stored on on-chip ROM. Data is as follows
53,94,56,92,74,65,43,23,83
- Write an ALP to add 897F9A h and 34BC48 h and save the result in Ram locations starting from 40h.
- Find machine cycle if the crystal frequency is 25MHz
- Write a program to monitor the status of pins p1.2 and p1.3 of port 1 and save the result in bit locations 06h and 07h respectively.
- Write a program to find the number of zeros in register r2.

Data Types

- unsigned Char- (0-255)
- signed char(-128 to +127)
- unsigned int- 16bit (0 to 65535)
- signed int 16 bit(-32,786 to+32767)
- sbit 1bit SFR bit addressable only
- bit 1bit RAM bit addressable only
- sfr- 8bit RAM addresses 80-FFH only



8051 program in C

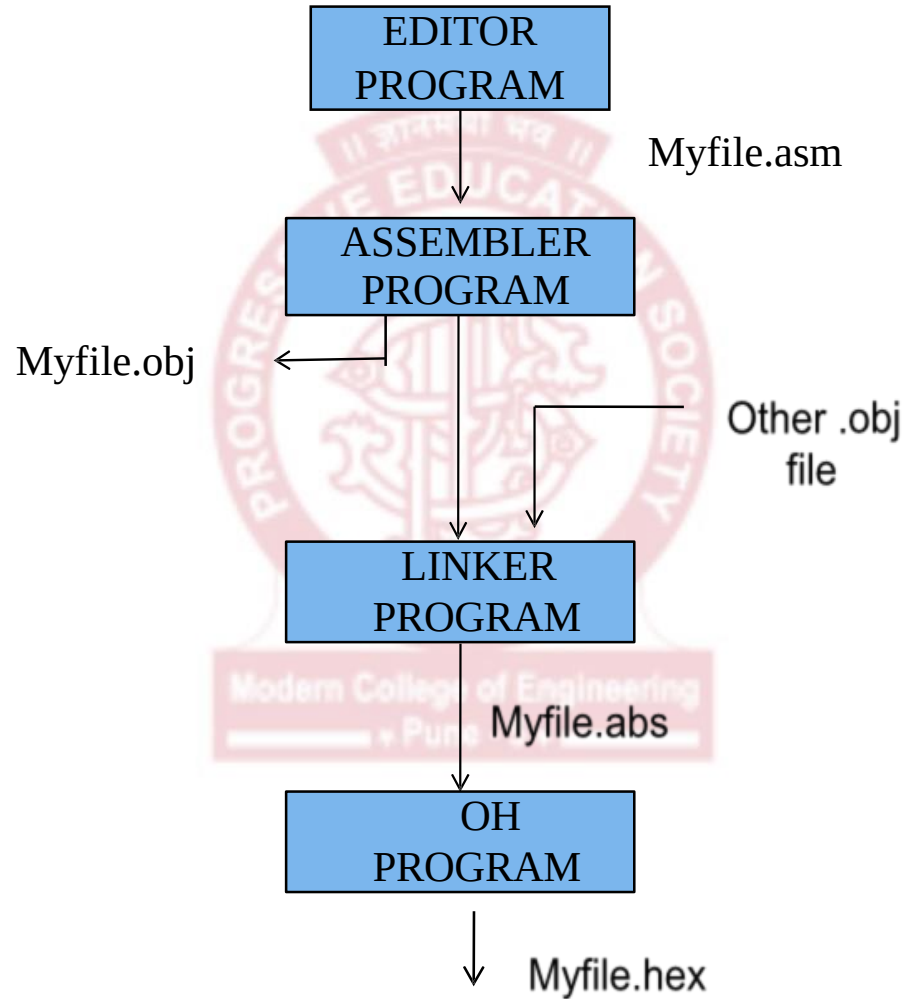
Why assembly

- Microcontroller limited on chip ROM
- Code space for the 8051 is limited 64k

Reasons for using C instead of assembly language

- Easier and less time consuming
- C is easier to modify and update
- Code available in function libraries
- C code is portable to other microcontrollers

Steps to create a Hex file

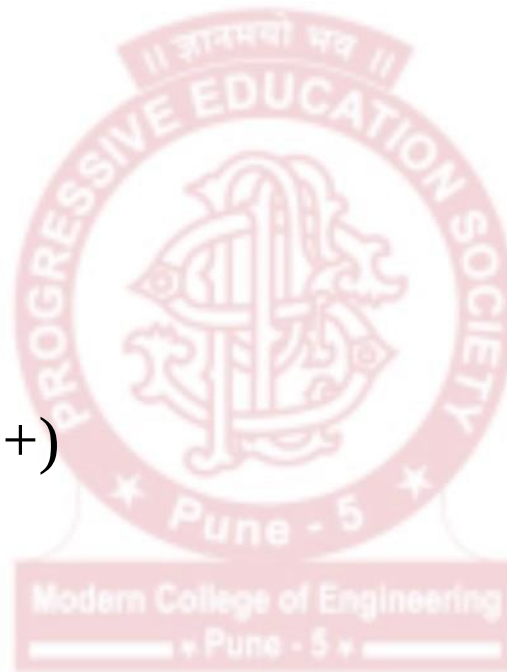


C Programming

Write an 8051 C program to send values 00 – FF - 256 values to port P1.

Solution:

```
#include <reg51.h>
void main(void)
{
    unsigned char z;
    for (z=0;z<=255;z++)
        P1=z;
}
```

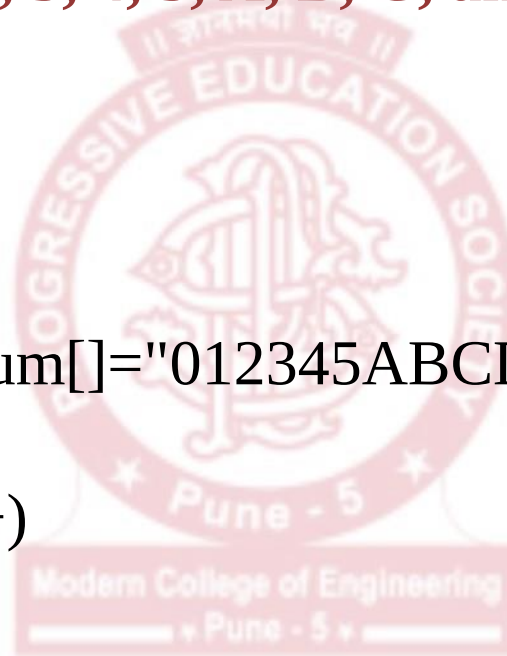


C Programming

Write an 8051 C program to send hex values for ASCII characters of 0, 1, 2, 3, 4, 5, A, B, C, and D to port P1.

Solution:

```
#include <reg51.h>
void main(void)
{
    unsigned char mynum[]="012345ABCD";
    unsigned char z;
    for (z=0;z<=10;z++)
        P1=mynum[z];
}
```

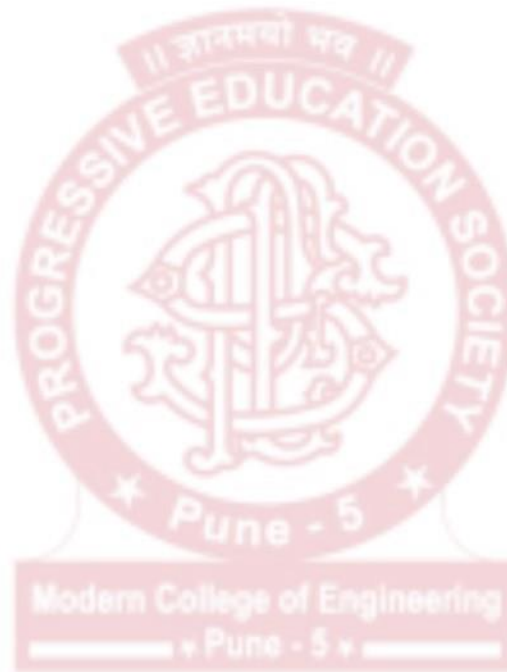


C Programming

Write an 8051 C program to toggle all the bits of P1 continuously.

Solution:

```
//Toggle P1 forever
#include <reg51.h>
void main(void)
{
    for (;;)
    {
        P1=0x55;
        P1=0xAA;
    }
}
```

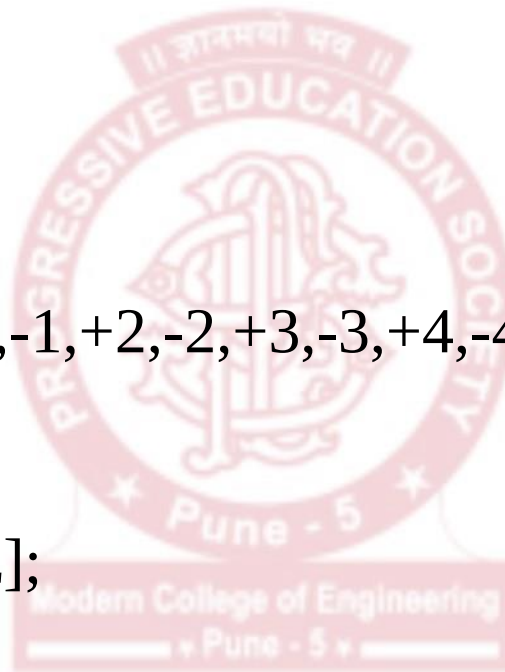


C Programming

Write an 8051 C program to send values of -4 to +4 to port P1.

Solution:

```
////Signed numbers
#include <reg51.h>
void main(void)
{
    char mynum[]={+1,-1,+2,-2,+3,-3,+4,-4};
    unsigned char z;
    for (z=0;z<=8;z++)
        P1=mynum[z];
}
```



C Programming

Write an 8051 C program to toggle bit D0 of the port P1 (P1.0) 50,000 times.

Solution:

```
#include <reg51.h>
```

```
sbit MYBIT=P1^0;
```

```
void main(void)
```

```
{
```

```
    unsigned int z;
```

```
    for (z=0;z<=50000;z++)
```

```
    {
```

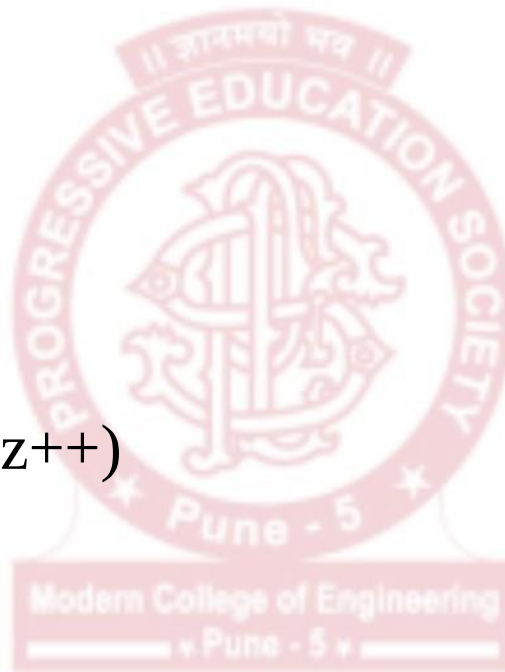
```
        MYBIT=0;
```

```
        MYBIT=1;
```

```
    }
```

```
}
```

\\(P1.0)



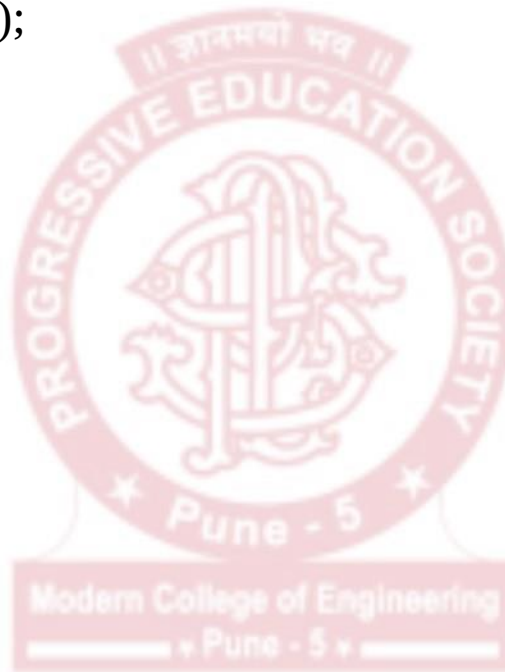
C Programming

Write an 8051 C program to toggle bits of P1 ports continuously with a 250 ms.

Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);
void main(void)
{
    while (1) //repeat forever
    {
        P1=0x55;
        MSDelay(250);
        P1=0xAA;
        MSDelay(250);
    }
}

void MSDelay(unsigned int itime)
{
    unsigned int i,j;
    for (i=0;i<itime;i++)
        for (j=0;j<1275;j++);
}
```

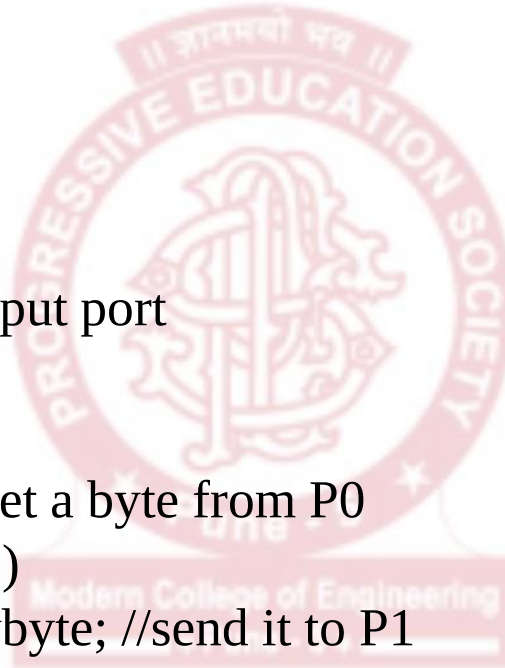


C Programming

Write an 8051 C program to get a byte of data form P0. If it is less than 100, send it to P1; otherwise, send it to P2.

Solution:

```
#include <reg51.h>
void main(void)
{
    unsigned char mybyte;
    P0=0xFF; //make P0 input port
    while (1)
    {
        mybyte=P0; //get a byte from P0
        if (mybyte<100)
            P1=mybyte; //send it to P1
        else
            P2=mybyte; //send it to P2
    }
}
```



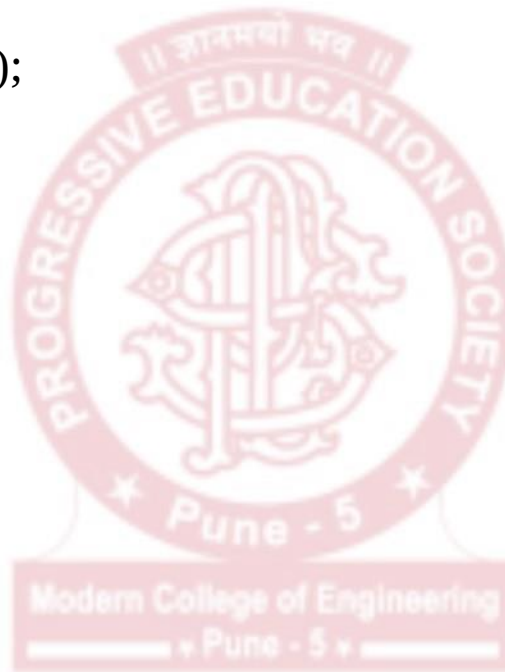
C Programming

Write an 8051 C program to toggle all the bits of P0 and P2 continuously with a 250 ms delay. Using the inverting and Ex-OR operators, respectively.

Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);
void main(void)
{
    P0=0x55;
    P2=0x55;
    while (1)
    {
        P0=~P0;
        P2=P2^0xFF;
        MSDelay(250);
    }
}

void MSDelay(unsigned int itime)
{
    unsigned int i,j;
    for (i=0;i<itime;i++)
        for (j=0;j<1275;j++);
}
```

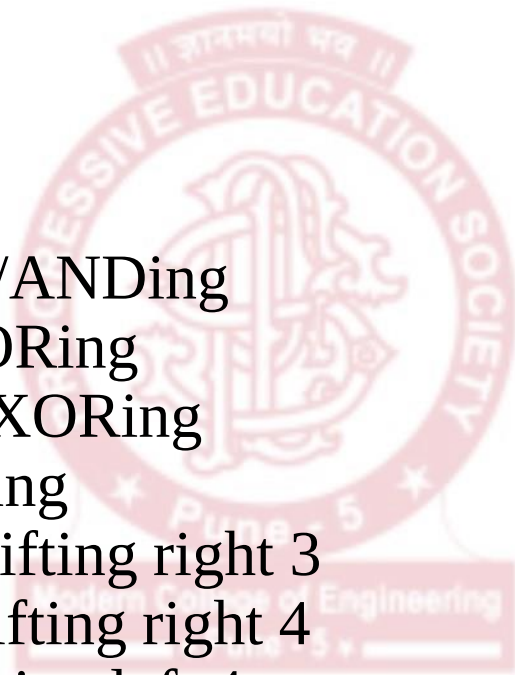


C Programming

Run the following program on your simulator and examine the results.

Solution:

```
#include <reg51.h>
void main(void)
{
    P0=0x35 & 0x0F; //ANDing
    P1=0x04 | 0x68; //ORing
    P2=0x54 ^ 0x78; //XORing
    P0=~0x55; //inversing
    P1=0x9A >> 3; //shifting right 3
    P2=0x77 >> 4; //shifting right 4
    P0=0x6 << 4; //shifting left 4
}
```



C Programming

Write an 8051 C program to convert 11111101 (FD hex) to decimal and display the digits on P0, P1 and P2.

Solution:

```
#include <reg51.h>
```

```
void main(void)
```

```
{
```

```
    unsigned char x,binbyte,d1,d2,d3;
```

```
    binbyte=0xFD;
```

```
    x=binbyte/10;
```

```
    d1=binbyte%10;
```

```
    d2=x%10;
```

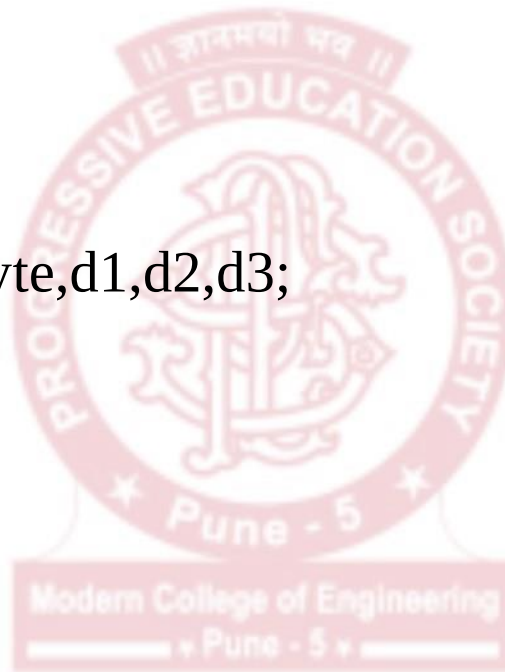
```
    d3=x/10;
```

```
    P0=d1;
```

```
    P1=d2;
```

```
    P2=d3;
```

```
}
```



C Programming

Write an 8051 C program to read the P1.0 and P1.1 bits and issue an ASCII character to P0 according to the following table. For P1.1 P1.0

0 0 send '0' to P0

0 1 send '1' to P0

1 0 send '2' to P0

1 1 send '3' to P0

Solution:

```
#include <reg51.h>
```

```
void main(void)
```

```
{
```

```
    unsigned char z;
```

```
    z=P1;
```

```
    z=z&0x3;
```

```
    switch (z)
```

```
    {
```

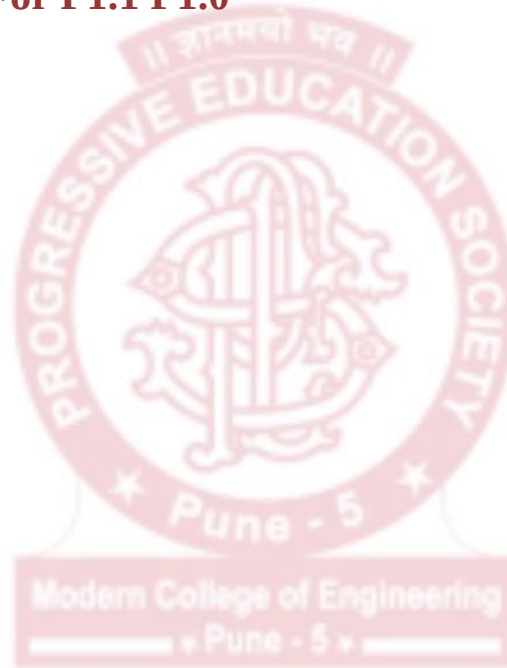
```
        case(0):
```

```
        {
```

```
            P0='0';
```

```
            break;
```

```
        }
```



```
        case(1):
```

```
        {
```

```
            P0='1';
```

```
            break;
```

```
        }
```

```
        case(2):
```

```
        {
```

```
            P0='2';
```

```
            break;
```

```
        }
```

```
        case(3):
```

```
        {
```

```
            P0='3';
```

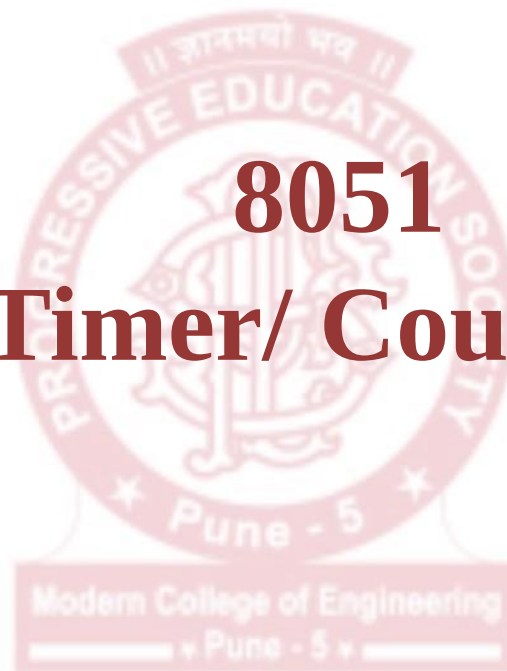
```
            break;
```

```
        }
```

```
    }
```

```
}
```

8051 Timer/ Counter

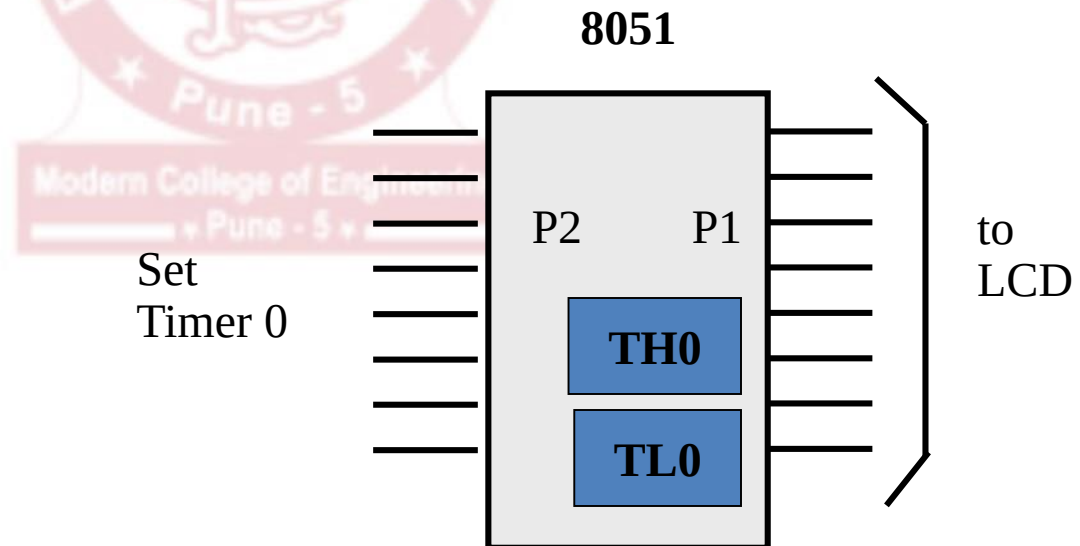


Timers /Counters Programming

- The 8051 has 2 timers/counters: timer/counter 0 and timer/counter 1. They can be used as
 1. The **timer** is used as a time delay generator.
 - The clock source is the **internal** crystal frequency of the 8051.
 2. An event **counter**.
 - **External input** from input pin to count the number of events on registers.
 - These clock pulses could represent the number of people passing through an entrance, or the number of wheel rotations, or any other event that can be converted to pulses.

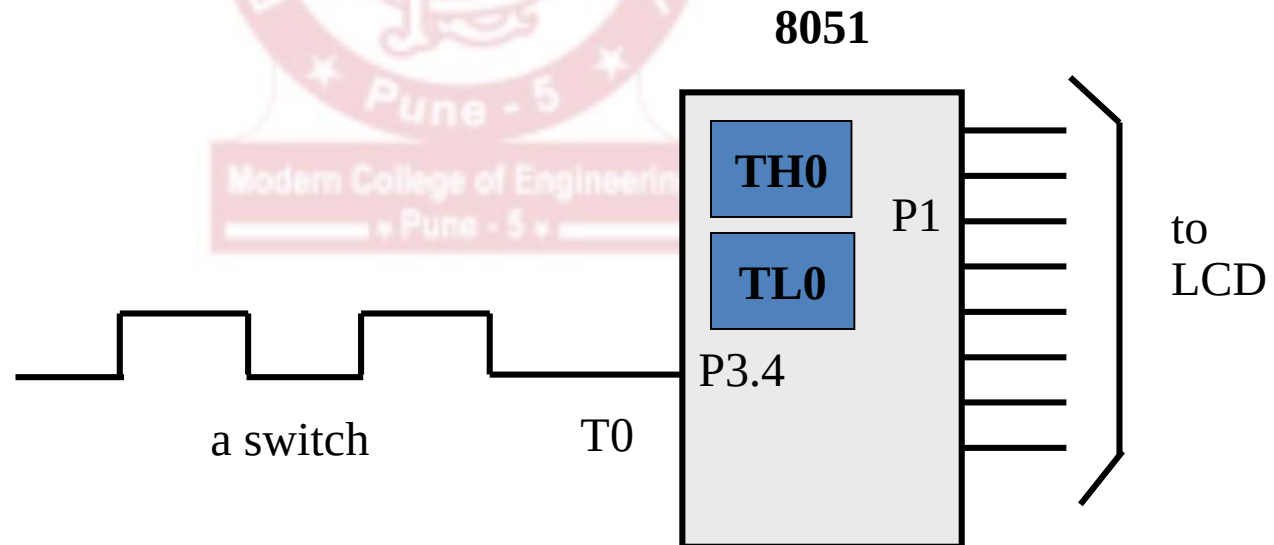
Timer

- Set the initial value of registers
- Start the timer and then the 8051 counts up.
- Input from internal system clock (machine cycle)
- When the registers equal to 0 and the 8051 sets a bit to denote time out



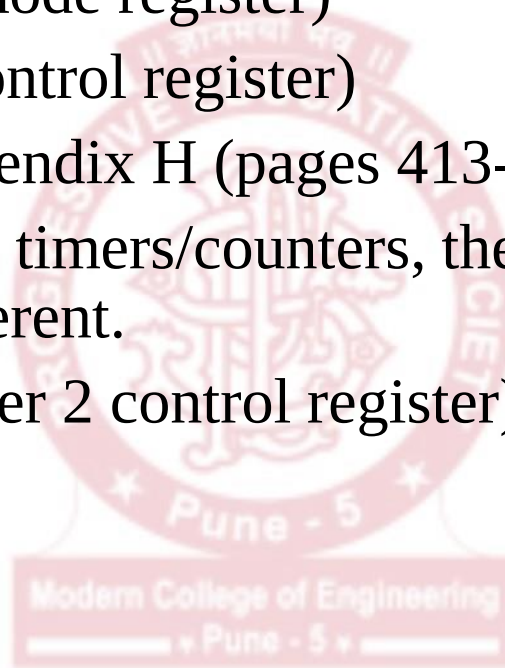
Counter

- Count the number of events
 - Show the number of events on registers
 - External input from T0 input pin (P3.4) for Counter 0
 - External input from T1 input pin (P3.5) for Counter 1
 - **External input** from Tx input pin.
 - We use Tx to denote T0 or T1.

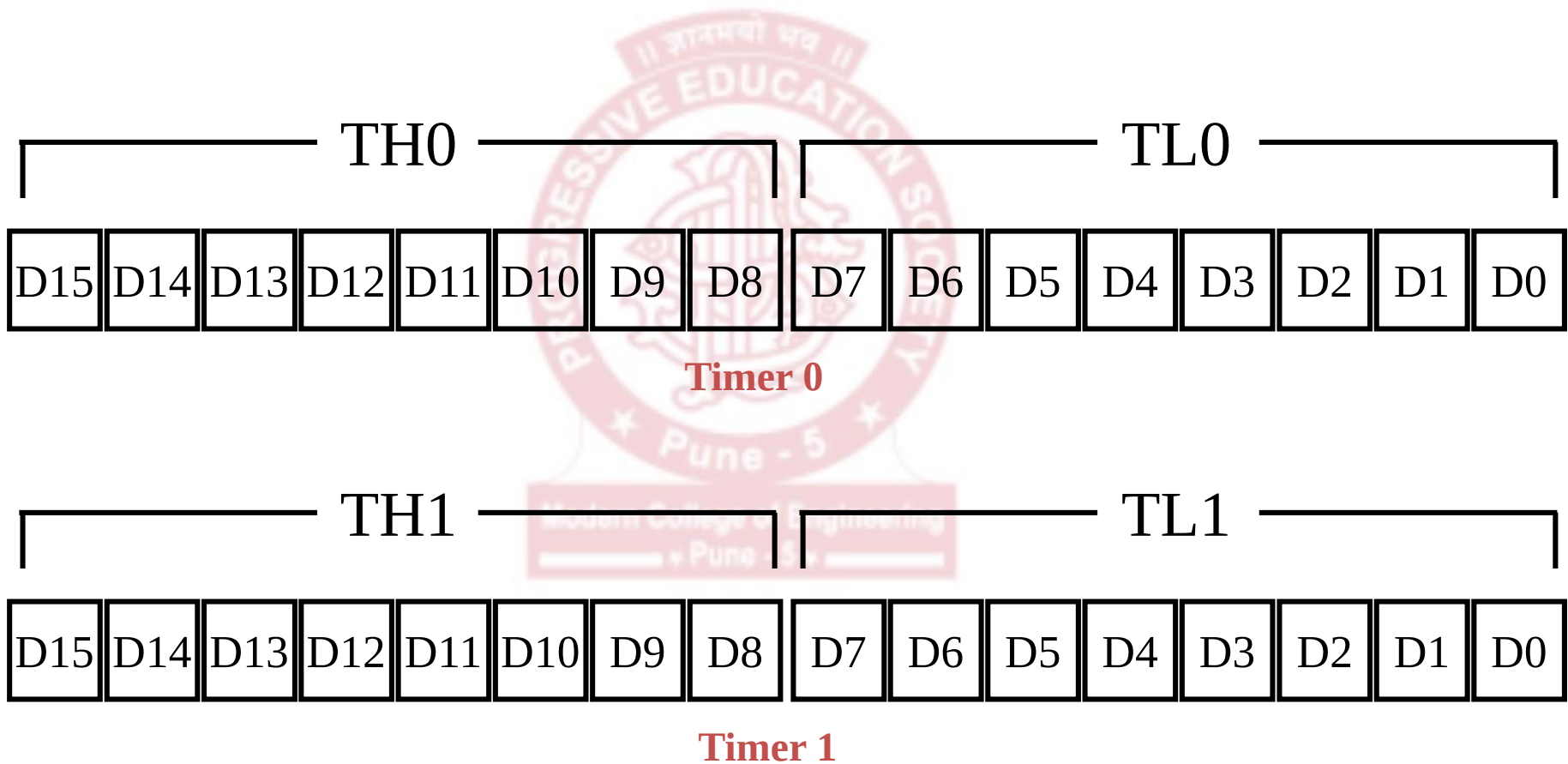


Registers Used in Timer/Counter

- TH0, TL0, TH1, TL1
- TMOD (Timer mode register)
- TCON (Timer control register)
- You can see Appendix H (pages 413-415) for details.
- Since 8052 has 3 timers/counters, the formats of these control registers are different.
 - T2CON (Timer 2 control register), TH2 and TL2 used for 8052 only.



Timer Registers



TMOD Register

(MSB)

(LSB)

GATE	C/T	M1	M0	GATE	C/T	M1	M0
Timer 1				Timer 0			

- Timer mode register: **TMOD**
MOV TMOD,#21H
 - An 8-bit register
 - Set the usage mode for two timers
 - Set lower 4 bits for Timer 0 (Set to 0000 if not used)
 - Set upper 4 bits for Timer 1 (Set to 0000 if not used)
 - Not bit-addressable

TMOD Register

(MSB)

(LSB)

GATE	C/T	M1	M0	GATE	C/T	M1	M0
Timer 1				Timer 0			

GATE Start and stop timer by using hardware or software.

1 start by hardware

0 start by software

C/T Timer or counter select bit

0 for timer operation (input from internal system clock).

1 for counter operation (input from Tx input pin).

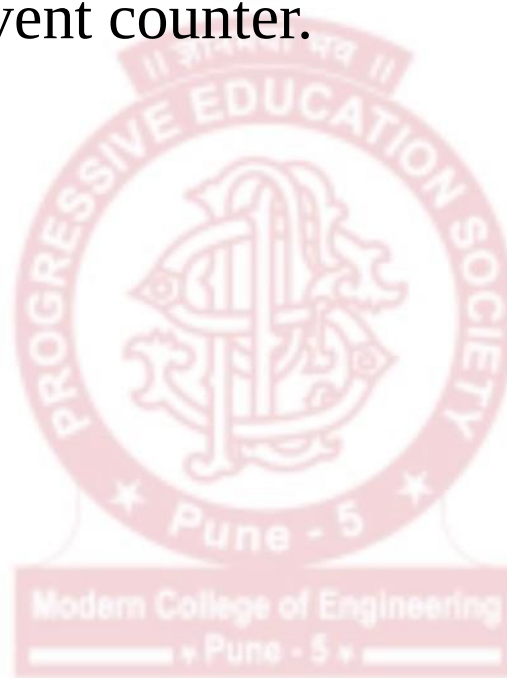
M1 M2 Mode select bit

Gate

- Every timer has a mean of starting and stopping.
 - GATE=0
 - **Internal** control
 - The start and stop of the timer are controlled by way of **software**.
 - Set/clear the TR bit from TCON register for start/stop timer.
 - GATE=1
 - **External** control
 - The hardware way of starting and stopping the timer by **software** and **an external source**.
 - Timer/counter is enabled only while the INT pin(external hardware interrupt pin of Port 3) is high and the TR control pin is set (TR from TCON register).

C/T (Clock/Timer)

- This bit is used to decide whether the timer is used as a delay generator or an event counter.
- $C/T = 0$: timer
- $C/T = 1$: counter



M1, M0

- M0 and M1 select the timer mode for timers 0 & 1.

M1 M0 Mode Operating Mode

0 0 **0** **13-bit timer** mode

8-bit TH_x + 5-bit TL_x (x= 0 or 1)

0 1 **1** **16-bit timer** mode

8-bit TH_x + 8-bit TL_x

1 0 **2** **8-bit auto reload**

8-bit auto reload timer/counter;

TH_x holds a value which is to be reloaded into TL_x each time it overflows.

1 1 **3** Split timer mode

D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	
																Mode 0
																Mode 1
																Mode 2
																Mode 3

PBS MCOF

Calculate

Find the value for TMOD if we want to program timer 0 in mode 2, use 8051 XTAL for the clock source, and use instructions to start and stop the timer.

Solution:

timer 1 **timer 0**

TMOD= 0000 0010 Timer 1 is not used.

 Timer 0, **mode 2**,

 C/T = 0 to use XTAL clock source (timer)

 gate = 0 to use internal (**software**)

 start and stop method.

Calculate

Indicate which mode and which timer are selected for each of the following.

- (a) MOV TMOD, #01H
- (b) MOV TMOD, #20H
- (c) MOV TMOD, #12H

Solution:

We convert the value from hex to binary.

- a) TMOD = 00000001, mode 1 of timer 0 is selected.
- b) TMOD = 00100000, mode 2 of timer 1 is selected.
- c) TMOD = 00010010, mode 2 of timer 0, and mode 1 of timer 1 are selected.

TCON Register

(MSB)

(LSB)

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
Timer 1 Timer0				for Interrupt			

- Timer control register: **TCON**
 - Upper nibble for timer/counter, lower nibble for interrupts
- **TR** (run control bit)
 - TR0 for Timer/counter 0; TR1 for Timer/counter 1.
 - TR is set by programmer to turn timer/counter on/off.
 - TR=0: off (stop)
 - TR=1: on (start)

TCON Register

- **TF** (timer flag, control flag)
 - TF0 for timer/counter 0; TF1 for timer/counter 1.
 - Originally, TF=0. When TH-TL roll over to 0000 from FFFFH, the TF is set to 1.
 - TF=0 : not reach
 - TF=1: reach
 - If we enable interrupt, TF=1 will trigger ISR.

(MSB)

(LSB)

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
Timer 1		Timer0		for Interrupt			

Equivalent Instructions for the Timer Control Register

For timer 0

SETB TR0	=	SETB TCON.4
----------	---	-------------

CLR TR0	=	CLR TCON.4
---------	---	------------

SETB TF0	=	SETB TCON.5
----------	---	-------------

CLR TF0	=	CLR TCON.5
---------	---	------------

For timer 1

SETB TR1	=	SETB TCON.6
----------	---	-------------

CLR TR1	=	CLR TCON.6
---------	---	------------

SETB TF1	=	SETB TCON.7
----------	---	-------------

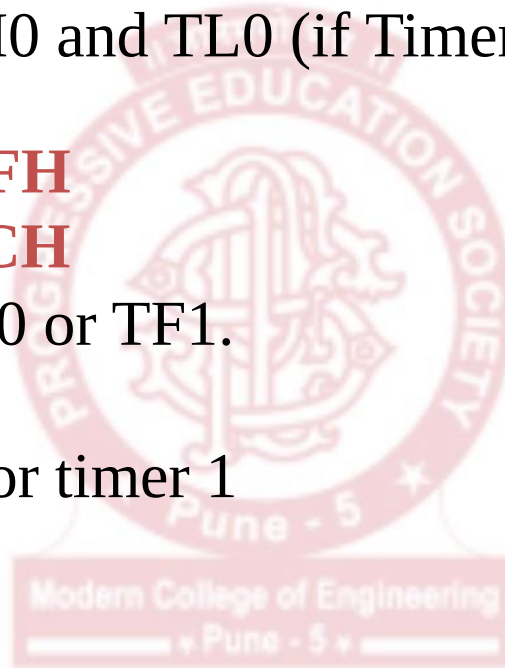
CLR TF1	=	CLR TCON.7
---------	---	------------

TCON: Timer/Counter Control Register

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
-----	-----	-----	-----	-----	-----	-----	-----

Steps of Mode 1 programming

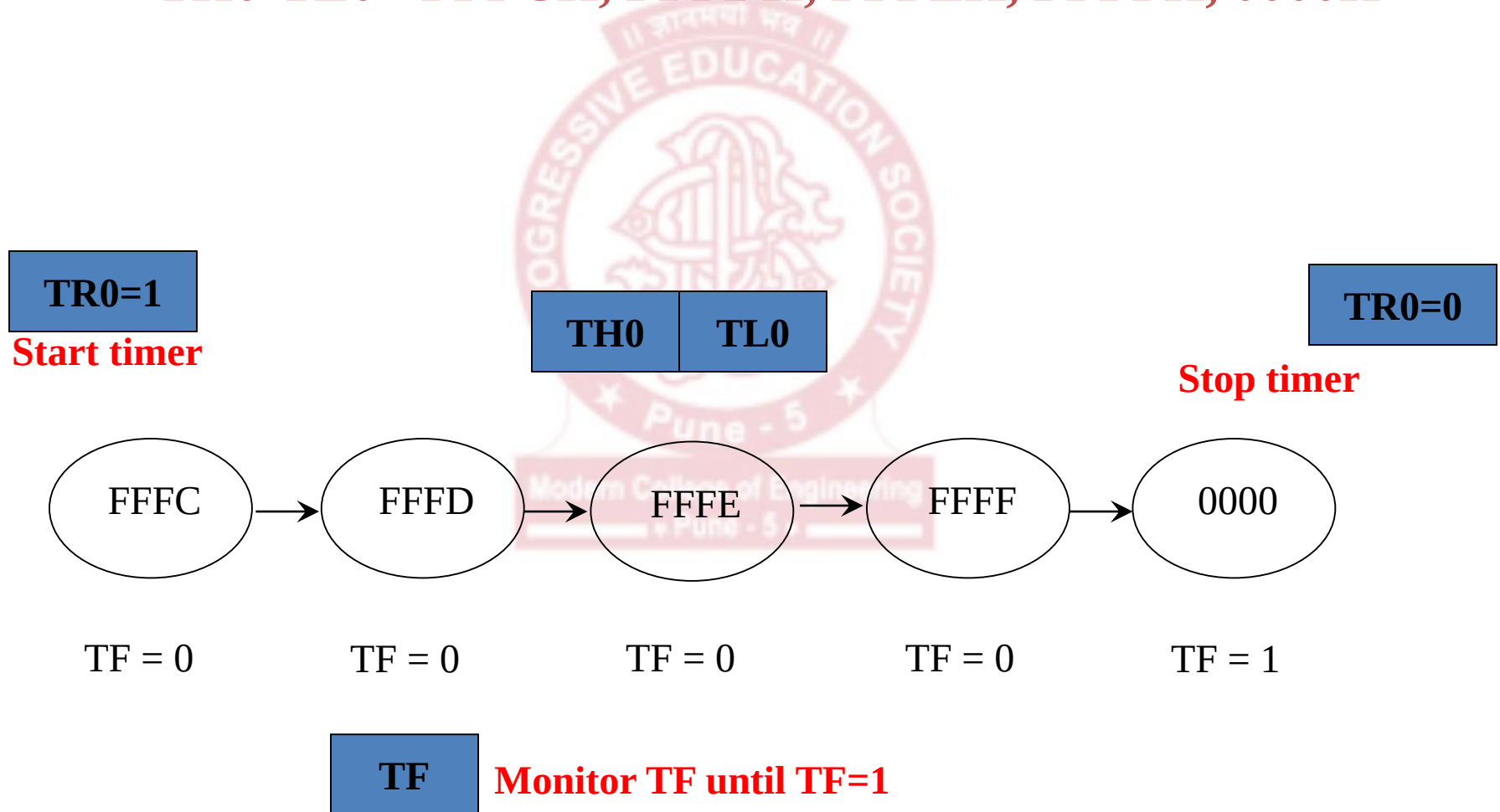
1. Load value in TMOD register to select mode 0 timer 0/timer 1
 - **MOV TMOD,#01H**
2. Load value in TH0 and TL0 (if Timer 0) or TH1 and TL1 (Timer 1).
 - **MOV TH0,#FFH**
 - **MOV TL0,#FCH**
3. clear the flag TF0 or TF1.
 - **CLR TF0**
4. Start the timer 0 or timer 1
 - **SETB TR0**



Steps of Mode 1

The 8051 starts to count up by incrementing the TH0-TL0.

– TH0-TL0= FFFCH, FFFDH, FFFE_H, FFFF_H, 0000_H



Steps of Mode 1

When TH0-TL0 rolls over from FFFFH to 0000, the 8051 set TF0=1.

TH0-TL0= FFFE H, FFFF H, 0000 H (Now TF0=1)

6. Keep monitoring the timer flag (TF) to see if it is raised.

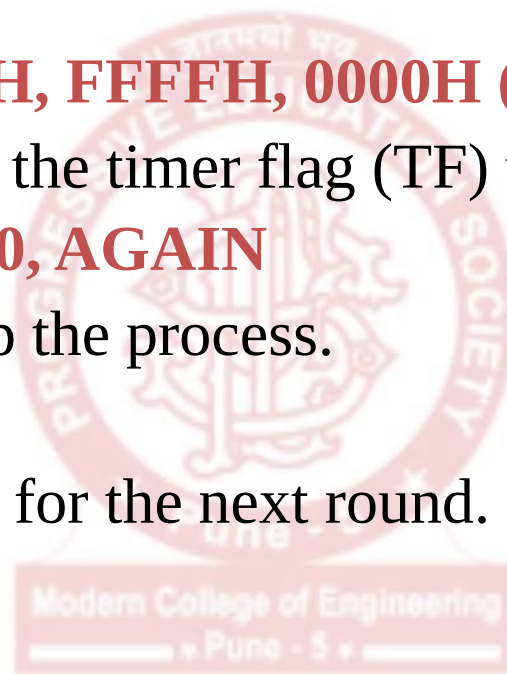
AGAIN: JNB TF0, AGAIN

7. Clear TR0 to stop the process.

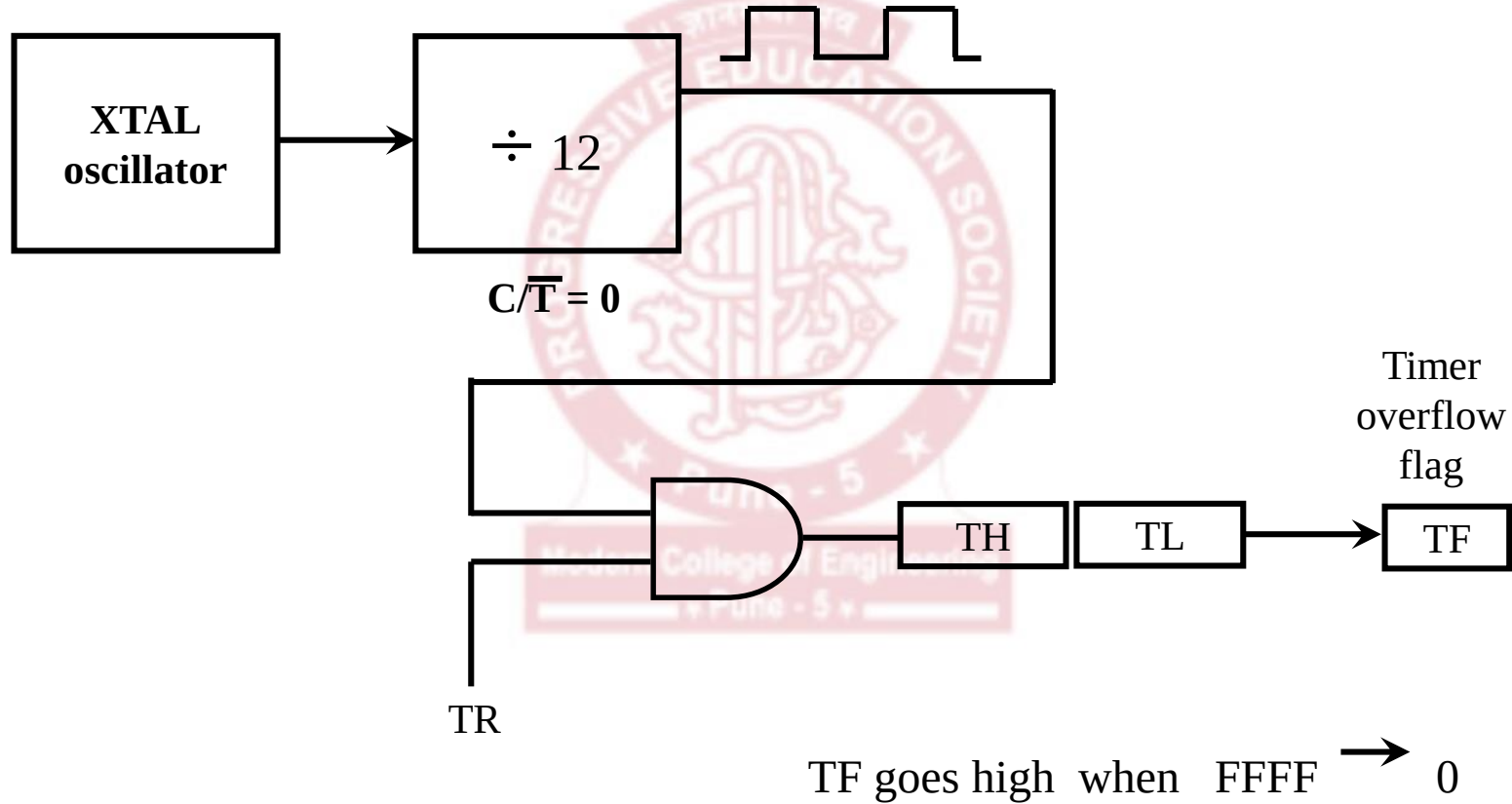
CLR TR0

8. Clear the TF flag for the next round.

CLR TF0



Mode 1 Programming



WAP to generate a square wave of 50% duty cycle on the P1.5 bit. Use timer 0 to generate the time delay

MOV TMOD,#01H ;Timer 0,mode 1(16-bit) –
;00000001 – 01H

HERE: MOV TL0,#0F2H ;Timer value = FFF2H

MOV TH0,#0FFH

SETB TR0

CPL P1.5

AGAIN: JNB TF0,AGAIN

CLR TR0 ;stop timer 0

CLR TF0 ;clear timer 0 flag

SJMP HERE

Explanation

- square wave of 50% duty on P1.5
- Timer 0 is used

;each loop is a half clock

MOV TMOD,#01 ;Timer 0,mode 1(16-bit)

HERE: MOV TL0,#0F2H ;Timer value = FFF2H

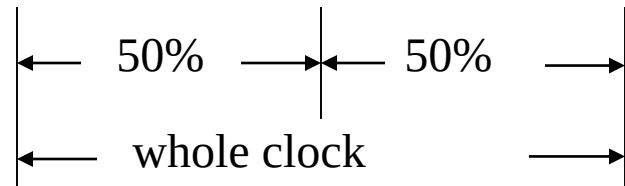
MOV TH0,#0FFH

CPL P1.5

ACALL DELAY

SJMP HERE

P1.5



Explanation

;generate delay using timer 0

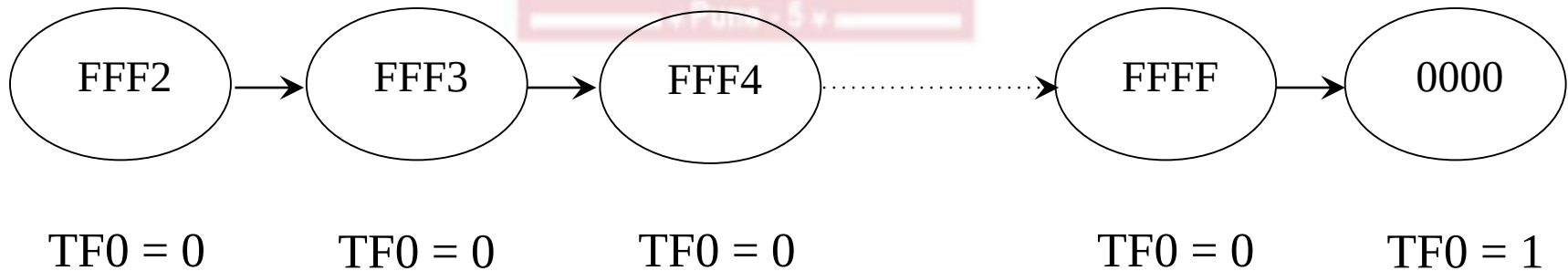
DELAY: SETB TR0 ;start the timer 0

AGAIN: JNB TF0, AGAIN

CLR TR0 ;stop timer 0

CLR TF0 ;clear timer 0 flag

RET



Solution:

In the above program notice the following steps.

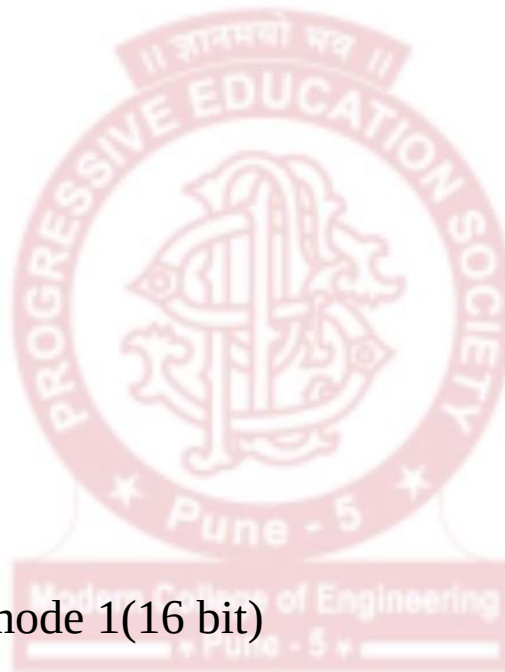
1. TMOD = **0000 0001** is loaded.
 2. **FFF2H** is loaded into TH0 – TL0.
 3. P1.5 is toggled for the high and low portions of the pulse.
 4. The DELAY subroutine using the timer is called.
 5. In the DELAY subroutine, timer 0 is started by the “**SETB TR0**” instruction.
 6. Timer 0 counts up with the passing of each clock, which is provided by the crystal oscillator.
- As the timer counts up, it goes through the states of FFF3, FFF4, FFF5, FFF6, FFF7, FFF8, FFF9, FFFA, FFFB, FFFC, FFFD, FFFE, FFFFH. One more clock rolls it to 0, raising the timer flag (**TF0 = 1**). At that point, the JNB instruction falls through.
7. Timer 0 is stopped by the instruction “**CLR TR0**”. The DELAY subroutine ends, and the process is repeated.

Notice that to repeat the process, we must reload the TL and TH registers, and start the timer again (in the main program).

Generation of square wave at P1.5 using timer 0

```
#include <reg51.h>
void Delay(void);
sbit output = P1^5;
void main(void)
{
while(1)
{
output =~output; Delay();
}

void Delay()
{
TMOD=0x01; // timer 0 ,mode 1(16 bit)
TL0=0xFD;// load TL0
TH0=0x4B; TR0=1;
while(TF0==0);// wait for TF0 to roll over
TR0=0;
TF0=0;
}
```



Timer Delay Calculation for XTAL = 11.0592 MHz

(a) in hex

- $(FFFF - YYXX + 1) \times 1.085 \mu s$
- where YYXX are TH, TL initial values respectively.
- Notice that values YYXX are in hex.

(b) in decimal

- Convert YYXX values of the TH, TL register to decimal to get a NNNNN decimal number
- then $(65536 - NNNNN) \times 1.085 \mu s$

Example

- This program generates a square wave on pin P1.5 Using timer 1
- Find the frequency generated by the timer XTAL = 11.0592 MHz

```
MOV TMOD,#10H ;timer 1, mode 1=00010000
AGAIN:MOV TL1,#34H ;timer value=7634H
MOV TH1,#76H
SETB TR1 ;start
BACK: JNB TF1,BACK
CLR TR1 ;stop
CPL P1.5 ;next half clock
CLR TF1 ;clear timer flag 1
SJMP AGAIN ;reload timer1
```

Solution:

$$(FFFF - YYXX + 1) \times 1.085 \mu s$$

$$FFFFH - 7634H + 1 = 89CCH = 35276 \text{ clock count}$$

$$\text{Half period} = 35276 \times 1.085 \mu s = 38.274 \text{ ms}$$

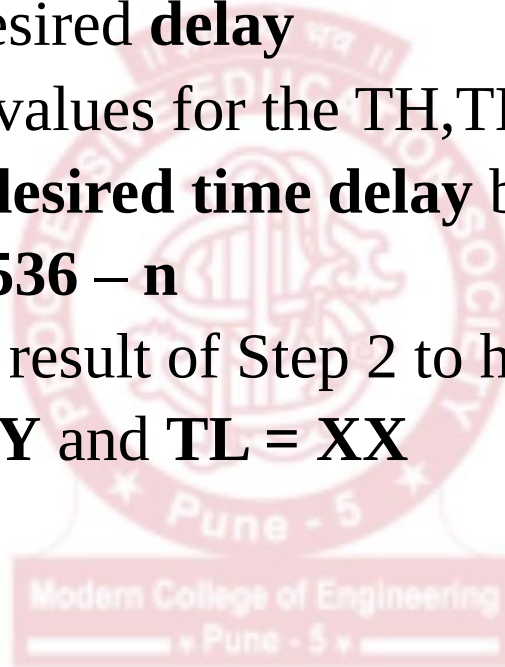
$$\text{Whole period} = 2 \times 38.274 \text{ ms} = 76.548 \text{ ms}$$

$$\text{Frequency} = 1 / 76.548 \text{ ms} = 13.064 \text{ Hz.}$$

Note: Mode 1 is not auto reload then the program must reload the TH1, TL1 register every timer overflow if we want to have a continuous wave.

Find Timer values when time is given

- Assume that XTAL = 11.0592 MHz .
- And we know desired **delay**
- How to find the values for the TH,TL ?
 1. Divide the **desired time delay** by **1.085 μ s** and get **n**.
 2. Perform **65536 – n**
 3. Convert the result of Step 2 to hex (**YYXX**)
 4. Set **TH = YY** and **TL = XX**



Find Timer values when frequency is given

- Assume that XTAL = 11.0592 MHz .
- And we know desired **delay**
- How to find the values for the TH,TL ?
 1. $T = 1/f$
 2. $\frac{1}{2}$ of it for high and low portion each
 3. Divide the **time delay** by **1.085 μ s** and get **n**.
 4. **N are the cycles.**
 5. Perform **65536 – n**
 6. Convert the result of Step 2 to hex (**YYXX**)
 7. Set **TH = YY** and **TL = XX**

Example

- Assuming XTAL = 11.0592 MHz,
- Write a program to generate a square wave of 50 Hz frequency on pin P2.3.

Solution:

1. The period of the square wave = $1 / 50 \text{ Hz} = 20 \text{ ms}$.
2. The high or low portion of the square wave = **10 ms**.
3. **$10 \text{ ms} / 1.085 \mu\text{s} = 9216$**
4. **$65536 - 9216 = 56320$** in decimal = **DC00H** in hex.
5. TL1 = 00H and TH1 = DCH.

Example

```
MOV TMOD,#10H    ;timer 1, mode 1
AGAIN: MOV TL1,#00    ;Timer value = DC00H
      MOV TH1,#0DCH
      SETB TR1        ;start
BACK:  JNB TF1,BACK
      CLR TR1         ;stop
      CPL P2.3
      CLR TF1         ;clear timer flag 1
      SJMP AGAIN      ;reload timer since mode 1 is not auto-
                      ;reload
```

Calculate values to be loaded in Timer Register

What value do we need to load into timer registers if we want to have a time delay of:

5 msec delay where crystal frequency is 11.0592 MHz

100 KHz delay where crystal frequency is 22 MHz

Freq = 100 kHz

Time = $1/100k = 0.01\text{msec} = \text{whole cycle}$

Half = $0.01\text{msec}/2 = 0.005 \text{ msec}$

$0.005\text{msec}/0.5454 \text{ microsec} = 9.1675 = 9$

Solution

Q1. 5 msec delay where crystal frequency is 11.0592 MHz

$$5 \text{ msec} / 1.085 \text{ usec} = 4608 \text{ clocks}$$

$$65536 - 4608 = 60928 = \text{EE00h}$$

Q2. 100 KHz delay where crystal frequency is 22 MHz.

$$T = 1/f = 1/100\text{KHz} = 0.01 \text{ msec} = 10 \text{ usec}$$

$$T/2 = 5 \text{ usec}$$

$$5 \text{ usec} / 0.546 \text{ usec} = 9 \text{ clocks}$$

$$65536 - \text{NNNNN} = 65536 - 9$$

$$65536 - 9 = 65527 = \text{FFF7h}$$

$$\text{TH} = \text{FFH} \text{ and } \text{TL} = \text{F7H}$$

Generate a Large Time Delay

- Cannot go beyond FFFFH, why?
- The size of the time delay depends on two factors:
 - The crystal frequency (11.0592 M, 16 M, 22 MHz)
 - The timer's 16-bit register, TH & TL
- The largest time delay is achieved by making TH=TL=0.

Highest delay using 0000 H

$$65536 - 0000 = 65536$$

$$65536 * 1.085 \mu s = 71.106 \text{ msec}$$

- What if that is not enough?
- Next Example show how to achieve large time delay

Example

Examine the following program and find the time delay in seconds.

MOV TMOD,#10H ; 00010000 = timer 1 and mode 1

UP: MOV R3,#200

AGAIN: MOV TL1,#08

MOV TH1,#01

SETB TR1

BACK: JNB TF1,BACK

CLR TR1

CLR TF1

DJNZ R3,AGAIN

CPL P1.6

SJMP UP

Solution:

TH – TL = 0108H = 264 in decimal

65536 – 264 = 65272.

timer delay = $65272 \times 1.085 \mu\text{s} = 70.820 \text{ ms}$

Total delay = $200 \times 70.820 \text{ ms} = 14.164024 \text{ seconds}$

highest delay using 0000 H

$65536 - 0000 = 65536;$

$65536 \times 1.085 \mu\text{s} = 71.106 \text{ msec}$

Exercise

- Write a ALP to generate a square wave with an ON time of 3 msec and OFF time of 10 msec on port 0. Assume XTAL frequency of 22 MHz.
- Assuming XTAL as 22 MHz, write an ALP to generate a pulse train of 2 sec period on pin P2.4. Use timer 1 in mode 1.

REMEMBER: The size of the time delay depends on two factors:

- The crystal frequency (11.0592 M, 16 M, 22 MHz)
- 11.0592 M – multiply/ divide by 1.085 μ s,
- 16 M – multiply/ divide by 0.75 μ s,
- 22 MHz – multiply/ divide by 0.546 μ s

Solution

Q1. Ton of 3 msec and Toff of 10msec

Calculation for 3msec

$$3 \text{ msec} / 0.546 \text{ usec} = 5494 \text{ clocks}$$

$$65536 - 5494 = 60042 = \text{EA8Ah}$$

Calculation for 10msec

$$10 \text{ msec} / 0.546 \text{ usec} = 18315 \text{ clock}$$

$$65536 - 18315 = 47221 = \text{B875h}$$

Q2. 2 seconds delay where crystal frequency is 22 MHz.

$65536 * 0.546 = 35.75 \text{ msec}$ when TH and TL are loaded
with 0000h

$$35.75 * 28 = 1001 \text{ msec}$$

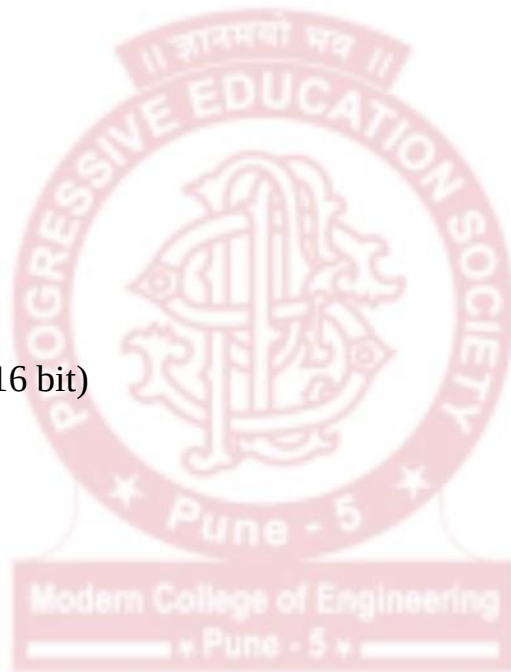
C programming mode 2

```
#include <reg51.h>
void Delay(void);
sbit output = P1^5;
void main(void)
{
while(1)
{
output =~output; Delay();
}
void Delay()
{
TMOD=0x20; // timer 1, mode 2(8 bit)
//TL0=0xFD; // load TL0 TH1=5;
TR1=1;
while(TF1==0); // wait for TF0 to roll over
TR1=0;
TF1=0;
}
```



Counter in mode2 to count up display the state of TL1 count

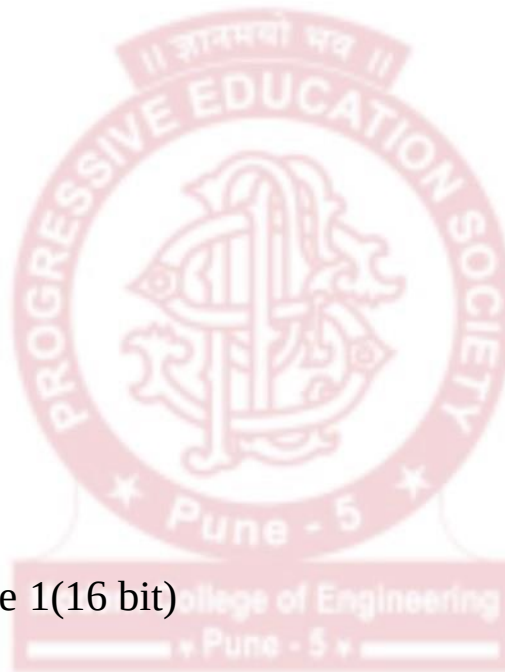
```
#include <reg51.h>
void Delay(void);
sbit output = P3^5;
unsigned char x;
void main(void)
{
while(1)
{
TMOD= 0x60; // timer 0 ,mode 1(16 bit)
//TL0=0xFD;// load TL0 TH1=00;
output=1;
TR1=1;
while(TF1==0)
{
x=TL1;
P2=x; // wait for TF0 to roll over
}
TR1=0;
TF1=0;
}
}
```



Write 8051 C program to toggle P1.5 continuously after 50ms

```
#include <reg51.h>
void delay(void);
sbit output = P1^5;
void main(void)
{
    While(1)
    {
        Output=~output; Delay()
    }

    void delay()
    {
        TMOD=0x01; // timer 0, mode 1(16 bit)
        TL0=0xFD; // load TL0
        TH0=0x4B;
        TR0=1;
        While(TF0==0); // wait for TF0 to roll over
        TR0=0;
        TF0=0;
    }
}
```



I/O ports

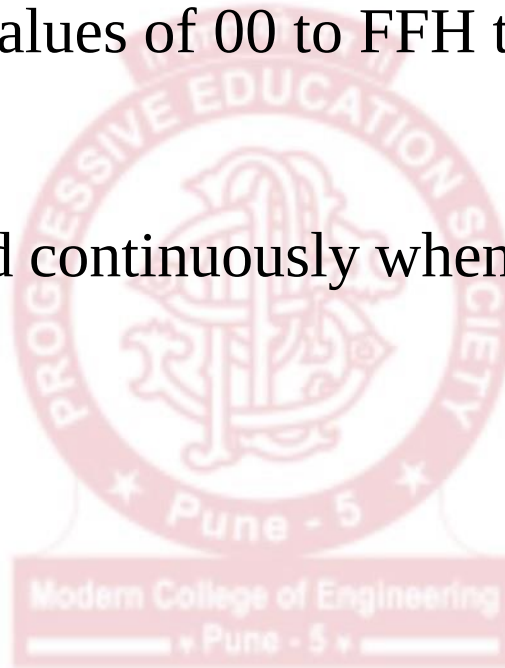
- PORT 0 required pull up registers
- Pins - Multifunction with address and data bus
- PORT1 has no multifunction pins
- PORT2 is multifunction with higher address bus
- PORT3 pins are multifunction with TxD, RxD T0 and T1
- If you want use port pin as other than function as port then these port of these pins is to be set(high). Hence in programs, port pin is set to 1 if it is to be used as Input pin

Timer Mode 0

- Mode 0 is exactly like mode 1 except that it is a **13-bit** timer instead of 16-bit.
 - 5-bit TH0
 - 8-bit TL0
- The counter can hold values between 0000 to 1FFF in TH0-TL0.
 - $2^{13}-1 = 2000H-1 = 1FFFH$
- We set the initial values TH0-TL0 to **count up**.
- When the timer reaches its maximum of 1FFFH, it rolls over to 0000, and TF0 is raised.

Timer Mode 2

- 8-bit timer.
 - It allows only values of 00 to FFH to be loaded into TH0.
- Auto-reloading
- TL0 is incremented continuously when TR0=1.



Steps of Mode 2

1. Choose mode 2 timer 0 or timer 1 using TMOD register

MOV TMOD,#02H

2. Set the original value to TH0 or TH1.

MOV TH0,#38H

3. Clear the flag to TF0=0 or TF1.

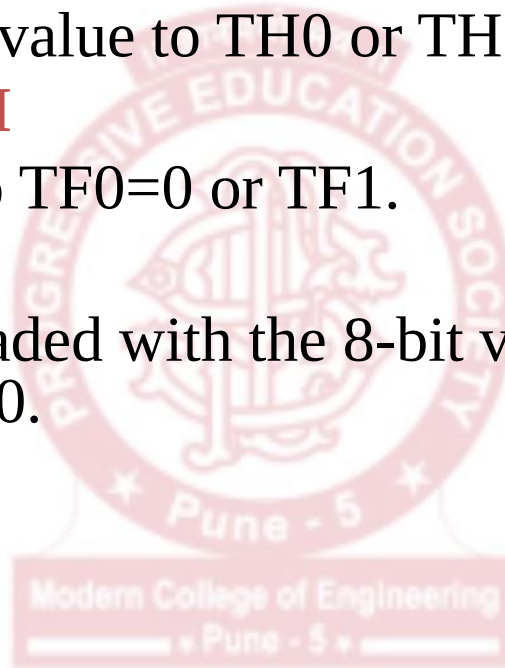
CLR TF0

4. After TH0 is loaded with the 8-bit value, the 8051 gives a copy of it to TL0.

TL0=TH0=38H

5. Start the timer.

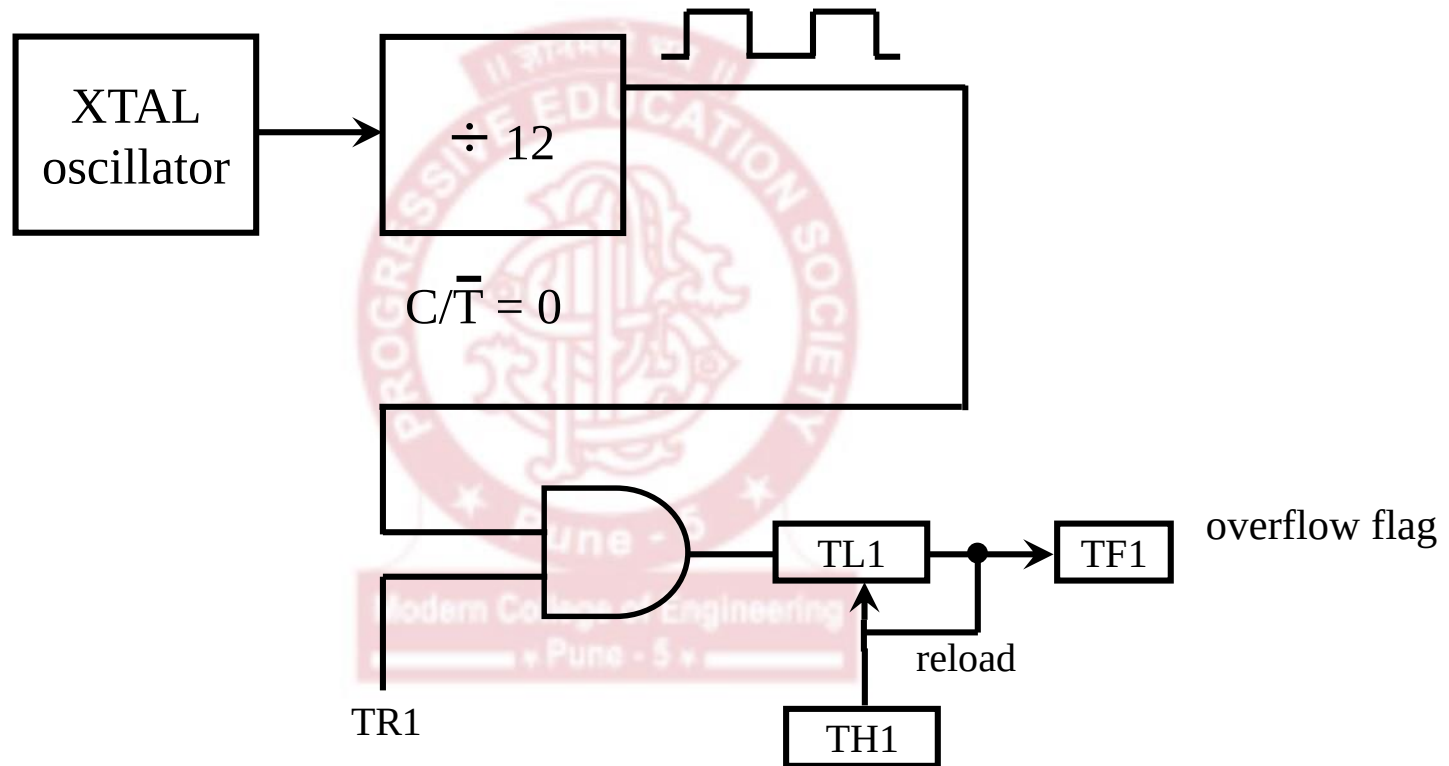
SETB TR0



Steps of Mode 2

6. The 8051 starts to count up by incrementing the TL0.
 - **TL0= 38H, 39H, 3AH,...FFH**
7. When TL0 rolls over from FFH to 00, the 8051 set TF0=1. Also, TL0 is reloaded automatically with the value kept by the TH0.
 - **TL0= FEH, FFH, 00H (Now TF0=1)**
 - The 8051 auto reload **TL0=TH0=38H.**
 - **Clr TF0**
 - Go to Step 6 (i.e., TL0 is incrementing continuously).
- Note that **we must clear TF0** when TL0 rolls over. Thus, we can monitor TF0 in next process.

Timer 1 Mode 2 with internal Input



TF goes high when FF $\rightarrow$ 0

Write a program to generate a square wave on pin P1.0. Program timer in mode 2

METHOD 1:

```
MOV TMOD,#20H ;Timer 0,mode 2
MOV TH0,#0
SETB TR1
HERE: JNB TF1,HERE
      CPL P1.0
      SJMP HERE
```

METHOD 2:

```
MOV TMOD,#20H ;Timer 0,mode 2
MOV TH0,#0
AGAIN: CPL P1.0
       ACALL DELAY
       SJMP AGAIN

DELAY: SETB TR1
HERE: JNB TF1,HERE
      RET
```

Time delay calculation for timer in mode 2 when freq is 11.592MHz

(a) in hex

- $(FF - XX + 1) \times 1.085 \mu s$
- where XX are TH initial values respectively.
- Notice that values XX are in hex.

(b) in decimal

- Convert XX values of the TH register to decimal to get a NNN decimal number
- then $(256 - NNN) \times 1.085 \mu s$

Example

If value loaded in TH register is 5 calculate the frequency. Assume XTAL as 11.059MHz.

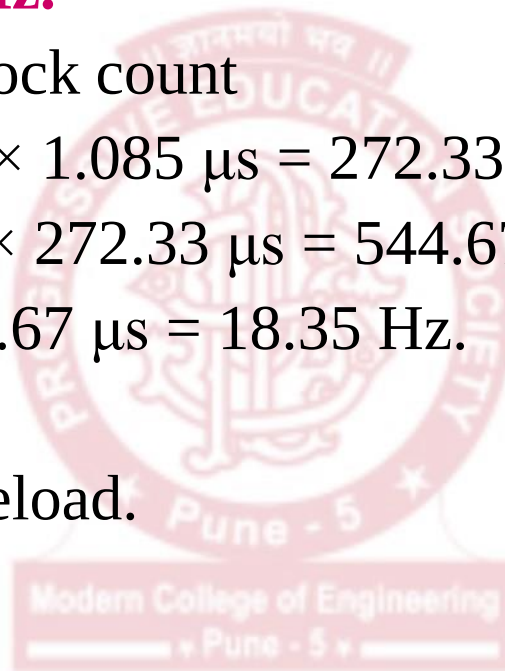
$$(256 - 05) = 251 \text{ clock count}$$

$$\text{Half period} = 251 \times 1.085 \mu\text{s} = 272.33 \mu\text{s}$$

$$\text{Whole period} = 2 \times 272.33 \mu\text{s} = 544.67 \mu\text{s}$$

$$\text{Frequency} = 1 / 544.67 \mu\text{s} = 18.35 \text{ Hz.}$$

Note: Mode 2 is auto reload.



Find Timer Values when frequency is given

- Assume that XTAL = 11.0592 MHz .
- And we know desired **delay**
- how to find the values for the TH?
 1. $T = 1/f$
 2. $\frac{1}{2}$ of it for high and low portion each
 3. Divide the **time delay** by **1.085 μ s** and get **n**.
 4. **N are the cycles.**
 5. Perform **256 – n**
 6. Convert the result of Step 2 to hex (**xx**)
 7. Set **TH = xx**

Example

- Assuming XTAL = 22MHz,
- write a program to generate a square wave of 1KHz frequency on pin P2.3.

Solution:

1. The period of the square wave = $1 / 1000 \text{ Hz} = 1 \text{ msec}$.
2. The high or low portion of the square wave = $1\text{m}/2 = 0.5\text{msec}$.
3. $0.5\text{ms} / 0.546 \mu\text{s} = 915 \text{ cycles}$ (915 is >>> 256)
4. 915 cycles can be achieved by dividing ($915/5 = 183 \text{ cycles}$) or $183*5 = 915$

Cross check

$$(183)*0.546*5 = 499\mu\text{sec} = 0.5\text{msec}$$

$$2*0.5\text{msec} = 1 \text{ msec} = 1\text{KHz}$$

Example

- WAP to generate a square wave of 50 Hz at pin P1.0. XTAL is 11.0592MHz

Solution:

```
MOV TMOD,#2H ;Timer 0,mode 2
MOV TH0,#0
AGAIN:MOV R5,#??? ;count ??? times
ACALL DELAY
CPL P1.0
SIMP AGAIN
```

```
DELAY:SETB TR0 ;start
BACK: JNB TF0,BACK ;wait until TL0 overflow auto-reload
CLR TR0 ;stop
CLR TF0 ;clear TF
DJNZ R5,DELAY
RET
```

$T = 2 (???? \times 256 \times 1.085 \mu s) = ????$, and frequency = 50 Hz.

Period = $1/50 = 0.02$, half period = $0.02/2 = 0.01$ seconds

$x \times 256 \times 1.085 = 0.01$

Example

Assuming that we are programming the timers for mode 2, find the value (in hex) loaded into TH for each of the following cases.

- (a) **MOV TH1,#-200** (b) **MOV TH0,#-60** (c) **MOV TH1,#-3**
(d) **MOV TH1,#-12** (e) **MOV TH0,#-48**

Solution:

Some 8051 assemblers provide this way.

1. Convert 200 decimal to binary.
2. Then take its 2's complement of that binary(which represents -200).
3. Convert the same to hex.(Its a negative hex)
4. Take 2's complement of that negative hex number.

Ex: $-200 = -C8H \Rightarrow 2\text{'s complement of } -200 = 100H - C8H = 38H$

Decimal	2's complement (TH value)
-200 = - C8H	38H
- 60 = - 3CH	C4H
- 3	FDH
- 12	F4H
- 48	D0H

Example

Find

(a) the frequency of the square wave generated in the following code

(b) the duty cycle of this wave.

```
MOV TMOD,#2H ;Timer 0,mode 2
```

```
MOV TH0,#-150 ;Count=150
```

```
AGAIN:SETB P1.3
```

```
ACALL DELAY
```

```
ACALL DELAY
```

```
CLR P1.3
```

```
ACALL DELAY
```

```
SJMP AGAIN
```

high period

low period

```
DELAY:SETB TR0 ;start
```

```
BACK: JNB TF0,BACK
```

```
CLR TF0 ;clear TF
```

```
RET
```

Example

Solution:

“**MOV TH0,#-150**” uses 150 clocks.

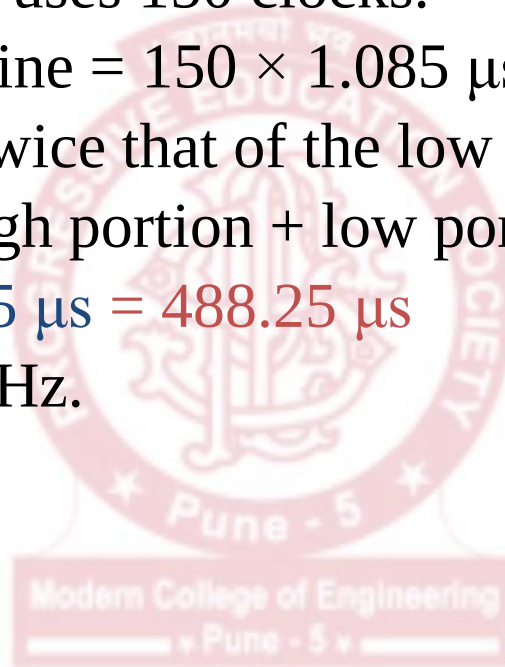
The DELAY subroutine = $150 \times 1.085 \mu\text{s} = 162.75 \mu\text{s}$.

The high portion is twice that of the low portion (66% duty cycle).

The total period = high portion + low portion

$$T = 325.5 \mu\text{s} + 162.75 \mu\text{s} = 488.25 \mu\text{s}$$

Frequency = 2.048 kHz.



Split timer mode

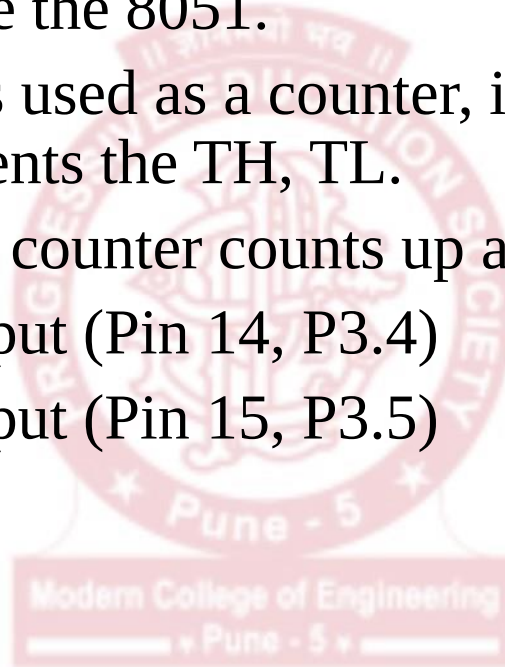
- This Mode is selected using TMOD register.
MOV TMOD,# 0010 0011b
- Mode3 is a split timer mode in which TL0 & TH0 acts as separate timer counting from 0-255.
- All the bits that are used to control timer1 are now tied to TH0.
- When the timer0 is in split mode the timer1 can be put into mode0,1,2 normally but we cannot start or stop this timer since its controlling bits are tied to TH0.
- In this case timer1 will be incremented every machine cycle.

The question which can come in mind is then what is the use of it?

Well it is helpful in baud rate generator.

Counter

- These timers can also be used as counters **counting events** happening outside the 8051.
- When the timer is used as a counter, it is a pulse outside of the 8051 that increments the TH, TL.
- When **C/T=1**, the counter counts up as pulses are fed from
 - T0: timer 0 input (Pin 14, P3.4)
 - T1: timer 1 input (Pin 15, P3.5)



Port 3 Pins Used For Timers 0 and 1

Pin	Port Pin	Function	Description
14	P3.4	T0	Timer/Counter 0 external input
15	P3.5	T1	Timer/Counter 1 external input

(MSB)

(LSB)

GATE	C/T=1	M1	M0	GATE	C/T=1	M1	M0
Timer 1				Timer 0			

Timer/Counter selection

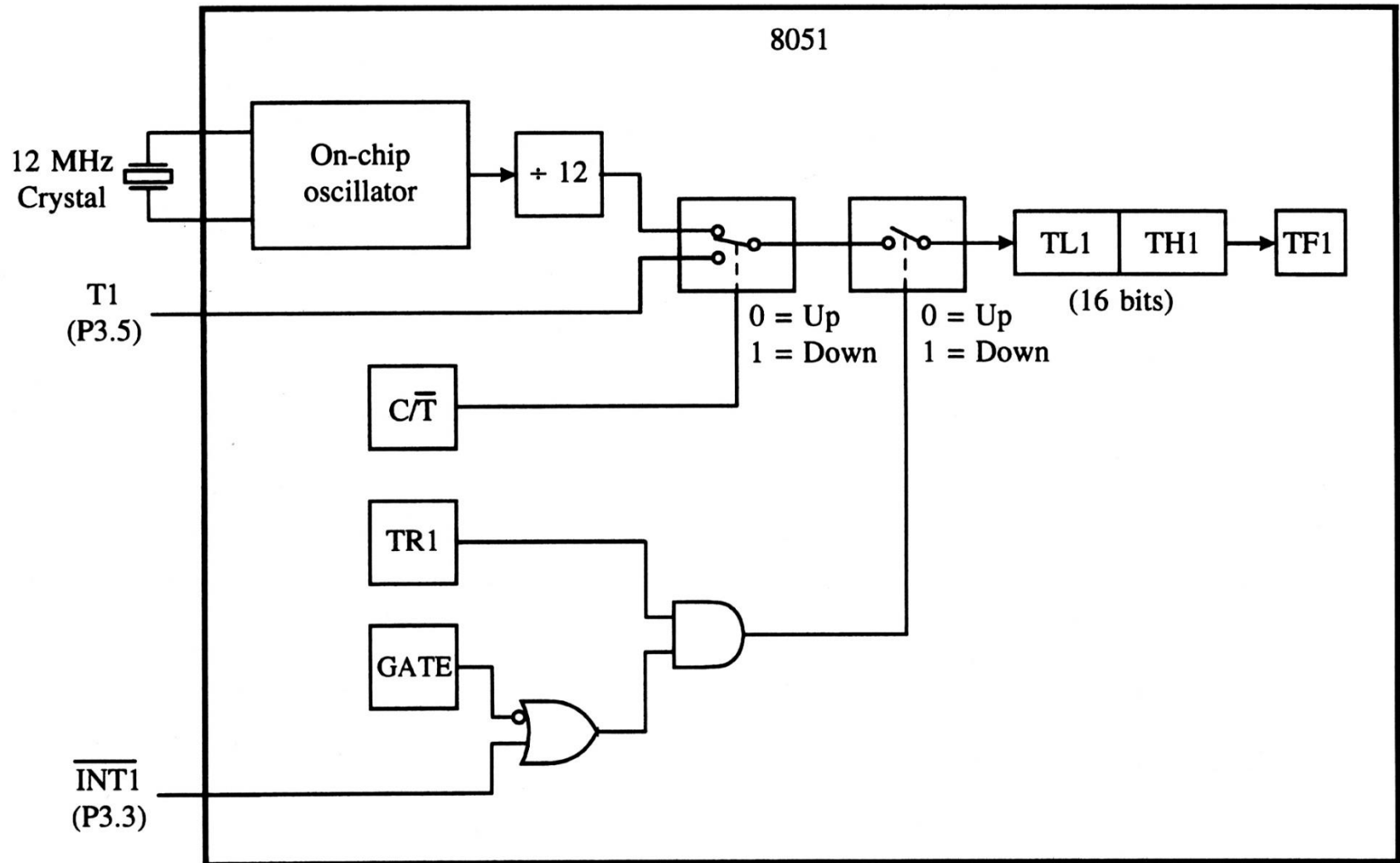


FIGURE 4-5

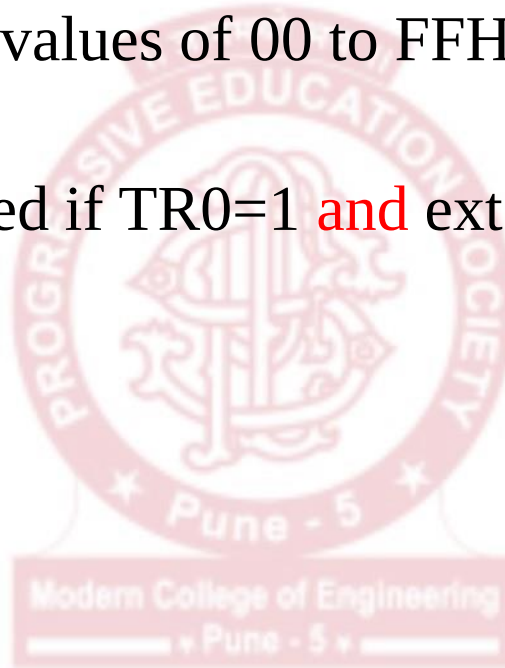
Timer 1 operating in mode 1

Counter Mode 1

- 16-bit counter (TH0 and TL0)
- TH0-TL0 is incremented when TR0 is set to 1 and an external pulse (in T0) occurs.
- When the counter (TH0-TL0) reaches its maximum of FFFFH, it rolls over to 0000, and TF0 is raised.
- Programmers should monitor TF0 continuously and stop the counter 0.
- Programmers can set the initial value of TH0-TL0 and let TF0=1 as an indicator to show a special condition. (ex: 100 people have come).

Counter Mode 2

- 8-bit **counter**.
 - It allows only values of 00 to FFH to be loaded into TH0.
- Auto-reloading
TL0 is incremented if TR0=1 **and** external pulse occurs.



Example

Assuming that people are entering the room. WAP for counter 1 in mode 2 to count the no of people and display the count of the TL 1 count on P2.

Solution:

MOV TMOD,#01100000B ;mode 2, counter 1

MOV TH1,#0

SETB P3.5 ;make T1 input port

AGAIN:SETB TR1 ;start

BACK: MOV A,TL1

MOV P2,A ;display in P2

JNB TF1,BACK ;overflow

CLR TF1 ;make TF=0

SJMP AGAIN ;keep doing



8051 Interrupts

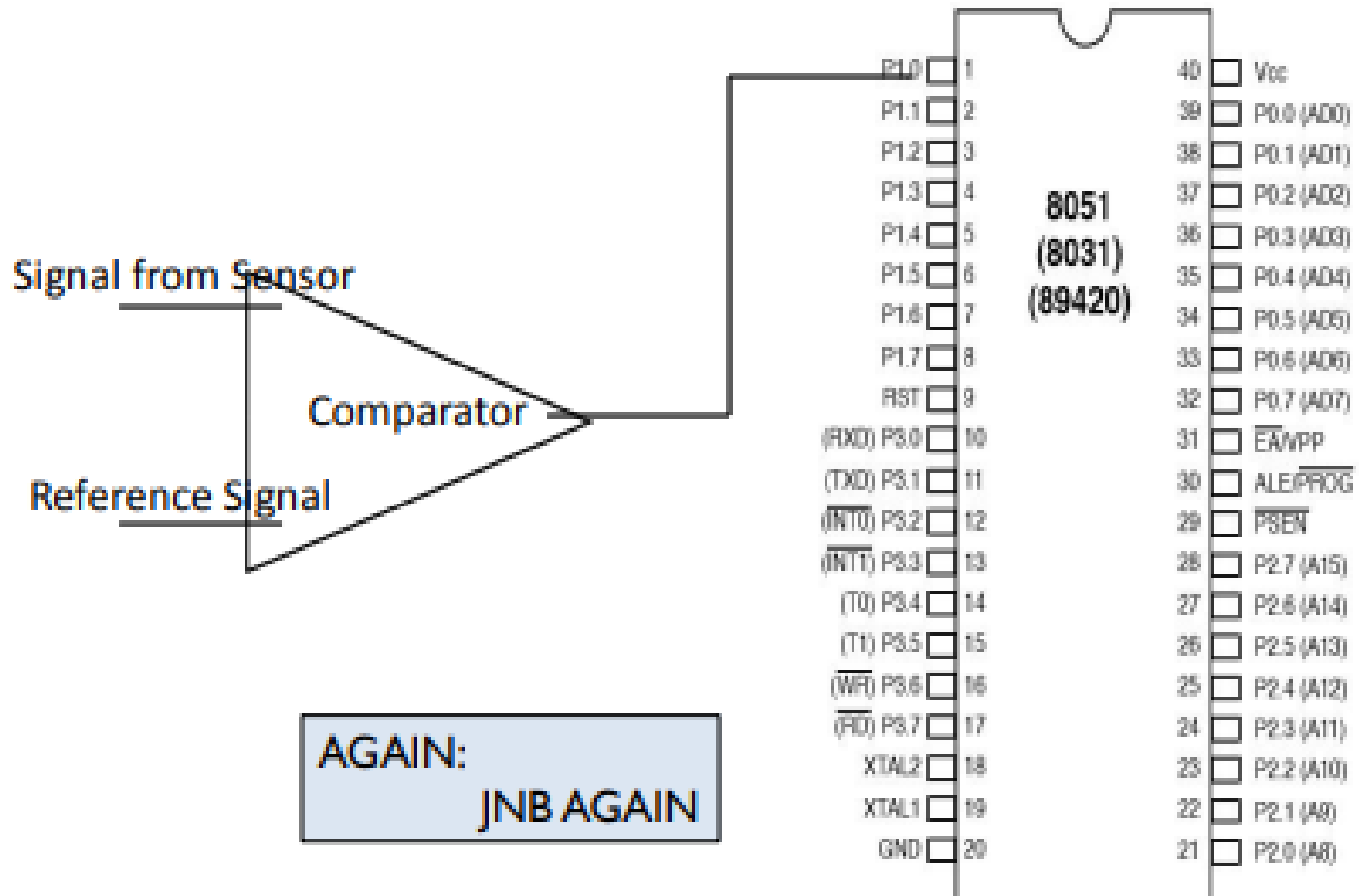
Interrupts Programming

- An *interrupt* is an external or internal event that interrupts the microcontroller to inform it that a device needs its service.

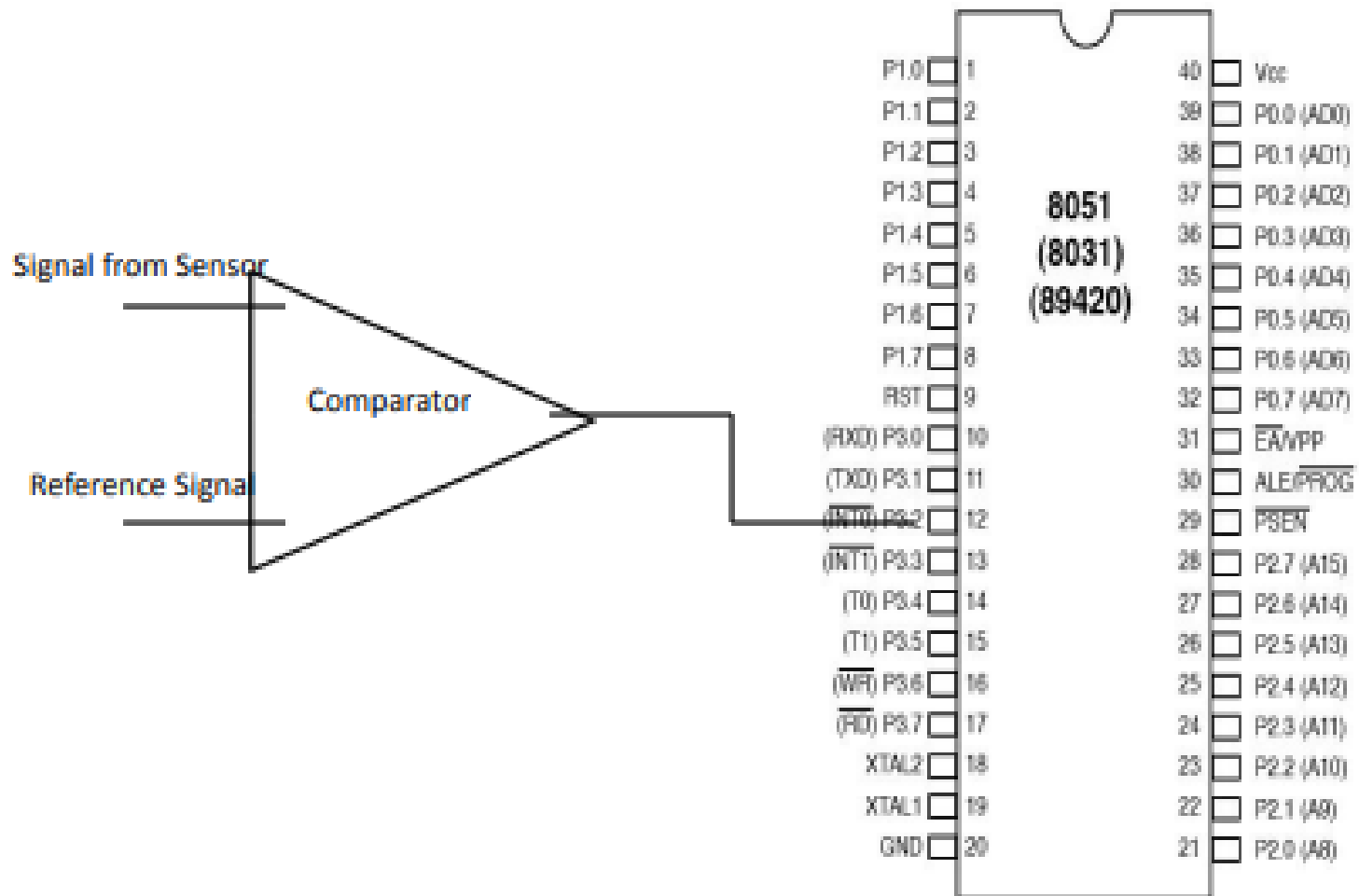
Interrupts vs. Polling

- A single microcontroller can serve *several* devices.
- There are *two* ways to do that:
 - interrupts
 - polling.
- The program which is associated with the interrupt is called the *interrupt service routine* (ISR) or *interrupt handler*.

Polling



Interrupt



Interrupt Vs Polling

Interrupt

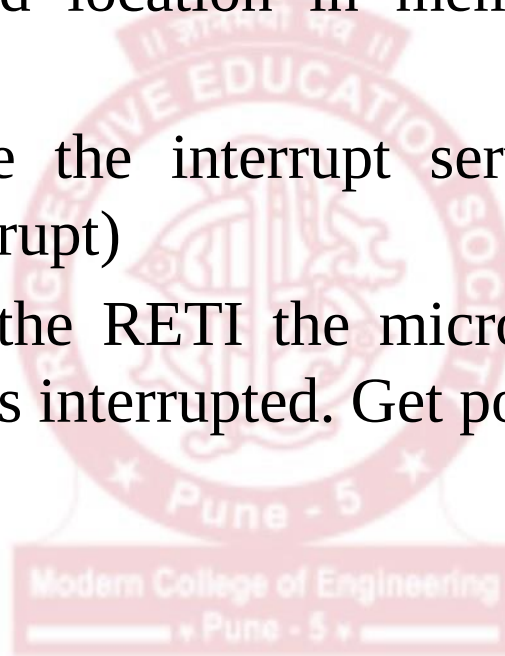
- It is a process with the help of which the CPU is notified of requiring attention.
- It is considered as a hardware mechanism.
- An interrupt handler services/works with the device.
- Interrupt-request line indicates that the device needs to be serviced.
- CPU is used only when a device requires servicing.
- This, in turn, saves the CPU cycles.
- An interrupt can occur at any point in time.
- It becomes inefficient if devices frequently interrupt the CPU.

Polling

- The process in which the CPU constantly checks the status of the device- to see if it needs the CPU's attention.
- It is a protocol. In this protocol, the CPU services the device.
- The command-ready bit indicates that the device needs to be serviced.
- The CPU needs to wait and check if a device needs to be serviced.
- This wastes many of the CPU cycles. CPU polls the devices at regular intervals of time.
- This protocol becomes inefficient when the CPU rarely finds a device that is ready to be serviced.

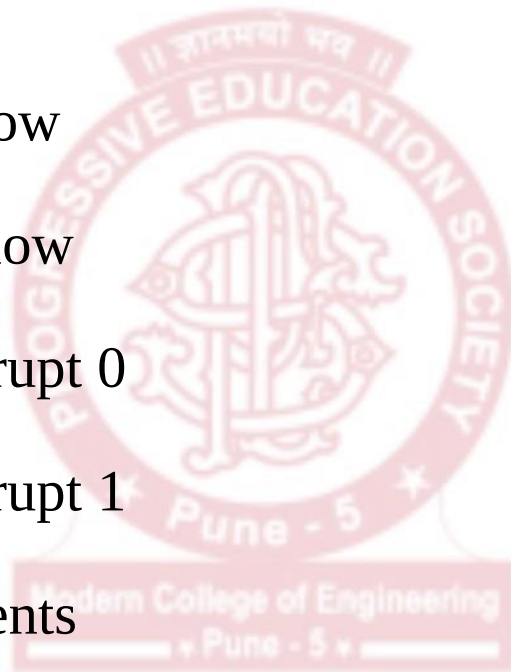
Steps in executing an interrupt

- Finish current instruction and saves the PC on stack.
- Jumps to a fixed location in memory depend on type of interrupt.
- Starts to execute the interrupt service routine until RETI (return from interrupt)
- Upon executing the RETI the microcontroller returns to the place where it was interrupted. Get pop PC from stack



Interrupt Sources

- Original 8051 has 6 sources of interrupts
 - Reset
 - Timer 0 overflow
 - Timer 1 overflow
 - External Interrupt 0
 - External Interrupt 1
 - Serial Port events



Interrupt Vector Table (IVT)

Each interrupt has a **specific** place in **code** memory where program execution (interrupt service routine) begins.

External Interrupt 0: 0003h

Timer 0 overflow: 000Bh

External Interrupt 1: 0013h

Timer 1 overflow: 001Bh

Serial : 0023h

Timer 2 overflow(8052+): 002bh

Note: that there are only 8 memory locations between vectors.

ISRs and Main Program in 8051

SJMP **main**

ORG **03H**

ljmp **int0**

ORG **0BH**

ljmp **t0**

ORG **13H**

ljmp **int1**

ORG **1BH**

ljmp **t1**

ORG **23H**

ljmp **serial**

ORG **30H**

main:

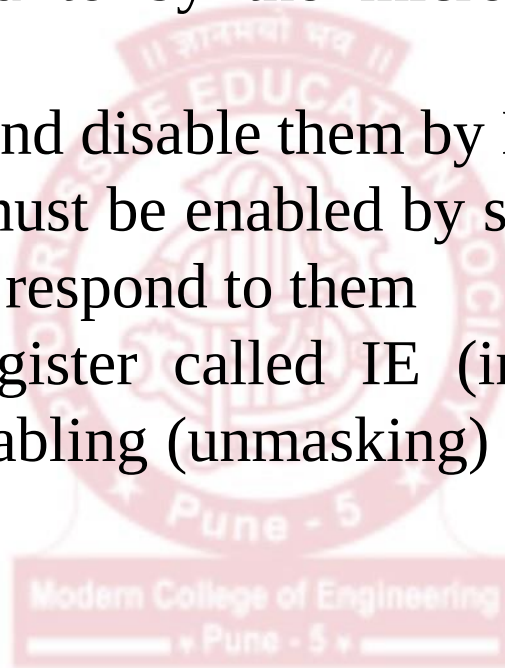
...

END



Interrupt Enable (IE) register

- ☐ All interrupt are disabled after reset, meaning that none will be responded to by the microcontroller if they are activated
- ☐ We can enable and disable them by IE.
- ☐ The interrupts must be enabled by software in order for the microcontroller to respond to them
- ☐ There is a register called IE (interrupt enable) that is responsible for enabling (unmasking) and disabling (masking) the interrupts.



Interrupt Enable (IE) register

D7

D0

EA	--	ET2	ES	ET1	EX1	ET0	EX0
----	----	-----	----	-----	-----	-----	-----

EA	IE.7	Disables all interrupts.
--	IE.6	No implemented, reserved for future use
ET2	IE.5	Enables or disables timer 2 overflow interrupt
ES	IE.4	Enables or disables the serial port interrupt
ET1	IE.3	Enables or disables timer 2 overflow interrupt
EX1	IE.2	Enables or disables external interrupt 1
ET0	IE.1	Enables or disables timer 0 overflow interrupt
EX0	IE.0	Enables or disables external interrupt

Enabling and disabling an interrupt

- ❑ To enable an interrupt, we take the following steps:
 1. Bit D7 of the IE register (EA) must be set to high to allow the rest of register to take effect
 2. The value of EA
 - If $EA = 1$, interrupts are enabled and will be responded to if their corresponding bits in IE are high
 - If $EA = 0$, no interrupt will be responded to, even if the associated bit in the IE register is high

Enabling and disabling an interrupt

- ❑ By bit operation
- ❑ Recommended in the middle of program

SETB EA	SETB IE.7	;Enable All
SETB ET0	SETB IE.1	;Enable Timer0 overflow
SETB ET1	SETB IE.3	;Enable Timer1 overflow
SETB EX0	SETB IE.0	;Enable INT0
SETB EX1	SETB IE.2	;Enable INT1
SETB ES	SETB IE.4	;Enable Serial port

- ❑ By mov instruction
- ❑ Recommended in the first of program

MOV IE, #10010110B

Timer Interrupts

- The timer flag (TF) is raised when the timer rolls over
- Polling – In polling, we have to wait until the TF is raised. The problem with this method is that the microcontroller is tied down while waiting for TF to be raised, and can not do anything else.
- Using interrupts solves this problem and, avoids tying down the controller
- If the timer interrupt in the IE register is enabled, whenever the timer rolls over, TF is raised, and the microcontroller is interrupted in whatever it is doing, and jumps to the interrupt vector table to service the ISR
- In this way, the microcontroller can do other until it is notified that the timer has rolled over

Steps for programming

- Load value in TMOD register for timer and its mode
- Load value in TH register
- Load value in IE register
- Start the timer.
- The moment timer finishes its job it will interrupt the controller in whatever it is doing.

The ISR routine should have program that controller will execute after it gets interrupted.

Example

- A 10khz square wave with 50% duty cycle at P0
- (2 tasks – Micro is generating square wave at P0, when timer interrupt occurs, port P1 displays a square wave of 10 KHz freq)

```
ORG    0           ;Reset entry point
LJMP   MAIN        ;Jump above interrupt

ORG    000BH       ;Timer 0 interrupt vector
CPL    P1.0        ;Toggle port bit
RETI                   ;Return from ISR to Main program
```

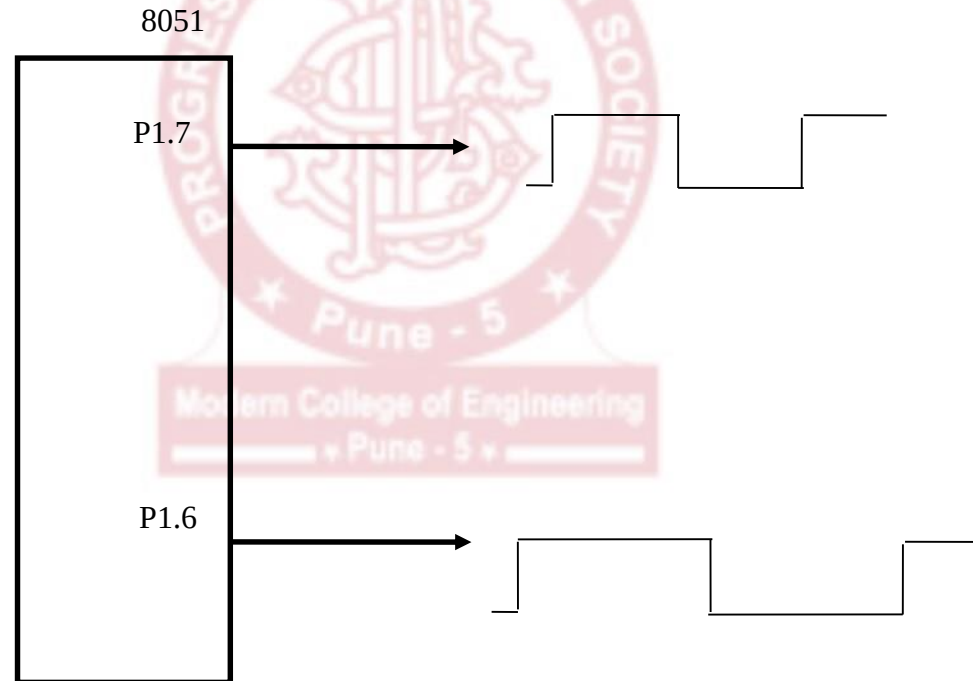
```
ORG 0030H           ;Main Program entry point
MAIN: MOV    TMOD,#02H ;Timer 0, mode 2
      MOV    TH0,#-50   ;50 us delay
      SETB   TR0        ;Start timer
      MOV    IE,#82H    ;Enable timer 0 interrupt
      MOV    P0,#00H
      ACALL  DELAY
      MOV    P0,#0FFH
      ACALL  DELAY
      END
```

Don't's: when using
timer interrupt in mode
2

```
CLR TRO
CLR TF0
```

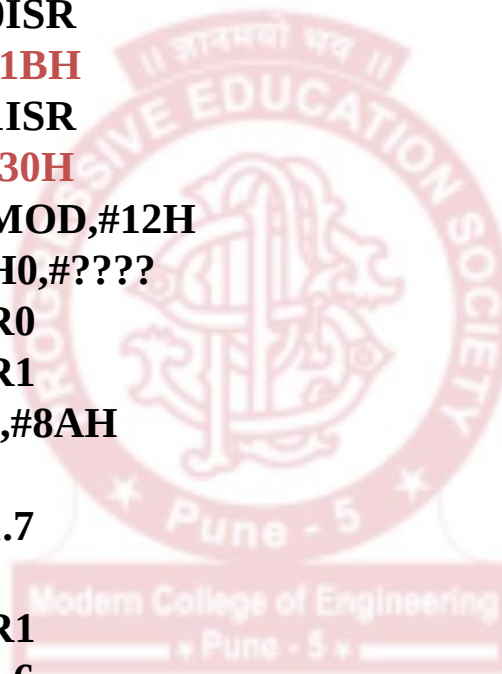
Example

- Write a program using interrupts to simultaneously create 7 kHz and 500 Hz square waves on P1.7 and P1.6.



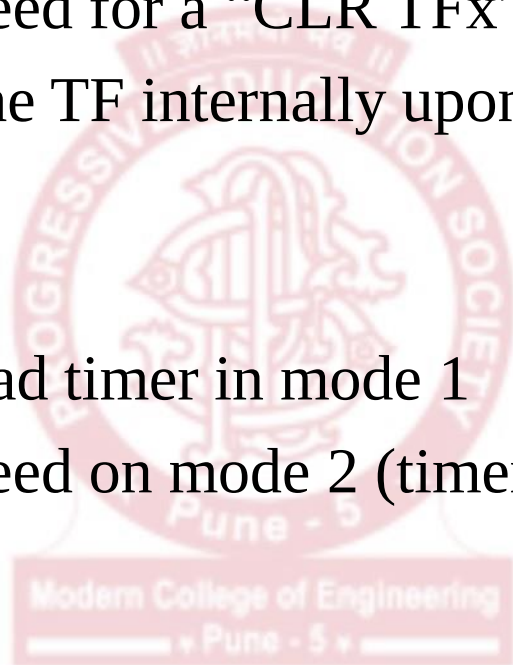
Solution

```
ORG 0
LJMP MAIN
ORG 000BH
LJMP T0ISR
ORG 001BH
LJMP T1ISR
ORG 0030H
MAIN: MOV TMOD,#12H
      MOV TH0,#???
      SETB TR0
      SETB TR1
      MOV IE,#8AH
      SJMP $
T0ISR: CPL P1.7
      RETI
T1ISR: CLR TR1
      CPL P1.6
      MOV TH1,# ???
      MOV TL1,# ???
      SETB TR1
      RETI
      END
```



Timer ISR

- Notice that
 - There is no need for a “CLR TFx” instruction in timer ISR
 - 8051 clears the TF internally upon jumping to ISR
- Notice that
 - We must reload timer in mode 1
 - There is no need on mode 2 (timer auto reload)



External hardware interrupts

- The 8051 has two external hardware interrupts
- Pin 12 (P3.2) and pin 13 (P3.3) of the 8051, designated as INT0 and INT1, are used as external hardware interrupts
- The interrupt vector table locations 0003H and 0013H are set aside for INT0 and INT1
- There are two activation levels for the external hardware interrupts
 - Level triggered interrupt
 - Edge triggered interrupt
- TCON register is used for edge triggered hardware interrupt
- Both the interrupts need IE register to enable the interrupt.

Level triggered mode

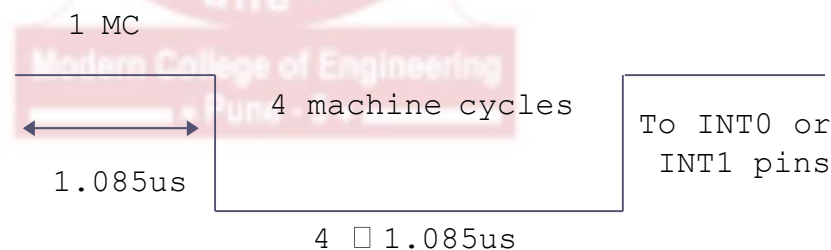
- In the level-triggered mode, INT0 and INT1 pins are normally high
- If a low-level signal is applied to them, it triggers the interrupt
- Then the microcontroller stops whatever it is doing and jumps to the interrupt vector table to service that interrupt
- The low-level signal at the INT pin must be removed before the execution of the last instruction of the ISR, RETI; otherwise, another interrupt will be generated
- This is called a level-triggered or level-activated interrupt and is the default mode upon reset of the 8051

Sampling Low Level Triggered Interrupt

- Pins P3.2 and P3.3 are used for normal I/O unless the INT0 and INT1 bits in the IE register are enabled
- After the hardware interrupts in the IE register are enabled, the controller keeps sampling the INTn pin for a low-level signal once each machine cycle
- According to one manufacturer's data sheet,
 - The pin must be held in a low state until the start of the execution of ISR
 - If the INTn pin is brought back to a logic high before the start of the execution of ISR there will be no interrupt
 - If INTn pin is left at a logic low after the RETI instruction of the ISR, another interrupt will be activated after one instruction is executed

Sampling Low Level Triggered Interrupt

- To ensure the activation of the hardware interrupt at the INTn pin, make sure that the duration of the low-level signal is around 4 machine cycles, but no more
- This is due to the fact that the level-triggered interrupt is not latched
- Thus the pin must be held in a low state until the start of the ISR execution



note: On reset, IT0 (TCON.0)
and IT1 (TCON.2) are both
low, making external interrupt
level-triggered

Write a program to generate a square wave continuously at P2.2 pin and on the occurrence of external hardware interrupt flash LED's at Port 1.



???

External interrupt type control

- By low nibble of Timer control register **TCON**
- **IE0 (IE1)**: External interrupt 0(1) edge flag.
 - **set** by CPU when external interrupt edge (**H-to-L**) is detected.
 - Does not affected by H-to-L while ISR is executed(no int on int)
 - **Cleared** by CPU when **RETI** executed.
 - does **not** latch low-level triggered interrupt
- **IT0 (IT1)**: interrupt 0 (1) type control bit.
 - Set/cleared by software
 - IT=1 edge trigger
 - IT=0 low-level trigger

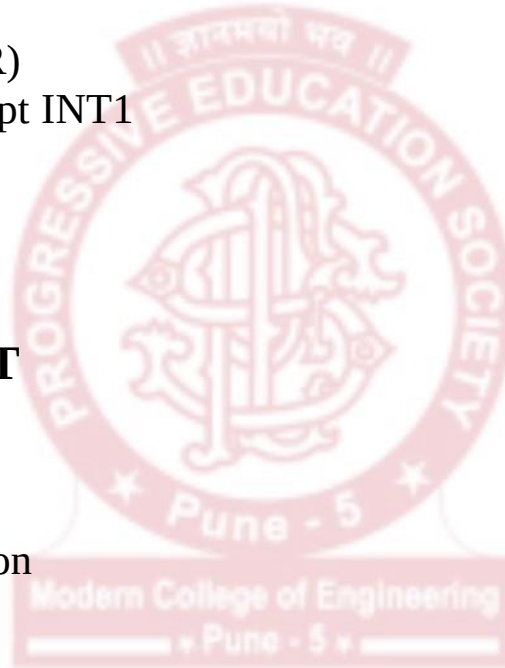
(MSB)

(LSB)

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
Timer 1 Timer0				for Interrupt			

Example of edge triggered external interrupt

```
ORG 0000H
LJMP MAIN
;
;interrupt service routine (ISR)
;for hardware external interrupt INT1
;
ORG 0013H
SETB P1.1
MOV R0,200
WAIT: DJNZ R0,WAIT
CLR P1.1
RETI
;
;main program for initialization
;
ORG 30H
MAIN: SETB IT1 ;on negative edge of INT1
MOV IE,#10000100B
WAIT2: SJMP WAIT2
END
```



Interrupt Priorities

- What if **two** interrupt sources interrupt at the **same time**?
- The interrupt with the **highest** PRIORITY gets serviced **first**.
- All interrupts have a power on **default** priority order.
 1. External interrupt 0 (INT0)
 2. Timer interrupt0 (TF0)
 3. External interrupt 1 (INT1)
 4. Timer interrupt1 (TF1)
 5. Serial communication (RI+TI)
- Priority can also be set to “high” or “low” by **IP** reg.

Interrupt Priorities (IP) Register



IP.7: reserved

IP.6: reserved

IP.5: timer 2 interrupt priority bit(8052 only)

IP.4: serial port interrupt priority bit

IP.3: timer 1 interrupt priority bit

IP.2: external interrupt 1 priority bit

IP.1: timer 0 interrupt priority bit

IP.0: external interrupt 0 priority bit

Interrupt Priorities Example

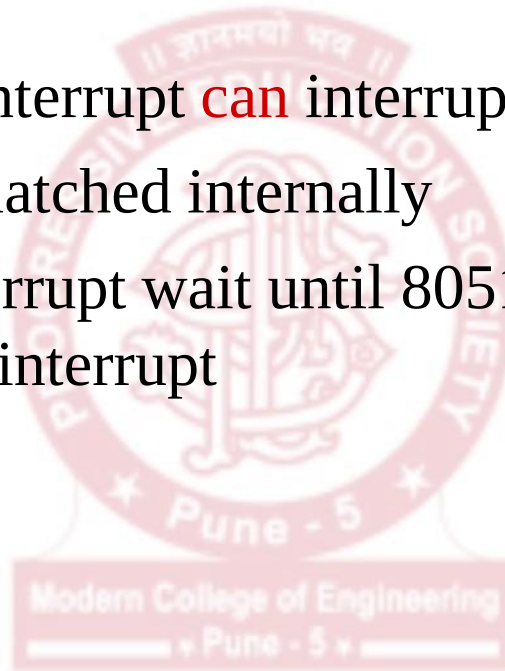


- **MOV IP, #00000100B** or **SETB IP.2** gives priority order
 1. Int1
 2. Int0
 3. Timer0
 4. Timer1
 5. Serial
- **MOV IP, #00001100B** gives priority order
 1. Int1
 2. Timer1
 3. Int0
 4. Timer0
 5. Serial

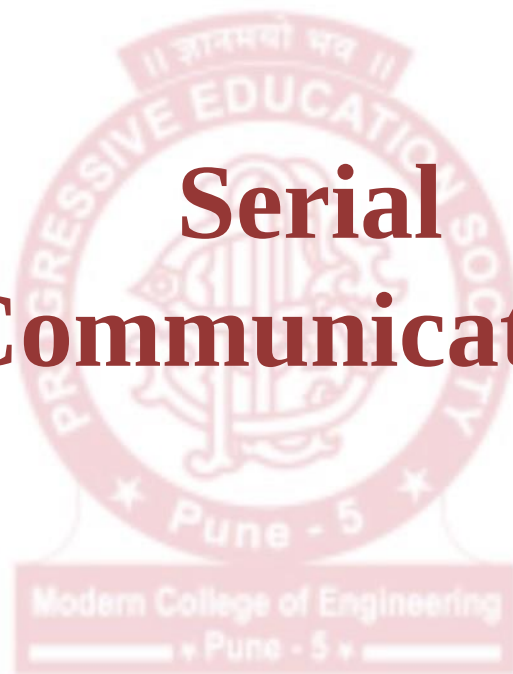
Interrupt inside an interrupt



- ❑ A high-priority interrupt **can** interrupt a low-priority interrupt
- ❑ All interrupt are latched internally
- ❑ Low-priority interrupt wait until 8051 has finished servicing the high-priority interrupt



Serial Communication



Basics of serial communication

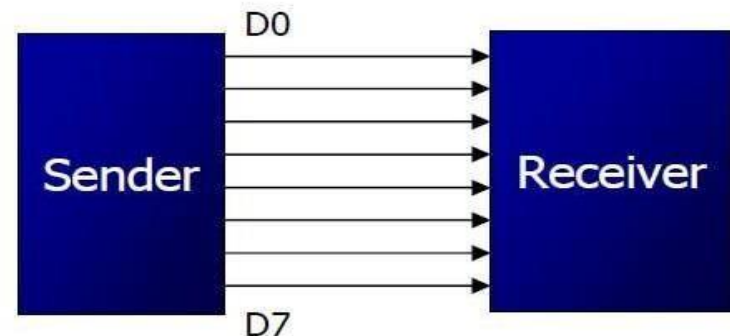
Computers transfer data in two ways:

- ❑ Parallel: Often 8 or more lines (wire conductors) are used to transfer data to a device that is only a few feet away.
- ❑ Parallel: expensive - short distance – fast
- ❑ Serial: To transfer to a device located many meters away, the serial method is used. The data is sent one bit at a time.
- ❑ Serial :cheaper– long (two different cities by modem)-slow

Serial Transfer



Parallel Transfer



Basics of serial communication

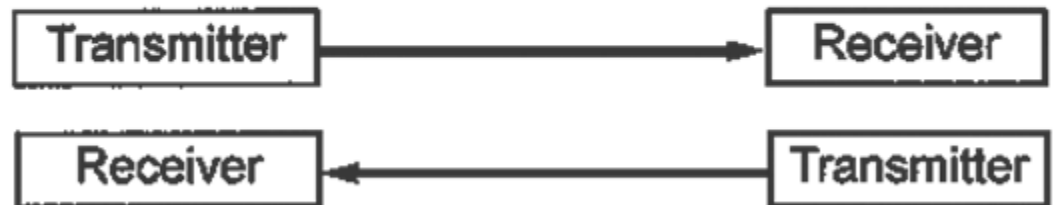
Simplex



Half Duplex

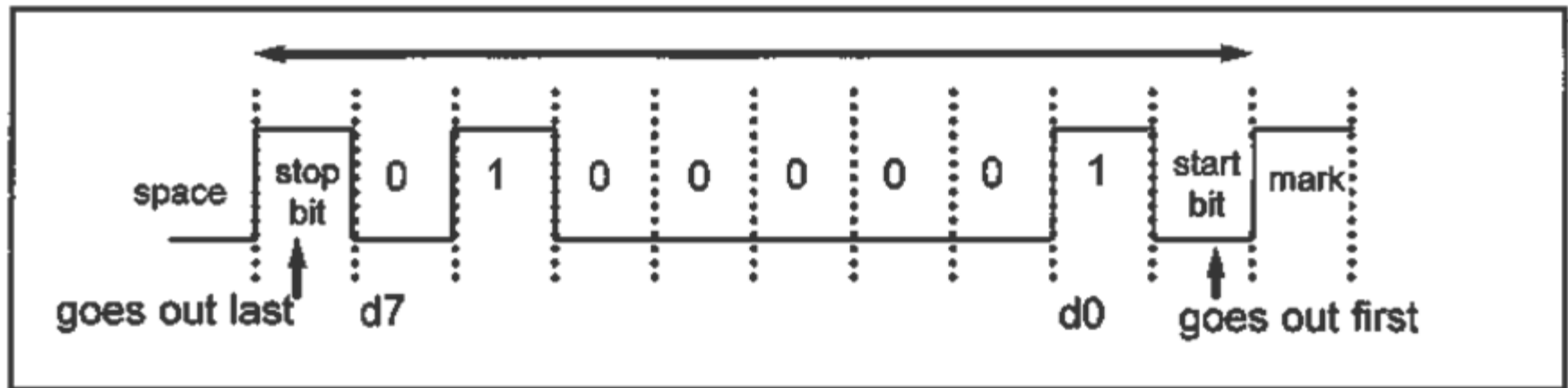


Full Duplex



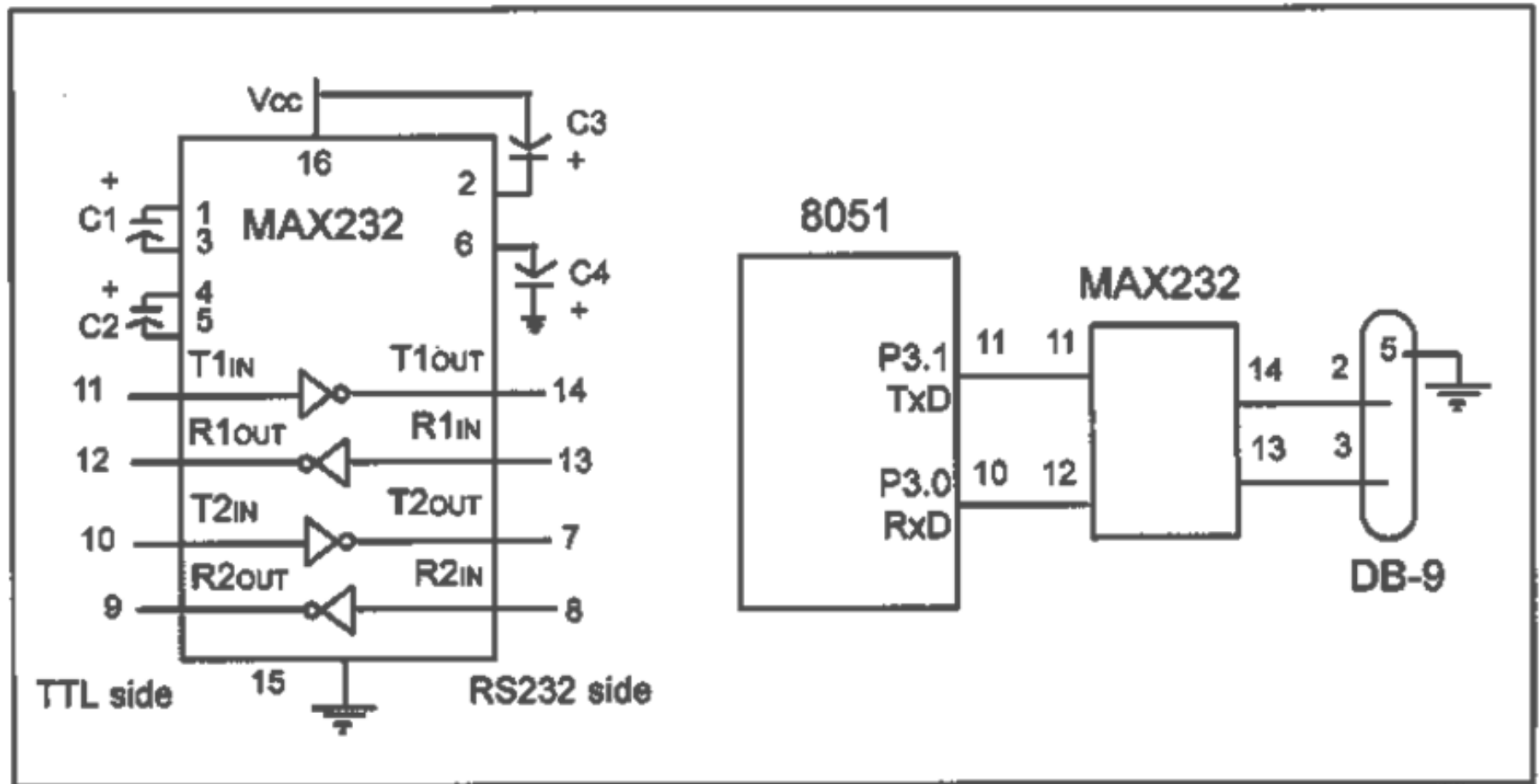
Data framing (Start and stop bits)

- ❑ When there is no transfer the signal is high
- ❑ Transmission begins with a start (low) bit
- ❑ LSB first
- ❑ Finally 1 stop bit (high)
- ❑ Data transfer rate (baud rate) is stated in bps
- ❑ bps: bit per second



Framing ASCII "A" (41H)

8051 interfacing with MAX 232

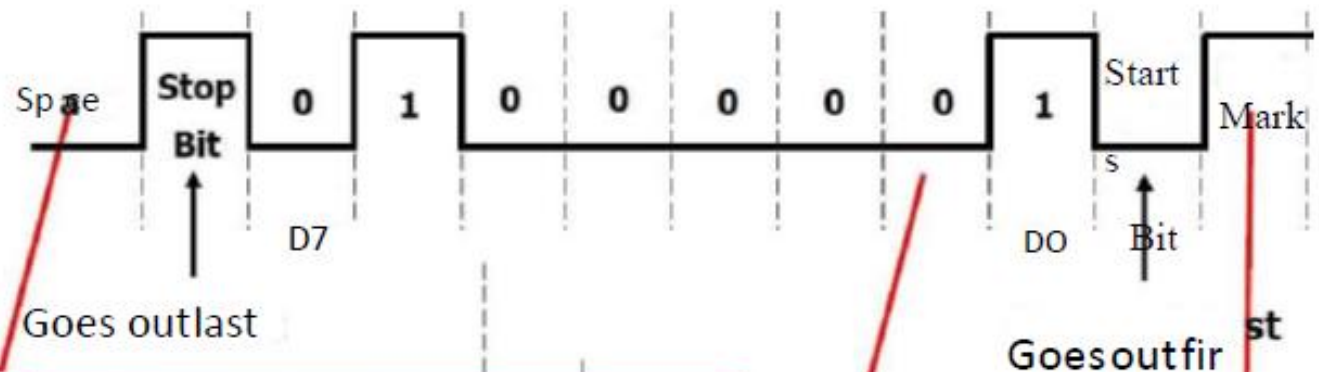


(a) Inside MAX232 and (b) its Connection to the 8051 (Null Modem)

Asynchronous – Start & Stop Bit

- Asynchronous serial data communication is widely used for character-oriented transmissions
 - Each character is placed in between start and stop bits, this is called framing.
 - Block-oriented data transfers use the synchronous method.
- The start bit is always one bit, but the stop bit can be one or two bits
- The start bit is always a 0 (low) and the stop bit(s) is 1 (high)

ASCII character “A”(8 bit binary 01000001)

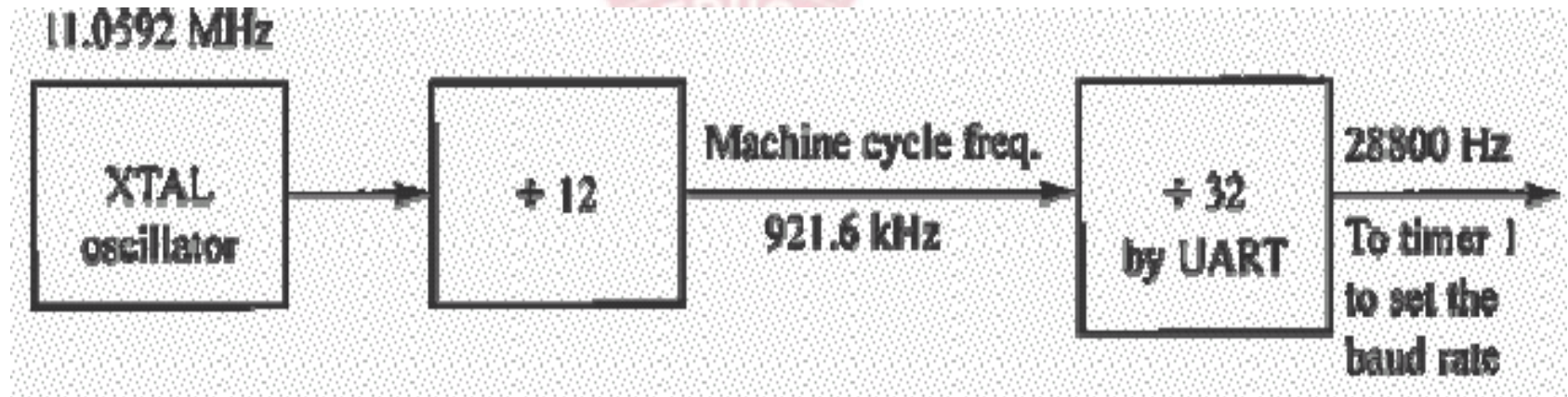


The 0 (low) is referred to as *space*

The transmission begins with a start bit followed by D0, the LSB, then the rest of the bits until MSB (D7), and finally, the one stop bit indicating the end of the character

When there is no transfer, the signal is 1 (high), which is referred to as *mark*

Baud rate in 8051



Baud rate

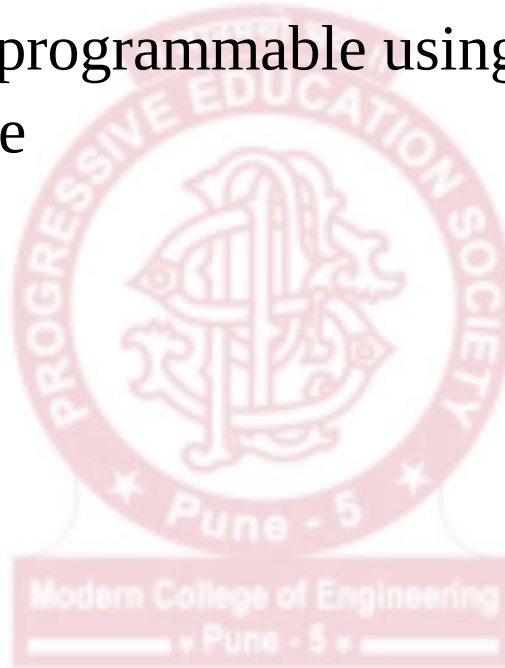
It is the rate at which information is transmitted.

The baud rate of the 8051 must match the baud rate of the PC

Baud rate in 8051 is programmable using timer 1

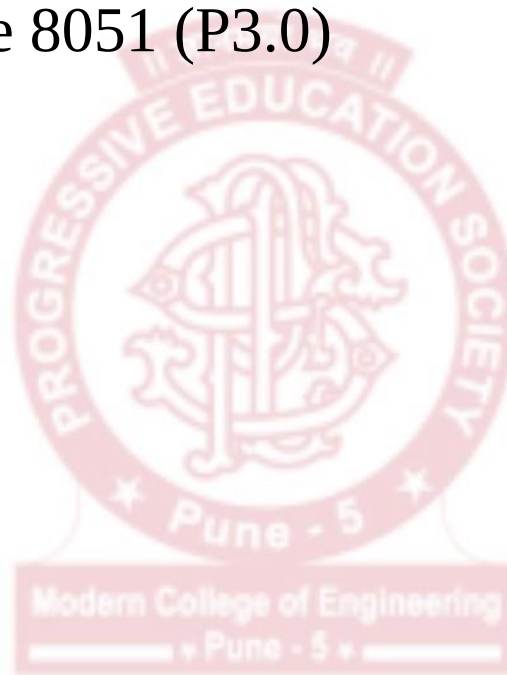
PC standard baud rate

- ❖ 2400
- ❖ 4800
- ❖ 9600
- ❖ 14400
- ❖ 19200
- ❖ 28800
- ❖ 33600
- ❖ 57600



RxD and TxD pins in the 8051

- TxD pin 11 of the 8051 (P3.1)
- RxD pin 10 of the 8051 (P3.0)



SBUF (Serial Buffer) register

- Serial Buffer is an 8 bit register
- For a byte (8 bit) of data to be transferred via the TXD line, it must be placed in the serial buffer register.
- Similarly SBUF register holds the byte of data when it is received by the 8051's RXD line.

MOV SBUF,#'D' ;load SBUF=44H, ASCII for 'D'
MOV SBUF,A ;copy accumulator into SBUF
MOV A,SBUF ;copy SBUF into accumulator

Serial control (SCON) Register

SM0	SM1	SM2	REN	TB8	RB8	TI	RI
-----	-----	-----	-----	-----	-----	----	----

SM0 (SCON.7) : mode specifier

SM1 (SCON.6) : mode specifier

SM2 (SCON.5) : used for multi processor communication

REN (SCON.4) : receive enable (by software enable/disable)

TB8 (SCON.3) : transmit bit 8

RB8 (SCON.2) : receive bit 8

TI (SCON.1) : transmit interrupt flag set by HW clear by SW

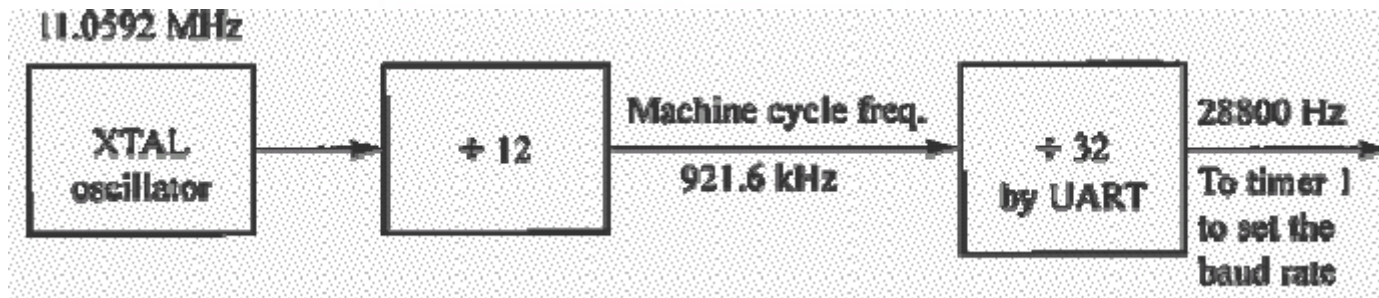
RI (SCON.0) : receive interrupt flag set by HW clear by SW

Mode of operation

SMO	SM1	MODE	Transmission format	Baud rate
0	1	0	shift register, 8 bits	fixed (xtal/12)
0	1	1	8 bit data, 1 stop bit, 1 start bit (10bits)	Variable
1	1	2	9 bit UART	1/32 or 1/64
1	0	3	11 bits(1 start, 1 stop, 8 bit data, 9th bit programmable	Variable

Mode of operation

- Mode 1
 - Ten bits are transmitted (through TxD) or received (through RxD)
 - A start bit (0), 8 data bits (LSB first), and a stop bit (1)
 - On receive, the stop bit goes into RB8 in SCON
 - the baud rate is determined by the Timer 1 overflow rate.
 - Timer1 clock is $1/32$ machine cycle (MC= $1/12$ XTAL)
 - Timer clock can be programmed as $1/16$ of machine cycle
 - Transmission is initiated by any instruction that uses SBUF as a destination register.



Steps in programming serial transfer

- Load value in TMOD register for timer 1 in mode 2
- Load value in TH1 for the desired baud rate.
- Load value in SCON register.
- Load the data to be transferred in SBUF register
- Start the timer
- Monitor the TI bit.
- Clear TI bit



Write a program to transfer a letter 'A' serially at 9600 baud rate continuously

```
#include <reg51.h>
void main (void) {
    SCON = 0x50 // SCON: mode 1, 8-bit UART, enable rcvr
    TMOD = 0x20; // TMOD: timer 1, mode 2, 8-bit reload
    TH1 = -3;    // TH1: reload value for baud @ 12MHz
    TR1 = 1;     // TR1: timer 1 run TI = 1;
    while (1)
    {
        SBUF='A';
        while(TI==0);
        TI=0;
    }
}
```

Write a program to serially transmit MCOE with 2400 baud rate with XTAL 11.0592 MHz

An example of sending a message.
;initialization

```
MOV TMOD,#20H
MOV TH1,#-12
MOV SCON,#52H
```

;begin to trnasmit

```
SETB TR1
```

```
AGAIN1:    MOV A,#'M'
```

```
CALL TRANSS
```

```
MOV A,#'C'
```

```
CALL TRANSS
```

```
MOV A,#'O'
```

```
CALL TRANSS
```

```
MOV A,#'E'
```

```
CALL TRANSS
```

```
SJMP AGAIN1
```

;seial transmitting subroutine

```
TRANSS:    MOV SBUF,A
```

```
AGAIN2:    JNB TI,AGAIN2
```

```
CLR TI
```

```
RET
```

```
END
```

Steps in programming serial receive

- Load value in TMOD register for timer 1 in mode 2
- Load value in TH1 for the desired baud rate.
- Load value in SCON register.
- Start the the timer
- Clear RI flag
- Monitor the RI flag
- When RI flag is raised move received data from SBUF to internal RAM



Write a program to serially receive a byte of data with baud rate 9600 with XTAL is 11.0592 MHz

```
#include <reg51.h>
void main (void) {
    SCON = 0x50 // SCON: mode 1, 8-bit UART, enable rcvr
    TMOD = 0x20; // TMOD: timer 1, mode 2, 8-bit reload
    TH1 = -3;    // TH1: reload value for baud @ 12MHz
    TR1 = 1;    // TR1: timer 1 run TI = 1;
    while (1)
    {
        while(RI==0);
        Mybyte=SBUF;
        P1=Mybyte
        RI=0;
    }
}
```

Baud rate comparison

Table 10-5: Baud Rate Comparison for SMOD = 0 and SMOD = 1

TH1	(Decimal)	(Hex)	SMOD = 0	SMOD = 1
-3		FD	9,600	19,200
-6		FA	4,800	9,600
-12		F4	2,400	4,800
-24		E8	1,200	2,400

Note: XTAL = 11.0592 MHz.



Programming serial interrupt

RI and TI Flags

RI flag is raised when the whole byte of data including stop bit is received by the controller

TI flag is raised when the whole byte in SBUF is transferred. TI flag indicates that SBUF is ready to transfer the next byte.

In polling method we monitored the TI and RI flag where as in Interrupt method we do not monitor the flag bits we check if they are 1

Clearing TI and RI flags

It is necessary in interrupt to clear the flags as there is only one interrupt for receive and transmit. So when interrupt occurs controller doesn't know who interrupted it. Whether it is TI or RI?

Write a program in which 8051 reads data from P1 and writes data to P2 continuously while giving a copy of it to serial COM port. Assume baud rate as 9600 and XTAL is 11.0592 MHz

```
ORG 0000H
    LJMP MAIN
;jump to serial ISR
    ORG 23H
    LJMP ISR
;main program
    ORG 30H
;initialization
MAIN:  MOV P0,#0FFH
       MOV TMOD,#20H
       MOV TH1,#-13
       MOV SCON,#50H
       MOV IE,#90H
;2-begin
       SETB TR1
AGAIN: MOV A, P1
       MOV SBUF,A
       MOV P2,A
       SJMP AGAIN

;ISR for receive from serial to p0
;transmitting to serial from p1
ISR:   JB TI,TRANSM
       MOV A,SBUF
       mov P0,A
       CLR RI
       RETI

TRANSM: MOV A,P1
        MOV SBUF,A
        CLR TI
        RETI
        END
```

Write a program in which 10 bytes of data stored in RAM locations starting from 45h are transferred serially using interrupt. Assume baud rate as 7800 and XTAL as 11.0592 MHz



??????

Power control register (PCON)

MSB				LSB			
SMOD	—	—	—	GF1	GF0	PD	IDL

BIT	SYMBOL	FUNCTION
PCON.7	SMOD	Double Baud rate bit. When set to a 1 and Timer 1 is used to generate baud rate, and the Serial Port is used in modes 1, 2, or 3.
PCON.6	—	Reserved.
PCON.5	—	Reserved.
PCON.4	—	Reserved.
PCON.3	GF1	General-purpose flag bit.
PCON.2	GF0	General-purpose flag bit.
PCON.1	PD	Power-Down bit. Setting this bit activates power-down operation.
PCON.0	IDL	Idle mode bit. Setting this bit activate idle mode operation.

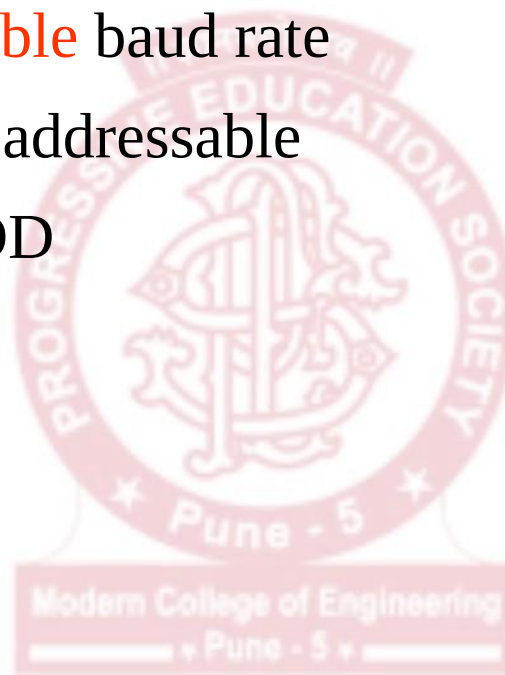
What is SMOD

- ❑ Bit 7 of PCON register
- ❑ If SMOD=1 **double** baud rate
- ❑ PCON is not bit addressable
- ❑ How to set SMOD

Mov a, pcon

Setb acc.7

Mov pcon,a



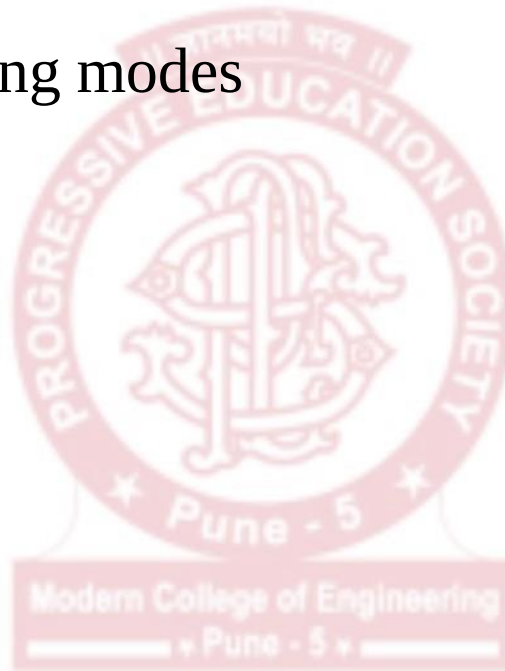
MSB

LSB

SMOD	—	—	—	GF1	GF0	PD	IDL
------	---	---	---	-----	-----	----	-----

Power control

- A standard for applications where power consumption is critical
- two power reducing modes
 - Idle
 - Power down

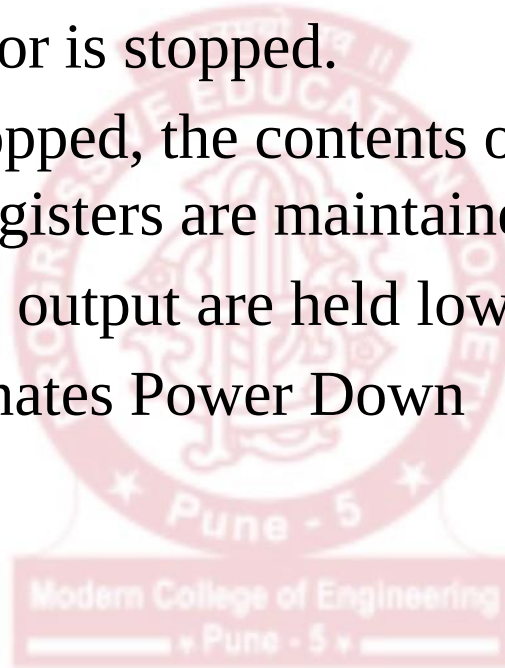


Idle mode

- An instruction that sets PCON.0 causes Idle mode
 - Last instruction executed before going into the Idle mode
 - the internal CPU clock is gated off
 - Interrupt, Timer, and Serial Port functions act normally.
 - All of registers, ports and internal RAM maintain their data during Idle
 - ALE and PSEN hold at logic high levels
- Any interrupt
 - will cause PCON.0 to be cleared by HW (terminate Idle mode)
 - then execute ISR
 - with RETI return and execute next instruction after Idle instruction.
- RST signal clears the IDL bit directly

Power-Down Mode

- An instruction that sets PCON.1 causes power down mode
- Last instruction executed before going into the power down mode
- The on-chip oscillator is stopped.
- All functions are stopped, the contents of the on-chip RAM and Special Function Registers are maintained.
- The ALE and PSEN output are held low
- The **reset** that terminates Power Down



References:

- Textbooks prescribed in the university syllabus.
- As per the presentation slides received during Online Faculty Orientation Workshop under the aegis of BOS (E&TC), SPPU, Pune for syllabus discussions.

